

Modern Tkinter Python for Developers



George W New

About the Authors

George W New was received Bachelor of Computer Science from the American University, and Bachelor of Business Administration from the American University, USA.

Table of Contents

Contents

[About the Authors](#)

[Table of Contents](#)

[Modern Tkinter Python for Developers](#)

[PART 1: Introduction](#)

[CHAPTER 1: Introduction to Tkinter](#)

[What is Tkinter?](#)

[What are Widgets?](#)

[CHAPTER 2: What are Widgets in Tkinter?](#)

[Widgets](#)

[Widget Classes](#)

[Geometry Management](#)

[CHAPTER 3: Hello World in Tkinter](#)

[Creating the Hello World program in Tkinter](#)

[CHAPTER 4: Create First GUI Application using Python-Tkinter](#)

[Getting Started](#)

[Widgets](#)

[Geometry Management](#)

[CHAPTER 5: Python GUI – tkinter](#)

[PART 2: Widgets](#)

[CHAPTER 1: Creating a button in tkinter](#)

[CHAPTER 2: Add style to tkinter button](#)

[CHAPTER 3: Add image on a Tkinter button](#)

[CHAPTER 4: Python Tkinter – Label](#)

[Label Widget](#)

[Python3](#)

[CHAPTER 5: Create LabelFrame and add widgets to it](#)

[RadioButton in Tkinter](#)

[CHAPTER 6: Python Tkinter – Checkbutton Widget](#)

[Checkbutton Widget](#)

[CHAPTER 7: Python Tkinter – Canvas Widget](#)

[Canvas widget](#)

[Some common drawing methods:](#)

[CHAPTER 8: Create different shapes using Canvas class](#)

[CHAPTER 9: Create different type of lines using Canvas class](#)

[CHAPTER 10: Moving objects using Canvas.move\(\) method](#)

[Python3](#)

[CHAPTER 11: Combobox Widget in tkinter](#)

[CHAPTER 12: maxsize\(\) method in Tkinter](#)

[Python3](#)

[Python3](#)

[CHAPTER 13: minsize\(\) method in Tkinter](#)

[CHAPTER 14: resizable\(\) method in Tkinter](#)

[CHAPTER 15: Python Tkinter – Entry Widget](#)

[The Entry Widget](#)

[CHAPTER 16: Tkinter – Read only Entry Widget](#)

[CHAPTER 17: Python Tkinter – Text Widget](#)

[Text Widget](#)

[Python3](#)

[Python3](#)

[CHAPTER 18: Python Tkinter – Message](#)

[Message widget](#)

[CHAPTER 19: Menu widget in Tkinter](#)

[CHAPTER 20: Python Tkinter – Menubutton Widget](#)

[Menubutton widget](#)

[CHAPTER 21: Python Tkinter – SpinBox](#)

[Spinbox widget](#)

[CHAPTER 22: Progressbar widget in Tkinter](#)

[CHAPTER 23: Python Tkinter - Scrollbar](#)

[Scrollbar Widget](#)

[CHAPTER 24: Python Tkinter – ScrolledText Widget](#)

[CHAPTER 25: Python Tkinter – ListBox Widget](#)

[ListBox widget](#)

[Python3](#)

[Python3](#)

[CHAPTER 26: Scrollable ListBox in Python-tkinter](#)

[Adding Scrollbar to ListBox](#)

[CHAPTER 27: Python Tkinter – Frame Widget](#)

[Frame](#)

[CHAPTER 28: Scrollable Frames in Tkinter](#)

[CHAPTER 29: Make a proper double scrollbar frame in Tkinter](#)

[CHAPTER 30: Python Tkinter – Scale Widget](#)

[Scale widget](#)

[CHAPTER 31: Hierarchical treeview in Python GUI application](#)

[Treeview widgets](#)

[CHAPTER 32: Tkinter Treeview scrollbar](#)

[Treeview scrollbar](#)

[PART 3: Toplevel Widgets](#)

[CHAPTER 1: Python Tkinter – Toplevel Widget](#)

[Toplevel widget](#)

[CHAPTER 2: askopenfile\(\) function in Tkinter](#)

[CHAPTER 3: asksaveasfile\(\) function in Tkinter](#)

[CHAPTER 4: Tkinter askquestion Dialog](#)

[Askquestion\(\)](#)

[CHAPTER 5: Python Tkinter – MessageBox Widget](#)

[MessageBox Widget](#)

[CHAPTER 6: Create a Yes/No Message Box in Python using tkinter](#)

[CHAPTER 7: Change the size of MessageBox – Tkinter](#)

[CHAPTER 8: Different messages in Tkinter](#)

[CHAPTER 9: Change Icon for Tkinter MessageBox](#)

[Default icon of MessageBox](#)

[CHAPTER 10: Tkinter Choose color Dialog](#)

[Creating choose color dialog box using Tkinter](#)

[askcolor\(\)](#)

[CHAPTER 11: Popup Menu in Tkinter](#)

[PART 4: Geometry Management=====](#)

[CHAPTER 1: place\(\) method in Tkinter](#)

[Python3](#)

[Python3](#)

[CHAPTER 2: grid\(\) method in Tkinter](#)

[CHAPTER 3: grid_location\(\) and grid_size\(\) method](#)

[grid_location\(\) method –](#)

[grid_size\(\) method –](#)

[pack\(\) method in Tkinter](#)

[CHAPTER 4: forget_pack\(\) and forget_grid\(\) method in Tkinter](#)

[forget_pack\(\) method –](#)

[forget_grid\(\) method –](#)

[CHAPTER 5: PanedWindow Widget in Tkinter](#)

[CHAPTER 6: Geometry Method in Python Tkinter](#)

[Tkinter window without using geometry method.](#)

[Examples of Tkinter Geometry Method in Python](#)

[Using the geometry method to set the dimensions and positions of the Tkinter window.](#)

[CHAPTER 7: Setting the position of TKinter labels](#)

[PART 5: Binding Functions](#)

[CHAPTER 1: Binding function in Tkinter](#)

[Python3](#)

[Python3](#)

[Python3](#)

[CHAPTER 2: Binding Function with double click with Tkinter ListBox](#)

[CHAPTER 3: Right Click menu using Tkinter](#)

[CHAPTER 4: Reading Images With Python – Tkinter](#)

[Reading Images With Tkinter](#)

[CHAPTER 5: iconphoto\(\) method in Tkinter](#)

[CHAPTER 6: Loading Images in Tkinter using PIL](#)

[PART 6: Applications and Projects](#)

[CHAPTER 1: Simple GUI calculator using Tkinter](#)

[CHAPTER 2: Create a stopwatch using python](#)

[CHAPTER 3: Real time currency converter using Tkinter](#)

[Python3](#)

[CHAPTER 4: Create Table Using Tkinter](#)

[Creating Tables Using Tkinter](#)

[CHAPTER 5: GUI Calendar using Tkinter](#)

[Python3](#)

[CHAPTER 6: File Explorer in Python using Tkinter](#)

[Conclusion](#)

Modern Tkinter Python for Developers

PART 1: Introduction

CHAPTER 1: Introduction to Tkinter

Graphical User Interface(GUI) is a form of user interface which allows users to interact with computers through visual indicators using items such as icons, menus, windows, etc. It has advantages over the Command Line Interface(CLI) where users interact with computers by writing commands using keyboard only and whose usage is more difficult than GUI.

What is Tkinter?

Tkinter is the inbuilt python module that is used to create GUI applications. It is one of the most commonly used modules for creating GUI applications in Python as it is simple and easy to work with. You don't need to worry about the installation of the Tkinter module separately as it comes with Python already. It gives an object-oriented interface to the Tk GUI toolkit. Some other Python Libraries available for creating our own GUI applications are

- Kivy
- Python Qt
- wxPython

Among all Tkinter is most widely used

Here are some common use cases for Tkinter:

1. Creating windows and dialog boxes: Tkinter can be used to create windows and dialog boxes that allow users to interact with your

program. These can be used to display information, gather input, or present options to the user.

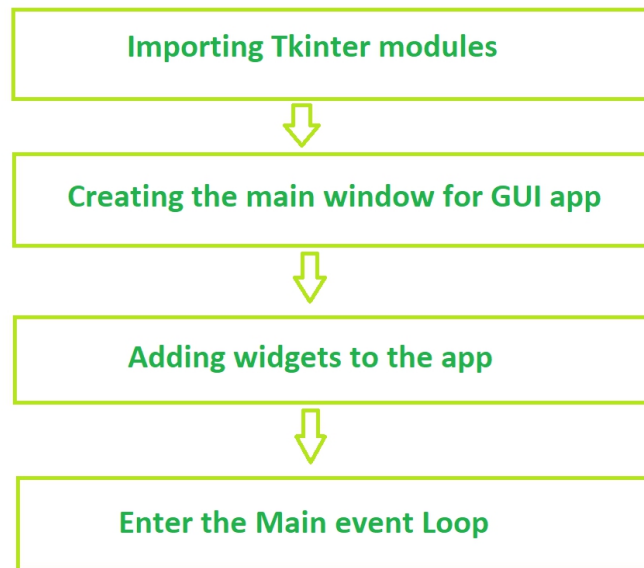
2. Building a GUI for a desktop application: Tkinter can be used to create the interface for a desktop application, including buttons, menus, and other interactive elements.
3. Adding a GUI to a command-line program: Tkinter can be used to add a GUI to a command-line program, making it easier for users to interact with the program and input arguments.
4. Creating custom widgets: Tkinter includes a variety of built-in widgets, such as buttons, labels, and text boxes, but it also allows you to create your own custom widgets.
5. Prototyping a GUI: Tkinter can be used to quickly prototype a GUI, allowing you to test and iterate on different design ideas before committing to a final implementation.

In summary, Tkinter is a useful tool for creating a wide variety of graphical user interfaces, including windows, dialog boxes, and custom widgets. It is particularly well-suited for building desktop applications and adding a GUI to command-line programs.

What are Widgets?

Widgets in Tkinter are the elements of GUI application which provides various controls (such as Labels, Buttons, ComboBoxes, CheckBoxes, MenuBars, RadioButtons and many more) to users to interact with the

application. **Fundamental structure of tkinter program**



Basic Tkinter Widgets:

Widgets	Description
Label	It is used to display text or image on the screen
Button	It is used to add buttons to your application
Canvas	It is used to draw pictures and others layouts like texts, graphics etc.
ComboBox	It contains a down arrow to select from list of available options
CheckButton	It displays a number of options to the user as toggle buttons from which user can select any number of options.
RadioButton	It is used to implement one-of-many selection as it allows only one option to be selected
Entry	It is used to input single line text entry from user
Frame	It is used as container to hold and organize the widgets
Message	It works same as that of label and refers to multi-line and non-editable text
Scale	It is used to provide a graphical slider which allows to select any value from that scale

Scrollbar	It is used to scroll down the contents. It provides a slide controller.
SpinBox	It is allows user to select from given set of values
Text	It allows user to edit multiline text and format the way it has to be displayed
Menu	It is used to create all kinds of menu used by an application

Example

- PYTHON

```

from tkinter import *

from tkinter.ttk import *

# writing code needs to
# create the main window of
# the application creating
# main window object named root

root = Tk()

# giving title to the main window
root.title("First_Program")

```

```
# Label is what output will be  
# show on the window  
  
label = Label(root, text ="Hello World !").pack()  
  
# calling mainloop method which is used  
# when your application is ready to run  
# and it tells the code to keep displaying  
root.mainloop()
```

Output

 First_Program

- □ ×

Hello World!

CHAPTER 2: What are Widgets in Tkinter?

[Tkinter](#) is Python's standard GUI (Graphical User Interface) package. **tkinter** provides us with a variety of common GUI elements which we can use to build out interface – such as buttons, menus and various kind of entry fields and display areas. We call these elements **Widgets**.

Widgets

In general, **Widget** is an element of Graphical User Interface (GUI) that displays/illustrates information or gives a way for the user to interact with the OS. In **Tkinter**, **Widgets** are objects ; instances of classes that represent buttons, frames, and so on.

Each separate widget is a Python object. When creating a widget, you must pass its parent as a parameter to the widget creation function. The only exception is the “root” window, which is the top-level window that will contain everything else and it does not have a parent.

Example :

- Python

```
from tkinter import *  
  
# create root window  
  
root = Tk()  
  
# frame inside root window
```

```
frame = Frame(root)

# geometry method

frame.pack()

# button inside frame which is

# inside root

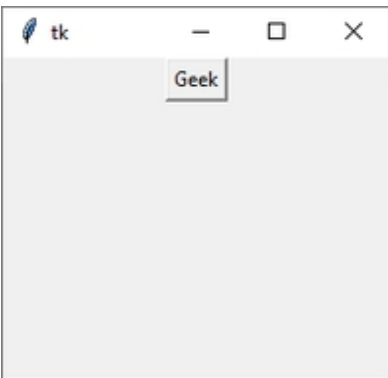
button = Button(frame, text ='Geek')

button.pack()

# Tkinter event loop

root.mainloop()
```

Output :



Widget Classes

Tkinter supports the below mentioned core widgets –

Widgets	Description
---------	-------------

Label	It is used to display text or image on the screen
Button	It is used to add buttons to your application
Canvas	It is used to draw pictures and others layouts like texts, graphics etc.
ComboBox	It contains a down arrow to select from list of available options
CheckBoxButton	It displays a number of options to the user as toggle buttons from which user can select any number of options.
Radio Button	It is used to implement one-of-many selection as it allows only one option to be selected
Entry	It is used to input single line text entry from user
Frame	It is used as container to hold and organize the widgets
Message	It works same as that of label and refers to multi-line and non-editable text
Scale	It is used to provide a graphical slider which allows to select any value from that scale
Scrollbar	It is used to scroll down the contents. It provides a slide controller.
SpinBox	It is allows user to select from given set of values
Text	It allows user to edit multiline text and format the way it has to be displayed
Menu	It is used to create all kinds of menu used by an application

Geometry Management

Creating a new widget doesn't mean that it will appear on the screen. To display it, we need to call a special method: either **grid**, **pack**(example above), or **place**.

Metho d	Description
pack ()	The Pack geometry manager packs widgets in rows or columns.
grid ()	The Grid geometry manager puts the widgets in a 2-dimensional table. The master widget is split into a number of rows and columns, and each “cell” in the resulting table can hold a widget.
place ()	The Place geometry manager is the simplest of the three general geometry managers provided in Tkinter. It allows you explicitly set the position and size of a window, either in absolute terms, or relative to another window.

CHAPTER 3: Hello World in Tkinter

[Tkinter](#) is the Python *GUI framework* that is build into the Python standard library. Out of all the GUI methods, tkinter is the most commonly used method as it provides the fastest and the easiest way to create the GUI application.

Creating the Hello World program in Tkinter

Lets start with the ‘hello world’ tutorial. Here is the explanation for the first program in tkinter:

- `from tkinter import *`

In Python3 firstly we import all the classes, functions and variables from the tkinter package.

- `root=Tk()`

Now we create a root widget, by calling the `Tk()`. This automatically creates a graphical window with the title bar, minimize, maximize and close buttons. This handle allows us to put the contents in the window and reconfigure it as necessary.

- `a = Label(root, text="Hello, world!")`

Now we create a Label widget as a child to the root window. Here root is the parent of our label widget. We set the default text to “Hello, World!”

Note: This gets displayed in the window. A **Label** widget can display either text or an icon or other image.

- `a.pack()`

Next, we call the **pack()** method on this widget. This tells it to size itself to fit the given text, and make itself visible. It just tells the geometry manager to put widgets in the same row or column. It’s usually the easiest to use if you just want one or a few widgets to appear.

- `root.mainloop()`

The application window does not appear before you enter the main loop. This method says to take all the widgets and objects we created, render them on our screen, and respond to any interactions. The program stays in the loop until we close the window.

Below is the implementation.

```
# Python tkinter hello world program

from tkinter import *

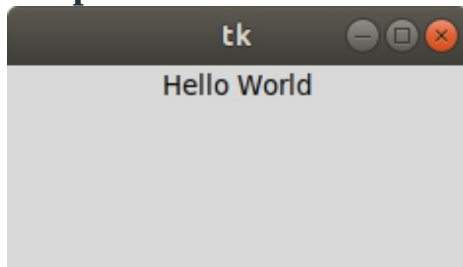
root = Tk()

a = Label(root, text ="Hello World")

a.pack()

root.mainloop()
```

Output:



CHAPTER 4: Create First GUI Application using Python-Tkinter

[Tkinter](#) is a Python Package for creating GUI applications. Python has a lot of GUI frameworks, but Tkinter is the only framework that's built into the Python standard library. Tkinter has several strengths; it's cross-platform, so the same code works on Windows, macOS, and Linux. Tkinter is lightweight

and relatively painless to use compared to other frameworks. This makes it a compelling choice for building GUI applications in Python, especially for applications where a modern shine is unnecessary, and the top priority is to build something that's functional and cross-platform quickly.

Here are some common use cases for Tkinter in more detail:

1. Creating windows and dialog boxes: Tkinter can be used to create windows and dialog boxes that allow users to interact with your program. These can be used to display information, gather input, or present options to the user. To create a window or dialog box, you can use the **Tk()** function to create a root window, and then use functions like **Label**, **Button**, and **Entry** to add widgets to the window.
2. Building a GUI for a desktop application: Tkinter can be used to create the interface for a desktop application, including buttons, menus, and other interactive elements. To build a GUI for a desktop application, you can use functions like **Menu**, **Checkbutton**, and **RadioButton** to create menus and interactive elements, and use layout managers like **pack** and **grid** to arrange the widgets on the window.
3. Adding a GUI to a command-line program: Tkinter can be used to add a GUI to a command-line program, making it easier for users to interact with the program and input arguments. To add a GUI to a command-line program, you can use functions like **Entry** and **Button** to create input fields and buttons, and use event handlers like **command** and **bind** to handle user input.
4. Creating custom widgets: Tkinter includes a variety of built-in widgets, such as buttons, labels, and text boxes, but it also allows you to create your own custom widgets. To create a custom widget, you can define a class that inherits from the **Widget** class and overrides its methods to define the behavior and appearance of the widget.
5. Prototyping a GUI: Tkinter can be used to quickly prototype a GUI, allowing you to test and iterate on different design ideas before committing to a final implementation. To prototype a GUI with Tkinter, you can use the **Tk()** function to create a root window, and then use functions like **Label**, **Button**, and **Entry** to add widgets to the window and test different layout and design ideas.

Tkinter Alternatives

There are a number of libraries that are similar to Tkinter and can be used for creating graphical user interfaces (GUIs) in Python. Some examples include:

1. **PyQt**: PyQt is a library that allows you to create GUI applications using the Qt framework. It is a comprehensive library with a large number of widgets and features.
2. **wxPython**: wxPython is a library that allows you to create GUI applications using the wxWidgets framework. It includes a wide range of widgets and is cross-platform, meaning it can run on multiple operating systems.
3. **PyGTK**: PyGTK is a library that allows you to create GUI applications using the GTK+ framework. It is a cross-platform library with a wide range of widgets and features.
4. **Kivy**: Kivy is a library that allows you to create GUI applications using a modern, responsive design. It is particularly well-suited for building mobile apps and games.
5. **PyForms**: PyForms is a library that allows you to create GUI applications using a simple, declarative syntax. It is designed to be easy to use and has a small footprint.

In summary, there are several libraries available for creating GUI applications in Python, each with its own set of features and capabilities. Tkinter is a popular choice, but you may want to consider other options depending on your specific needs and requirements.

To understand Tkinter better, we will create a simple GUI.

Getting Started

1. Import tkinter package and all of its modules.
2. Create a root window. Give the root window a title(using **title()**) and dimension(using **geometry()**). All other widgets will be inside the root window.
3. Use **mainloop()** to call the endless loop of the window. If you forget to call this nothing will appear to the user. The window will wait for any user interaction till we close it.

Example:

- Python3

```
# Import Module

from tkinter import *

# create root window

root = Tk()

# root window title and dimension

root.title("Welcome to GeekForGeeks")

# Set geometry (widthxheight)

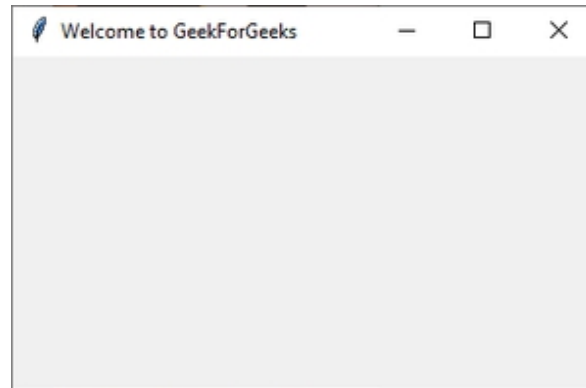
root.geometry('350x200')

# all widgets will be here

# Execute Tkinter

root.mainloop()
```

Output:



Root Window

4. We'll add a label using the Label Class and change its text configuration as desired. The **grid()** function is a geometry manager which keeps the label in the desired location inside the window. If no parameters are mentioned by default it will place it in the empty cell; that is 0,0 as that is the first location.

Example:

- Python3

```
# Import Module

from tkinter import *

# create root window

root = Tk()

# root window title and dimension

root.title("Welcome to GeekForGeeks")

# Set geometry(widthxheight)

root.geometry('350x200')
```

```
#adding a label to the root window

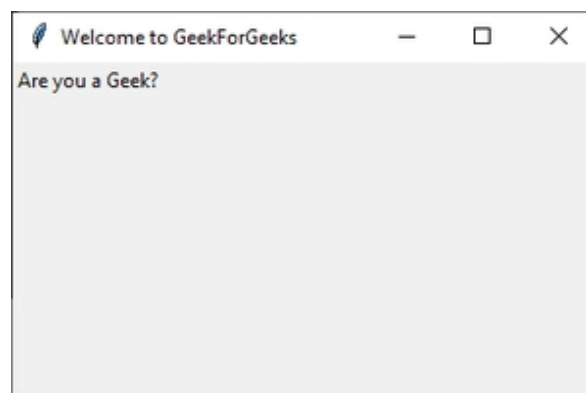
lbl = Label(root, text = "Are you a Geek?")

lbl.grid()

# Execute Tkinter

root.mainloop()
```

Output:



Label inside root window

5. Now add a button to the root window. Changing the button configurations gives us a lot of options. In this example we will make the button display a text once it is clicked and also change the color of the text inside the button.

Example:

- Python3

```
# Import Module
```



```
from tkinter import *

# create root window

root = Tk()

# root window title and dimension
root.title("Welcome to GeekForGeeks")

# Set geometry(widthxheight)
root.geometry('350x200')

# adding a label to the root window

lbl = Label(root, text = "Are you a Geek?")

lbl.grid()

# function to display text when
# button is clicked

def clicked():

    lbl.configure(text = "I just got clicked")

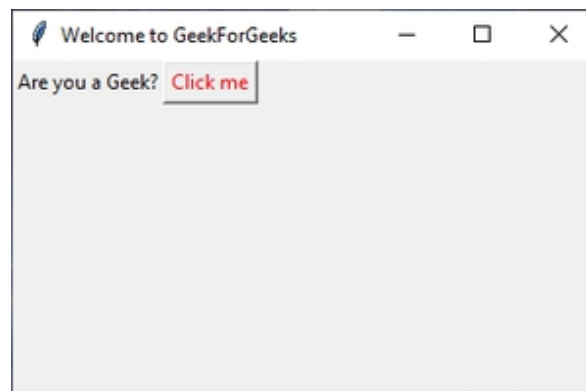
# button widget with red color text
# inside

btn = Button(root, text = "Click me" ,

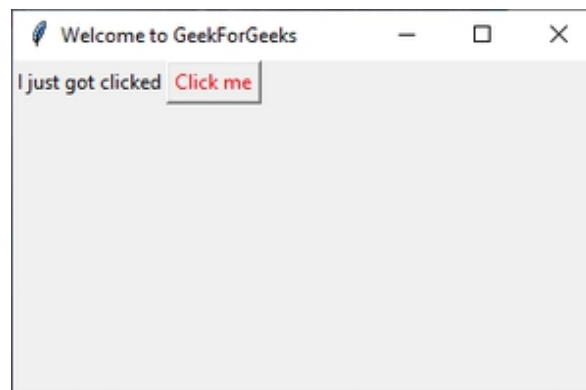
              fg = "red", command=clicked)
```

```
# set Button grid  
  
btn.grid(column=1, row=0)  
  
# Execute Tkinter  
  
root.mainloop()
```

Output:



Button added



After clicking "Click me"

6. Using the **Entry()** class we will create a text box for user input. To display the user input text, we'll make changes to the function **clicked()**. We can get the user entered text using the **get()** function. When the **Button** after entering

of the text, a default text concatenated with the user text. Also change button grid location to column 2 as **Entry()** will be column 1.

Example:

- Python3

```
# Import Module

from tkinter import *

# create root window

root = Tk()

# root window title and dimension
root.title("Welcome to GeekForGeeks")

# Set geometry(widthxheight)
root.geometry('350x200')

# adding a label to the root window

lbl = Label(root, text = "Are you a Geek?")

lbl.grid()

# adding Entry Field

txt = Entry(root, width=10)

txt.grid(column =1, row =0)
```

```
# function to display user text when
# button is clicked

def clicked():

    res = "You wrote" + txt.get()

    lbl.configure(text = res)

# button widget with red color text inside

btn = Button(root, text = "Click me" ,

             fg = "red", command=clicked)

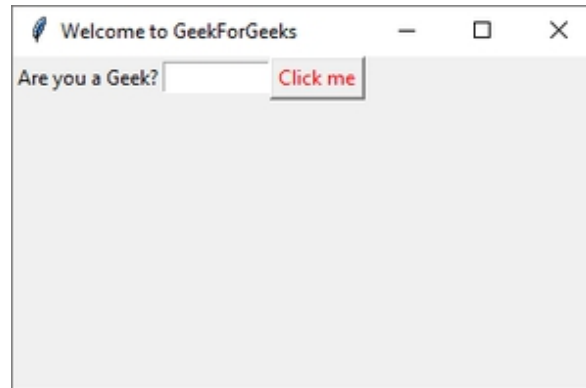
# Set Button Grid

btn.grid(column=2, row=0)

# Execute Tkinter

root.mainloop()
```

Output:



Entry Widget at column 2 row 0



Displaying user input text.

7. To add a menu bar, you can use **Menu** class. First, we create a menu, then we add our first label, and finally, we assign the menu to our window. We can add menu items under any menu by using **add_cascade()**.

Example:

- Python3

```
# Import Module
```

```
from tkinter import *
```

```
# create root window

root = Tk()


# root window title and dimension

root.title("Welcome to GeekForGeeks")

# Set geometry(widthxheight)

root.geometry('350x200')


# adding menu bar in root window

# new item in menu bar labelled as 'New'

# adding more items in the menu bar

menu = Menu(root)

item = Menu(menu)

item.add_command(label='New')

menu.add_cascade(label='File', menu=item)

root.config(menu=menu)


# adding a label to the root window

lbl = Label(root, text = "Are you a Geek?")

lbl.grid()


# adding Entry Field

txt = Entry(root, width=10)
```

```
txt.grid(column =1, row =0)

# function to display user text when
# button is clicked

def clicked():

    res = "You wrote" + txt.get()

    lbl.configure(text = res)

# button widget with red color text inside

btn = Button(root, text = "Click me" ,

              fg = "red", command=clicked)

# Set Button Grid

btn.grid(column=2, row=0)

# Execute Tkinter

root.mainloop()
```

Output :



Menu bar

This simple GUI covers the basics of Tkinter package. Similarly, you can add more widgets and change their configurations as desired.

Widgets

Tkinter provides various controls, such as buttons, labels and text boxes used in a GUI application. These controls are commonly called **Widgets**. The list of commonly used **Widgets** are mentioned below –

S N o .	Widget	Description
1	Label	The Label widget is used to provide a single-line caption for other widgets. It can also contain images.
2	Button	The Button widget is used to display buttons in your application.
3	Entry	The Entry widget is used to display a single-line text

		field for accepting values from a user.
4	Menu	The Menu widget is used to provide various commands to a user. These commands are contained inside Menubutton.
5	Canvas	The Canvas widget is used to draw shapes, such as lines, ovals, polygons and rectangles, in your application.
6	Checkbutton	The Checkbutton widget is used to display a number of options as checkboxes. The user can select multiple options at a time.
7	Frame	The Frame widget is used as a container widget to organize other widgets.
8	Listbox	The Listbox widget is used to provide a list of options to a user.
9	Menubutton	The Menubutton widget is used to display menus in your application.
10	Message	The Message widget is used to display multiline text fields for accepting values from a user.
11	Radiobutton	The Radiobutton widget is used to display a number of options as radio buttons. The user can select only one option at a time.
1	Scale	The Scale widget is used to provide a slider widget.

2		
1 3	Scrollbar	The Scrollbar widget is used to add scrolling capability to various widgets, such as list boxes.
1 4	Text	The Text widget is used to display text in multiple lines.
1 5	Toplevel	The Toplevel widget is used to provide a separate window container.
1 6	LabelFrame	A labelframe is a simple container widget. Its primary purpose is to act as a spacer or container for complex window layouts.
1 7	tkMessageBox	This module is used to display message boxes in your applications.
1 8	Spinbox	The Spinbox widget is a variant of the standard Tkinter Entry widget, which can be used to select from a fixed number of values.
1 9	PanedWindow	A PanedWindow is a container widget that may contain any number of panes, arranged horizontally or vertically.

Geometry Management

All Tkinter widgets have access to specific geometry management methods, which have the purpose of organizing widgets throughout the parent widget area. Tkinter exposes the following geometry manager classes: pack, grid, and place. Their description is mentioned below –

S N o.	Widget	Description
1	pack()	This geometry manager organizes widgets in blocks before placing them in the parent widget.
2	grid()	This geometry manager organizes widgets in a table-like structure in the parent widget.
3	place()	This geometry manager organizes widgets by placing them in a specific position in the parent widget.

CHAPTER 5: Python GUI – tkinter

Python offers multiple options for developing GUI (Graphical User Interface). Out of all the GUI methods, tkinter is the most commonly used method. It is a standard Python interface to the Tk GUI toolkit shipped with Python. Python tkinter is the fastest and easiest way to create GUI applications. Creating a GUI using tkinter is an easy task.

To create a tkinter Python app:

1. Importing the module – tkinter
2. Create the main window (container)
3. Add any number of widgets to the main window
4. Apply the event Trigger on the widgets.

Importing a tkinter is the same as importing any other module in the [Python](#) code. Note that the name of the module in Python 2.x is 'Tkinter' and in Python 3.x it is 'tkinter'.

```
import tkinter
```

There are two main methods used which the user needs to remember while creating the Python application with GUI.

1. **Tk(screenName=None, baseName=None, className='Tk', useTk=1):** To create a main window, tkinter offers a method 'Tk(screenName=None, baseName=None, className='Tk', useTk=1)'. To change the name of the window, you can change the className to the desired one. The basic code used to create the main window of the application is:

m=tkinter.Tk() where m is the name of the main window object

2. **mainloop():** There is a method known by the name mainloop() is used when your application is ready to run. mainloop() is an infinite loop used to run the application, wait for an event to occur and process the event as long as the window is not closed.

```
m.mainloop()
```

- Python

```
import tkinter
```

```
m = tkinter.Tk()
```

```
'''
```

```
widgets are added here
```

```
'''
```

```
m.mainloop()
```

Tkinter also offers access to the geometric configuration of the widgets which can organize the widgets in the parent windows. There are mainly three geometry manager classes class.

1. **pack() method:**It organizes the widgets in blocks before placing in the parent widget.
2. **grid() method:**It organizes the widgets in grid (table-like structure) before placing in the parent widget.
3. **place() method:**It organizes the widgets by placing them on specific positions directed by the programmer.

There are a number of widgets which you can put in your tkinter application. Some of the major widgets are explained below:

1. **Button:**To add a button in your application, this widget is used. The general syntax is:

```
w=Button(master, option=value)
```

master is the parameter used to represent the parent window. There are number of options which are used to change the format of the Buttons. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **activebackground:** to set the background color when button is under the cursor.
- **activeforeground:** to set the foreground color when button is under the cursor.
- **bg:** to set the normal background color.
- **command:** to call a function.
- **font:** to set the font on the button label.

- **image:** to set the image on the button.
- **width:** to set the width of the button.
- **height:** to set the height of the button.

- Python

```
import tkinter as tk

r = tk.Tk()

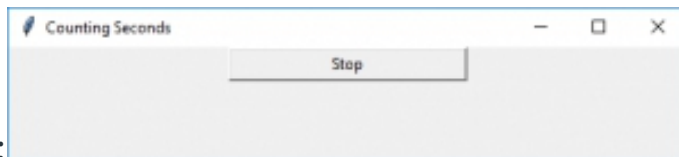
r.title('Counting Seconds')

button = tk.Button(r, text='Stop', width=25, command=r.destroy)

button.pack()

r.mainloop()
```

Output:



Canvas: It is used to draw pictures and other complex layout like graphics, text and widgets. The general syntax is:

```
w = Canvas(master, option=value)
```

master is the parameter used to represent the parent window.

There are number of options which are used to change the format of the widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **bd:** to set the border width in pixels.

- **bg:** to set the normal background color.
- **cursor:** to set the cursor used in the canvas.
- **highlightcolor:** to set the color shown in the focus highlight.
- **width:** to set the width of the widget.
- **height:** to set the height of the widget.

- Python

```
from tkinter import *

master = Tk()

w = Canvas(master, width=40, height=60)

w.pack()

canvas_height=20

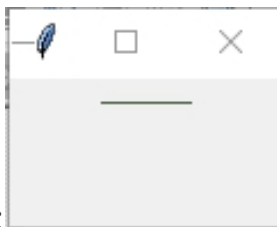
canvas_width=200

y = int(canvas_height / 2)

w.create_line(0, y, canvas_width, y )

mainloop()
```

Output:



CheckBox: To select any number of options by displaying a number of options to a user as toggle buttons. The general syntax is:

```
w = CheckButton(master, option=value)
```

There are number of options which are used to change the format of this widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **Title:** To set the title of the widget.
- **activebackground:** to set the background color when widget is under the cursor.
- **activeforeground:** to set the foreground color when widget is under the cursor.
- **bg:** to set the normal background color.
- **command:** to call a function.
- **font:** to set the font on the button label.
- **image:** to set the image on the widget.

- Python

```
from tkinter import *

master = Tk()

var1 = IntVar()

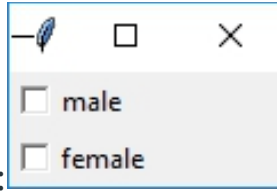
Checkbutton(master, text='male', variable=var1).grid(row=0, sticky=W)

var2 = IntVar()

Checkbutton(master, text='female', variable=var2).grid(row=1, sticky=W)

mainloop()
```


Output:



Entry: It is used to input the single line text entry from the user.. For multi-line text input, Text widget is used. The general syntax is:

`w=Entry(master, option=value)`

master is the parameter used to represent the parent window. There are number of options which are used to change the format of the widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **bd:** to set the border width in pixels.
- **bg:** to set the normal background color.
- **cursor:** to set the cursor used.
- **command:** to call a function.
- **highlightcolor:** to set the color shown in the focus highlight.
- **width:** to set the width of the button.
- **height:** to set the height of the button.

- Python

```
from tkinter import *  
  
master = Tk()  
  
Label(master, text='First Name').grid(row=0)  
  
Label(master, text='Last Name').grid(row=1)
```

```
e1 = Entry(master)

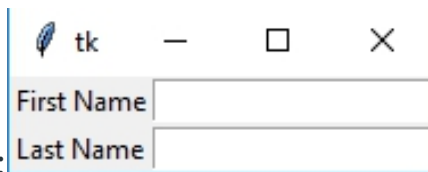
e2 = Entry(master)

e1.grid(row=0, column=1)

e2.grid(row=1, column=1)

mainloop()
```

Output:



Frame: It acts as a container to hold the widgets. It is used for grouping and organizing the widgets. The general syntax is:

```
w = Frame(master, option=value)
```

master is the parameter used to represent the parent window.

There are number of options which are used to change the format of the widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **highlightcolor:** To set the color of the focus highlight when widget has to be focused.
- **bd:** to set the border width in pixels.
- **bg:** to set the normal background color.
- **cursor:** to set the cursor used.
- **width:** to set the width of the widget.
- **height:** to set the height of the widget.

- Python

```
from tkinter import *

root = Tk()

frame = Frame(root)

frame.pack()

bottomframe = Frame(root)

bottomframe.pack( side = BOTTOM )

redbutton = Button(frame, text = 'Red', fg='red')

redbutton.pack( side = LEFT)

greenbutton = Button(frame, text = 'Brown', fg='brown')

greenbutton.pack( side = LEFT )

bluebutton = Button(frame, text='Blue', fg='blue')

bluebutton.pack( side = LEFT )

blackbutton = Button(bottomframe, text='Black', fg='black')

blackbutton.pack( side = BOTTOM)

root.mainloop()
```

Output:



Label: It refers to the display box where you can put any text or image which can be updated any time as per the code. The general syntax is:

```
w=Label(master, option=value)
```

master is the parameter used to represent the parent window.

There are number of options which are used to change the format of the widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **bg:** to set the normal background color.
- **bg** to set the normal background color.
- **command:** to call a function.
- **font:** to set the font on the button label.
- **image:** to set the image on the button.
- **width:** to set the width of the button.
- **height**” to set the height of the button.

- Python

```
from tkinter import *  
  
root = Tk()  
  
w = Label(root, text='GeeksForGeeks.org!')  
  
w.pack()  
  
root.mainloop()
```



Listbox: It offers a list to the user from which the user can accept any number of options. The general syntax is:

```
w = Listbox(master, option=value)
```

master is the parameter used to represent the parent window.

There are number of options which are used to change the format of the widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **highlightcolor:** To set the color of the focus highlight when widget has to be focused.
- **bg:** to set the normal background color.
- **bd:** to set the border width in pixels.
- **font:** to set the font on the button label.
- **image:** to set the image on the widget.
- **width:** to set the width of the widget.
- **height:** to set the height of the widget.

- Python

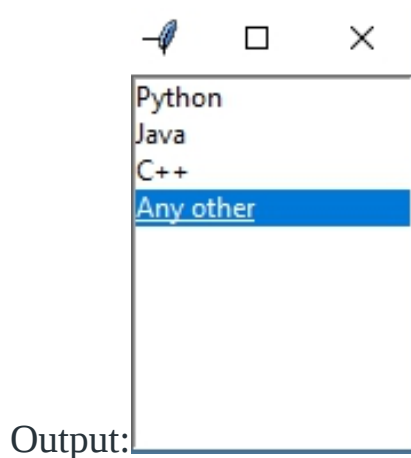
```
from tkinter import *
```

```
top = Tk()
```

```
Lb = Listbox(top)
```

```
Lb.insert(1, 'Python')
```

```
Lb.insert(2, 'Java')  
Lb.insert(3, 'C++')  
Lb.insert(4, 'Any other')  
Lb.pack()  
top.mainloop()
```



MenuButton: It is a part of top-down menu which stays on the window all the time. Every menubutton has its own functionality. The general syntax is:

```
w = MenuButton(master, option=value)
```

master is the parameter used to represent the parent window.

There are number of options which are used to change the format of the widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **activebackground:** To set the background when mouse is over the widget.
- **activeforeground:** To set the foreground when mouse is over the widget.
- **bg:** to set the normal background color.

- **bd:** to set the size of border around the indicator.
- **cursor:** To appear the cursor when the mouse over the menubutton.
- **image:** to set the image on the widget.
- **width:** to set the width of the widget.
- **height:** to set the height of the widget.
- **highlightcolor:** To set the color of the focus highlight when widget has to be focused.

◦ Python

```
from tkinter import *

top = Tk()

mb = Menubutton ( top, text = '&quot;GfG&quot;')
mb.grid()

mb.menu = Menu ( mb, tearoff = 0 )

mb['&quot;menu&quot;'] = mb.menu

cVar = IntVar()

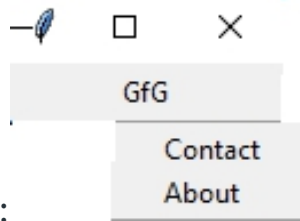
aVar = IntVar()

mb.menu.add_checkbutton ( label = 'Contact', variable = cVar )

mb.menu.add_checkbutton ( label = 'About', variable = aVar )

mb.pack()
```

```
top.mainloop()
```



Output:

Menu: It is used to create all kinds of menus used by the application. The general syntax is:

```
w = Menu(master, option=value)
```

master is the parameter used to represent the parent window.

There are number of options which are used to change the format of this widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **title:** To set the title of the widget.
- **activebackground:** to set the background color when widget is under the cursor.
- **activeforeground:** to set the foreground color when widget is under the cursor.
- **bg:** to set the normal background color.
- **command:** to call a function.
- **font:** to set the font on the button label.
- **image:** to set the image on the widget.

- Python


```
from tkinter import *

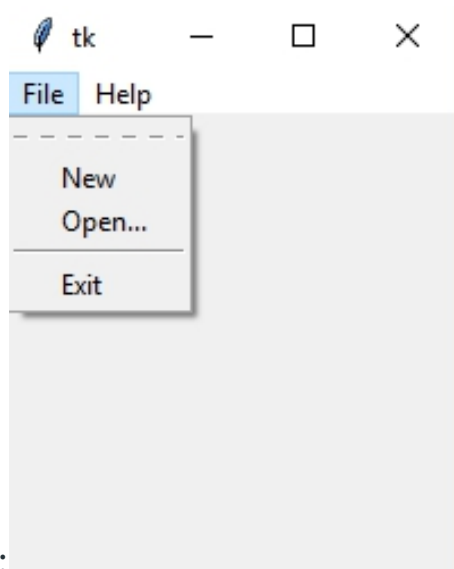
root = Tk()

menu = Menu(root)
root.config(menu=menu)

filemenu = Menu(menu)
menu.add_cascade(label='File', menu=filemenu)
filemenu.add_command(label='New')
filemenu.add_command(label='Open...')
filemenu.add_separator()
filemenu.add_command(label='Exit', command=root.quit)

helpmenu = Menu(menu)
menu.add_cascade(label='Help', menu=helpmenu)
helpmenu.add_command(label='About')

mainloop()
```



Output:

Message: It refers to the multi-line and non-editable text. It works same as that of Label. The general syntax is:

```
w = Message(master, option=value)
```

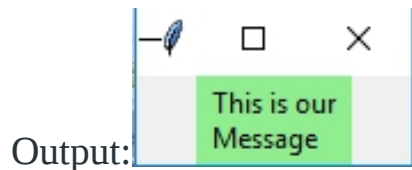
master is the parameter used to represent the parent window.

There are number of options which are used to change the format of the widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **bd:** to set the border around the indicator.
- **bg:** to set the normal background color.
- **font:** to set the font on the button label.
- **image:** to set the image on the widget.
- **width:** to set the width of the widget.
- **height:** to set the height of the widget.

- Python

```
from tkinter import *  
  
main = Tk()  
  
ourMessage ='This is our Message'  
  
messageVar = Message(main, text = ourMessage)  
  
messageVar.config(bg='lightgreen')  
  
messageVar.pack( )  
  
main.mainloop( )
```



RadioButton: It is used to offer multi-choice option to the user. It offers several options to the user and the user has to choose one option. The general syntax is:

```
w = RadioButton(master, option=value)
```

There are number of options which are used to change the format of this widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **activebackground:** to set the background color when widget is under the cursor.
- **activeforeground:** to set the foreground color when widget is under the cursor.
- **bg:** to set the normal background color.
- **command:** to call a function.

- **font:** to set the font on the button label.
- **image:** to set the image on the widget.
- **width:** to set the width of the label in characters.
- **height:** to set the height of the label in characters.

Python

```
from tkinter import *

root = Tk()

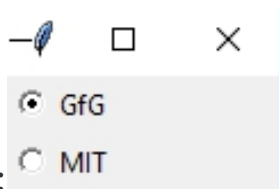
v = IntVar()

Radiobutton(root, text='GfG', variable=v, value=1).pack(anchor=W)

Radiobutton(root, text='MIT', variable=v, value=2).pack(anchor=W)

mainloop()
```

Output:



Scale: It is used to provide a graphical slider that allows to select any value from that scale. The general syntax is:

```
w = Scale(master, option=value)
```

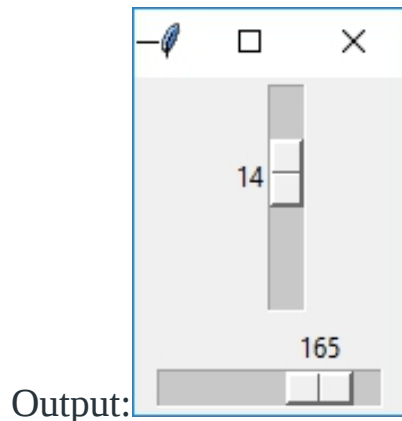
master is the parameter used to represent the parent window.

There are number of options which are used to change the format of the widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **cursor:** To change the cursor pattern when the mouse is over the widget.
- **activebackground:** To set the background of the widget when mouse is over the widget.
- **bg:** to set the normal background color.
- **orient:** Set it to HORIZONTAL or VERTICAL according to the requirement.
- **from_:** To set the value of one end of the scale range.
- **to:** To set the value of the other end of the scale range.
- **image:** to set the image on the widget.
- **width:** to set the width of the widget.

- Python

```
from tkinter import *  
  
master = Tk()  
  
w = Scale(master, from_=0, to=42)  
  
w.pack()  
  
w = Scale(master, from_=0, to=200, orient=HORIZONTAL)  
  
w.pack()  
  
mainloop()
```



Scrollbar: It refers to the slide controller which will be used to implement listed widgets. The general syntax is:

`w = Scrollbar(master, option=value)`

master is the parameter used to represent the parent window.

There are number of options which are used to change the format of the widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **width:** to set the width of the widget.
- **activebackground:** To set the background when mouse is over the widget.
- **bg:** to set the normal background color.
- **bd:** to set the size of border around the indicator.
- **cursor:** To appear the cursor when the mouse over the menubutton.

- Python

```
from tkinter import *
```

```
root = Tk()
```

```

scrollbar = Scrollbar(root)

scrollbar.pack( side = RIGHT, fill = Y )

mylist = Listbox(root, yscrollcommand = scrollbar.set )

for line in range(100):

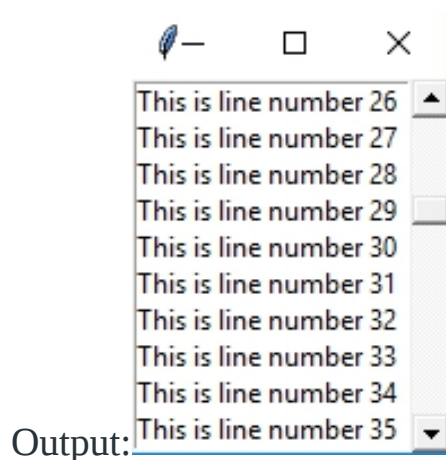
    mylist.insert(END, 'This is line number' + str(line))

mylist.pack( side = LEFT, fill = BOTH )

scrollbar.config( command = mylist.yview )

mainloop()

```



Text: To edit a multi-line text and format the way it has to be displayed. The general syntax is:

```
w =Text(master, option=value)
```

There are number of options which are used to change the format of the text. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **highlightcolor:** To set the color of the focus highlight when widget has to be focused.
- **insertbackground:** To set the background of the widget.

- **bg:** to set the normal background color.
- **font:** to set the font on the button label.
- **image:** to set the image on the widget.
- **width:** to set the width of the widget.
- **height:** to set the height of the widget.

◦ Python

```
from tkinter import *

root = Tk()

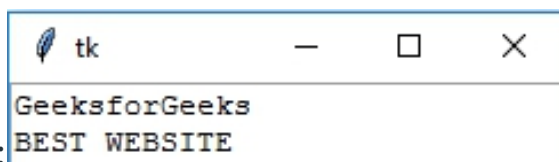
T = Text(root, height=2, width=30)

T.pack()

T.insert(END, 'GeeksforGeeks\nBEST WEBSITE\n')

mainloop()
```

Output:



TopLevel: This widget is directly controlled by the window manager. It don't need any parent window to work on. The general syntax is:

```
w = TopLevel(master, option=value)
```

There are number of options which are used to change the format of the widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

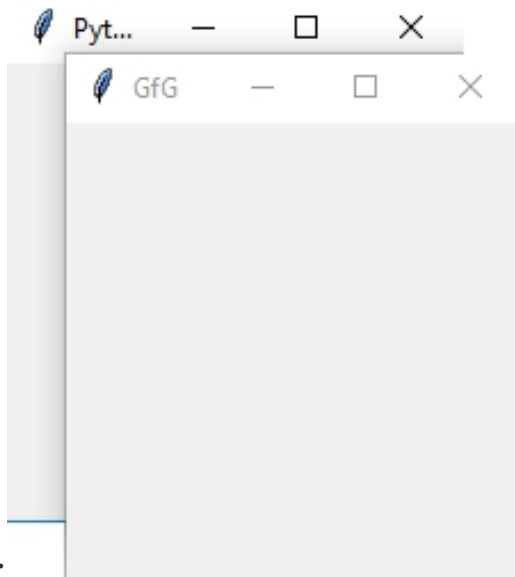
- **bg:** to set the normal background color.

- **bd:** to set the size of border around the indicator.
- **cursor:** To appear the cursor when the mouse over the menubutton.
- **width:** to set the width of the widget.
- **height:** to set the height of the widget.

- Python

```
from tkinter import *  
  
root = Tk()  
  
root.title('GfG')  
  
top = Toplevel()  
  
top.title('Python')  
  
top.mainloop()
```

Output:



SpinBox: It is an entry of 'Entry' widget. Here, value can be input by selecting a fixed value of numbers. The general syntax is:

`w = SpinBox(master, option=value)`

There are number of options which are used to change the format of the widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **bg:** to set the normal background color.
- **bd:** to set the size of border around the indicator.
- **cursor:** To appear the cursor when the mouse over the menubutton.
- **command:** To call a function.
- **width:** to set the width of the widget.
- **activebackground:** To set the background when mouse is over the widget.
- **disabledbackground:** To disable the background when mouse is over the widget.
- **from_:** To set the value of one end of the range.
- **to:** To set the value of the other end of the range.

- Python

```
from tkinter import *  
  
master = Tk()  
  
w = Spinbox(master, from_ = 0, to = 10)  
  
w.pack()
```

```
mainloop()
```



PanedWindow It is a container widget which is used to handle number of panes arranged in it. The general syntax is:

```
w = PanedWindow(master, option=value)
```

master is the parameter used to represent the parent window. There are number of options which are used to change the format of the widget. Number of options can be passed as parameters separated by commas. Some of them are listed below.

- **bg**: to set the normal background color.
- **bd**: to set the size of border around the indicator.
- **cursor**: To appear the cursor when the mouse over the menubutton.
- **width**: to set the width of the widget.
- **height**: to set the height of the widget.

- Python

```
from tkinter import *  
  
m1 = PanedWindow()  
  
m1.pack(fill = BOTH, expand = 1)  
  
left = Entry(m1, bd = 5)  
  
m1.add(left)
```

```
m2 = PanedWindow(m1, orient = VERTICAL)

m1.add(m2)

top = Scale( m2, orient = HORIZONTAL)

m2.add(top)

mainloop()
```

Output:



PART 2: Widgets

CHAPTER 1: Creating a button in tkinter

Tkinter is Python's standard GUI (Graphical User Interface) package. It is one of the most commonly used packages for GUI applications which comes with Python itself. Let's see how to create a button using Tkinter.

Follow the below steps:

1. Import tkinter module # Tkinter in Python 2.x. (Note Capital T)
2. Create main window (root = Tk())
3. Add as many widgets as you want.

Importing tkinter module is same as importing any other module.

```
import tkinter # In Python 3.x
```

```
import Tkinter # In python 2.x. (Note Capital T)
```

The **tkinter.ttk** module provides access to the Tk-themed widget set, introduced in Tk 8.5. If Python has not been compiled against Tk 8.5, this

module can still be accessed if *Tile* has been installed. The former method using Tk 8.5 provides additional benefits including anti-aliased font rendering under X11 and window transparency.

The basic idea for **tkinter.ttk** is to separate, to the extent possible, the code implementing a widget's behavior from the code implementing its appearance. **tkinter.ttk** is used to create modern GUI (Graphical User Interface) applications that cannot be achieved by *tkinter* itself.

Code #1: Creating button using Tkinter.

- Python3

```
# import everything from tkinter module

from tkinter import *

# create a tkinter window

root = Tk()

# Open window having dimension 100x100

root.geometry('100x100')

# Create a Button

btn = Button(root, text = 'Click me !', bd = '5',

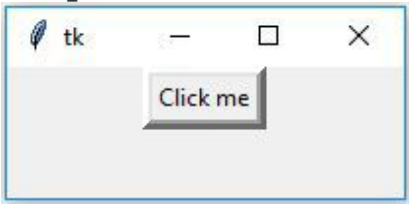
              command = root.destroy)

# Set the position of button on the top of window.

btn.pack(side = 'top')
```

```
root.mainloop()
```

Output:



Creation of Button without using *tk* themed widget.

Creation of Button using **tk** themed widget (tkinter.ttk). This will give you the effects of modern graphics. Effects will change from one OS to another because it is basically for the appearance.

Code #2:

- Python3

```
# import tkinter module

from tkinter import *

# Following will import tkinter.ttk module and
# automatically override all the widgets
# which are present in tkinter module.

from tkinter.ttk import *

# Create Object
```

```
root = Tk()

# Initialize tkinter window with dimensions 100x100

root.geometry('100x100')

btn = Button(root, text = 'Click me !',

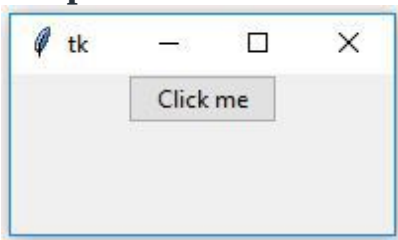
              command = root.destroy)

# Set the position of button on the top of window

btn.pack(side = 'top')

root.mainloop()
```

Output:



Note: See in the Output of both the code, BORDER is not present in 2nd output because **tkinter.ttk** does not support border. Also, when you hover the mouse over both the buttons **tk.Button** will change its color and become light blue (effects may change from one OS to another) because it supports modern graphics while in the case of a simple **Button** it won't change color as it does not support modern graphics.

CHAPTER 2: Add style to tkinter button

Tkinter is a Python standard library that is used to create GUI (Graphical User Interface) applications. It is one of the most commonly used packages of Python. Tkinter supports both traditional and modern graphics support with the help of Tk themed widgets. All the widgets that Tkinter also has available in **tkinter.ttk**.

Adding style in a tkinter.ttk button is a little creepy because it doesn't support direct implementation. To add styling in a ttk.Button we have to first create an object of style class which is available in tkinter.ttk.

We can create ttk.Button by using the following steps:

```
btn = ttk.Button(master, option = value, ...)
```

ttk.Button options –

command: *A function to be called when button is pressed.*

text: *Text which appears on the Button.*

image: *Image to be appeared on the Button.*

style: *Style to be used in rendering this button.*

To add styling on the ttk.Button we cannot directly pass the value in the options. Firstly, we have to create a Style object which can be created as follows:

```
style = ttk.Style()
```

Below code will be adding style to only selected Buttons i.e, only those buttons will get changed in which we will be passing style option.

Code #1:

- Python3

```
# Import Required Module
```

```
from tkinter import *
```

```
from tkinter.ttk import *

# Create Object

root = Tk()

# Set geometry (widthxheight)

root.geometry('100x100')

# This will create style object

style = Style()

# This will be adding style, and
# naming that style variable as
# W.Tbutton (TButton is used for ttk.Button).
style.configure('W.TButton', font =
                ('calibri', 10, 'bold', 'underline'),
                foreground = 'red')

# Style will be reflected only on
# this button because we are providing
# style only on this Button.

''' Button 1'''

btn1 = Button(root, text = 'Quit !',
              style = 'W.TButton',
              command = root.destroy)
```

```
btn1.grid(row = 0, column = 3, padx = 100)

''' Button 2'''

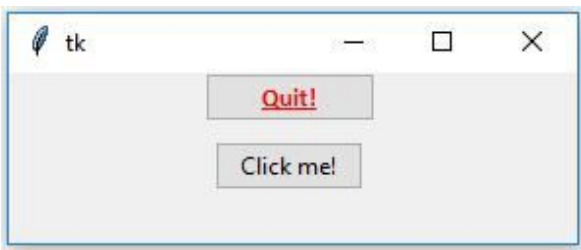
btn2 = Button(root, text = 'Click me !', command = None)

btn2.grid(row = 1, column = 3, pady = 10, padx = 100)

# Execute Tkinter

root.mainloop()
```

Output:



Only one button will get styled because in the above code we are providing styling only in one Button.

Code #2 Apply style on all the available buttons

- Python3

```
# Import Required Module

from tkinter import *

from tkinter.ttk import *
```

```
# Create Root Object

root = Tk()

# Set Geometry(widthxheight)

root.geometry('100x100')

# Create style Object

style = Style()

# Will add style to every available button

# even though we are not passing style

# to every button widget.

style.configure('TButton', font =

                ('calibri', 10, 'bold', 'underline'),

                foreground = 'red')

# button 1

btn1 = Button(root, text = 'Quit !',

               style = 'TButton',

               command = root.destroy)

btn1.grid(row = 0, column = 3, padx = 100)

# button 2

btn2 = Button(root, text = 'Click me !', command = None)
```

```
btn2.grid(row = 1, column = 3, pady = 10, padx = 100)
```

```
# Execute Tkinter
```

```
root.mainloop()
```

Output:



Now if you want to change the appearance of the buttons by the movement of the mouse i.e, now when we hover the mouse over the button it will change its color when we press it will change color, and so on.

Code #3 Change color on mouse hover

- Python3

```
# Import Required Module
```

```
from tkinter import *
```

```
from tkinter.ttk import *
```

```
# Create Root Object
```

```
root = Tk()
```

```
# Set Geometry(widthxheight)
```

```
root.geometry('500x500')
```

```
# Create style Object

style = Style()

style.configure('TButton', font =
                ('calibri', 20, 'bold'),

                borderwidth = '4')

# Changes will be reflected
# by the movement of mouse.

style.map('TButton', foreground = [('active', '!disabled', 'green')],

         background = [('active', 'black')])

# button 1

btn1 = Button(root, text = 'Quit !', command = root.destroy)

btn1.grid(row = 0, column = 3, padx = 100)

# button 2

btn2 = Button(root, text = 'Click me !', command = None)

btn2.grid(row = 1, column = 3, pady = 10, padx = 100)

# Execute Tkinter

root.mainloop()
```

Output:



CHAPTER 3: Add image on a Tkinter button

[Tkinter](#) is a Python module which is used to create GUI (Graphical User Interface) applications with the help of varieties of widgets and functions. Like any other GUI module it also supports images i.e you can use images in the application to make it more attractive.

Image can be added with the help of `PhotoImage()` method. This is a Tkinter method which means you don't have to import any other module in order to use it.

Important: If both image and text are given on [Button](#), the text will be dominated and only image will appear on the Button. But if you want to show both image and text then you have to pass **compound** in button options.

`Button(master, text = "Button", image = "image.png", compound=LEFT)`

`compound = LEFT` -> image will be at left side of the button

`compound = RIGHT` -> image will be at right side of button

`compound = TOP` -> image will be at top of button

`compound = BOTTOM` -> image will be at bottom of button

Syntax:

```
photo = PhotoImage(file = "path_of_file")
```

path_of_file is any valid path available on your local machine.

Code #1:

```
# importing only those functions
# which are needed

from tkinter import *

from tkinter.ttk import *

# creating tkinter window

root = Tk()

# Adding widgets to the root window

Label(root, text = 'GeeksforGeeks', font =(

    'Verdana', 15)).pack(side = TOP, pady = 10)

# Creating a photoimage object to use image

photo = PhotoImage(file = r"C:\Gfg\circle.png")

# here, image option is used to
# set image on button

Button(root, text = 'Click Me !', image = photo).pack(side = TOP)

mainloop()
```


Output:



In output observe that only image is shown on the button and the size of the button is also bigger than the usual size it is because we haven't set the size of the image.

Code #2: To show both image and text on [Button](#).

```
# importing only those functions
# which are needed

from tkinter import *

from tkinter.ttk import *

# creating tkinter window

root = Tk()

# Adding widgets to the root window

Label(root, text = 'GeeksforGeeks', font =(
    'Verdana', 15)).pack(side = TOP, pady = 10)
```

```
# Creating a photoimage object to use image

photo = PhotoImage(file = r"C:\Gfg\circle.png")

# Resizing image to fit on button

photoimage = photo.subsample(3, 3)

# here, image option is used to
# set image on button
# compound option is used to align
# image on LEFT side of button

Button(root, text = 'Click Me !', image = photoimage,

        compound = LEFT).pack(side = TOP)

mainloop()
```

Output:



Observe that both text and image are appearing as well as size of the image is also small.

CHAPTER 4: Python Tkinter – Label

Python offers multiple options for developing a GUI (Graphical User Interface). Out of all the GUI methods, Tkinter is the most commonly used method. It is a standard Python interface to the Tk GUI toolkit shipped with Python. Python with Tkinter is the fastest and easiest way to create GUI applications. Creating a GUI using Tkinter is an easy task using widgets. Widgets are standard graphical user interfaces (GUI) elements, like buttons and menus. **Note:** For more information, refer to [Python GUI – tkinter](#)

Label Widget

Tkinter Label is a widget that is used to implement display boxes where you can place text or images. The text displayed by this widget can be changed by the developer at any time you want. It is also used to perform tasks such as to underline the part of the text and span the text across multiple lines. It is important to note that a label can use only one font at a time to display text. To use a label, you just have to specify what to display in it (this can be text, a bitmap, or an image). **Syntax:**

w = Label (master, option, ...)

Parameters:

- **master:** This represents the parent window
- **options:** Below is the list of most commonly used options for this widget. These options can be used as key-value pairs separated by commas:

Various Options are:

- **anchor:** This options is used to control the positioning of the text if the widget has more space than required for the text. The default is anchor=CENTER, which centers the text in the available space.
- **bg:**This option is used to set the normal background color displayed behind the label and indicator.
- **height:**This option is used to set the vertical dimension of the new frame.
- **width:**Width of the label in characters (not pixels!). If this option is not set, the label will be sized to fit its contents.

- **bd:**This option is used to set the size of the border around the indicator. Default bd value is set on 2 pixels.
- **font:**If you are displaying text in the label (with the text or textvariable option), the font option is used to specify in what font that text in the label will be displayed.
- **cursor:**It is used to specify what cursor to show when the mouse is moved over the label. The default is to use the standard cursor.
- **textvariable:** As the name suggests it is associated with a Tkinter variable (usually a StringVar) with the label. If the variable is changed, the label text is updated.
- **bitmap:**It is used to set the bitmap to the graphical object specified so that, the label can represent the graphics instead of text.
- **fg:**The label color, used for text and bitmap labels. The default is system specific. If you are displaying a bitmap, this is the color that will appear at the position of the 1-bits in the bitmap.
- **image:** This option is used to display a static image in the label widget.
- **padx:**This option is used to add extra spaces between left and right of the text within the label.The default value for this option is 1.
- **pady:**This option is used to add extra spaces between top and bottom of the text within the label.The default value for this option is 1.
- **justify:**This option is used to define how to align multiple lines of text. Use LEFT, RIGHT, or CENTER as its values. Note that to position the text inside the widget, use the anchor option. Default value for justify is CENTER.
- **relief:** This option is used to specify appearance of a decorative border around the label. The default value for this option is FLAT.
- **underline:**This
- **wraplength:**Instead of having only one line as the label text it can be broken into to the number of lines where each line has the number of characters specified to this option.

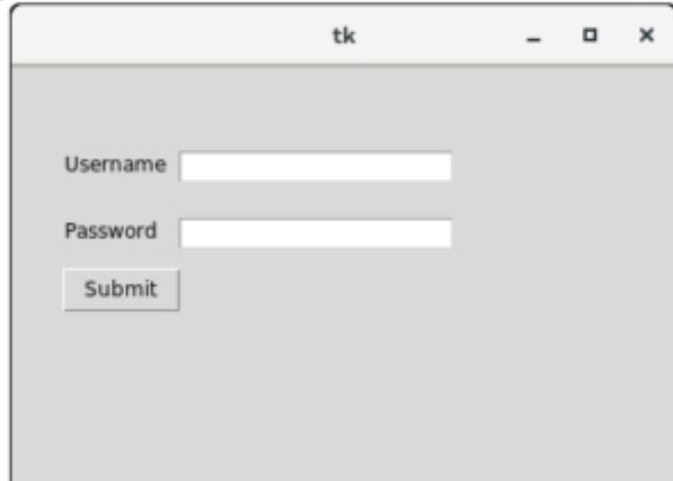
Example:

Python3

```
from tkinter import *
```


y = 100)

top.mainloop()



Output :

CHAPTER 5: Create LabelFrame and add widgets to it

Tkinter is a Python module which is used to create GUI (Graphical User Interface) applications. It is a widely used module which comes along with the Python. It consists of various types of widgets which can be used to make GUI more user-friendly and attractive as well as functionality can be increased. LabelFrame can be created as follows:

-> import tkinter

-> create root

-> create LabelFrame as child of root

```
label_frame = ttk.LabelFrame(parent, value = options, ...)
```

Code #1: Creating LabelFrame and adding a message to it.

- Python3

```
# Import only those methods
# which are mentioned below, this way of
# importing methods is efficient

from tkinter import Tk, mainloop

from tkinter.ttk import Label, LabelFrame

# Creating tkinter window with fixed geometry

root = Tk()

root.geometry('250x150')

# This will create a LabelFrame

label_frame = LabelFrame(root, text='This is Label Frame')

label_frame.pack(expand='yes', fill='both')

label1 = Label(label_frame, text='1. This is a Label.')

label1.place(x=0, y=5)

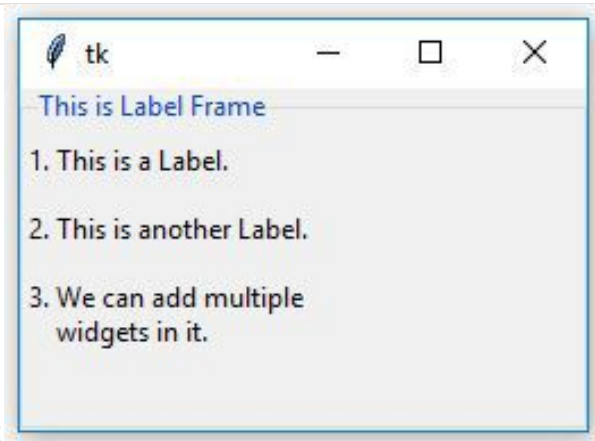
label2 = Label(label_frame, text='2. This is another Label.')

label2.place(x=0, y=35)
```

```
label3 = Label(label_frame,
               text='3. We can add multiple\n  widgets in it.')

label3.place(x=0, y=65)

# This creates an infinite loop which generally
# waits for any interrupt (like keyboard or
# mouse) to terminate
mainloop()
```



Output:

Code

#2: Adding [Button](#) and [CheckButton](#) widgets inside LabelFrame.

- Python3

```
# Import only those methods
# which are mentioned below, this way of
# importing methods is efficient

from tkinter import Tk, mainloop
```



```
from tkinter.ttk import Checkbutton, Button, LabelFrame

# Creating tkinter window with fixed geometry

root = Tk()

root.geometry('250x150')

# This will create a LabelFrame

label_frame = LabelFrame(root, text='This is Label Frame')

label_frame.pack(expand='yes', fill='both')

# Buttons

btn1 = Button(label_frame, text='Button 1')

btn1.place(x=30, y=10)

btn2 = Button(label_frame, text='Button 2')

btn2.place(x=130, y=10)

# Checkbuttons

chkbtn1 = Checkbutton(label_frame, text='Checkbutton 1')

chkbtn1.place(x=30, y=50)

chkbtn2 = Checkbutton(label_frame, text='Checkbutton 2')

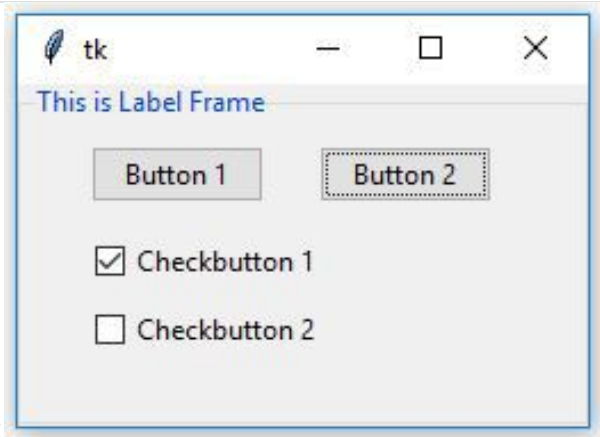
chkbtn2.place(x=30, y=80)

# This creates infinite loop which generally

# waits for any interrupt (like keyboard or

# mouse) to terminate
```

```
mainloop()
```



Output:

Note: One can also add another *LabelFrame* inside another *LabelFrame*, as well as one can do styling of any *LabelFrame* like we do the styling of other widgets.

RadioButton in Tkinter

The **Radiobutton** is a standard [Tkinter](#) widget used to implement one-of-many selections. **Radiobuttons** can contain text or images, and you can associate a Python function or method with each button. When the button is pressed, [Tkinter](#) automatically calls that function or method.

Syntax:

button = Radiobutton(master, text="Name on Button", variable = "shared variable", value = "values of each button", options = values, ...)

shared variable = A Tkinter variable shared among all Radio buttons

value = each radiobutton should have different value otherwise more than 1

radiobutton will get selected.

Code #1:

Radio buttons, but not in the form of buttons, in form of **button box**. In order to display button box, *indicatoron/indicator* option should be set to 0.

- Python3

```
# Importing Tkinter module

from tkinter import *

# from tkinter.ttk import *

# Creating master Tkinter window

master = Tk()

master.geometry("175x175")

# Tkinter string variable

# able to store any string value

v = StringVar(master, "1")

# Dictionary to create multiple buttons

values = {"RadioButton 1" : "1",

          "RadioButton 2" : "2",

          "RadioButton 3" : "3",

          "RadioButton 4" : "4",
```

```
"RadioButton 5" : "5"}


```

```
# Loop is used to create multiple Radiobuttons


```

```
# rather than creating each button separately


```

```
for (text, value) in values.items():


```

```
    Radiobutton(master, text = text, variable = v,


```

```
                value = value, indicator = 0,


```

```
                background = "light blue").pack(fill = X, ipady = 5)


```

```
# Infinite loop can be terminated by


```

```
# keyboard or mouse interrupt


```

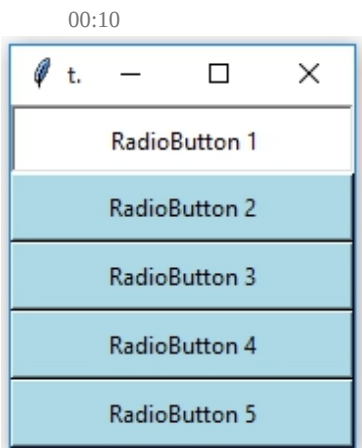
```
# or by any predefined function (destroy())


```

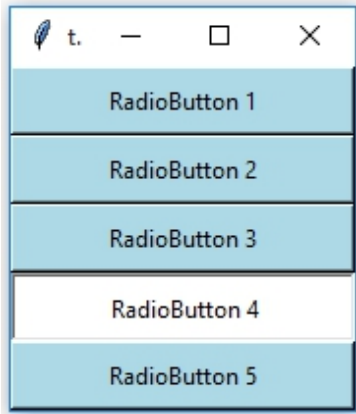
```
mainloop()


```

Output:



The background of these button boxes is light blue. Button boxes having a white backgrounds as well as sunken are selected ones.



Code #2: Changing button boxes into standard radio buttons. For this remove indicatoron option.

- Python3

```
# Importing Tkinter module

from tkinter import *

from tkinter.ttk import *

# Creating master Tkinter window

master = Tk()

master.geometry("175x175")

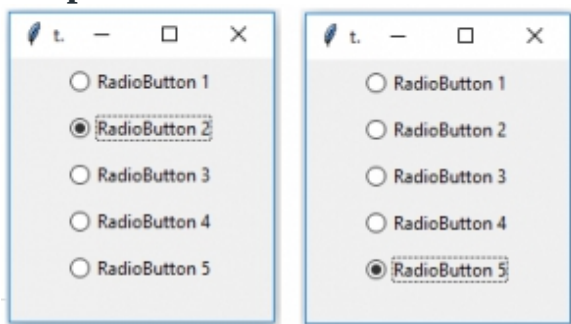
# Tkinter string variable
# able to store any string value

v = StringVar(master, "1")

# Dictionary to create multiple buttons
```

```
values = {"RadioButton 1" : "1",  
  
         "RadioButton 2" : "2",  
  
         "RadioButton 3" : "3",  
  
         "RadioButton 4" : "4",  
  
         "RadioButton 5" : "5"}  
  
# Loop is used to create multiple Radiobuttons  
# rather than creating each button separately  
for (text, value) in values.items():  
    Radiobutton(master, text = text, variable = v,  
               value = value).pack(side = TOP, ipady = 5)  
  
# Infinite loop can be terminated by  
# keyboard or mouse interrupt  
# or by any predefined function (destroy())  
mainloop()
```

Output:



Code #3: Adding Style to Radio Button using style class.

- Python3

```
# Importing Tkinter module

from tkinter import *

from tkinter.ttk import *

# Creating master Tkinter window

master = Tk()

master.geometry('175x175')

# Tkinter string variable

# able to store any string value

v = StringVar(master, "1")

# Style class to add style to Radiobutton

# it can be used to style any ttk widget

style = Style(master)

style.configure("TRadiobutton", background = "light green",

                foreground = "red", font = ("arial", 10, "bold"))

# Dictionary to create multiple buttons

values = {"RadioButton 1" : "1",
```

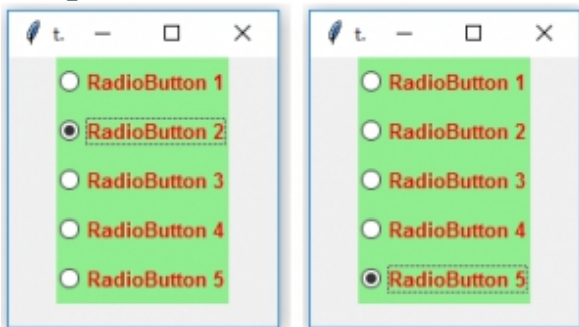
```
"RadioButton 2" : "2",
```

```
"RadioButton 3" : "3",
```

```
"RadioButton 4" : "4",
```

```
"RadioButton 5" : "5"}  
  
# Loop is used to create multiple Radiobuttons  
  
# rather than creating each button separately  
  
for (text, value) in values.items():  
  
    Radiobutton(master, text = text, variable = v,  
  
                value = value).pack(side = TOP, ipady = 5)  
  
# Infinite loop can be terminated by  
  
# keyboard or mouse interrupt  
  
# or by any predefined function (destroy())  
  
mainloop()
```

Output:



You may observe that font style is changed as well as background and foreground colors are also changed. Here, TRadiobutton is used in style class, it automatically applies styling to all the available Radiobuttons.

CHAPTER 6: Python Tkinter – Checkbutton Widget

Python offers multiple options for developing a GUI (Graphical User Interface). Out of all the GUI methods, Tkinter is the most commonly used method. It is a standard Python interface to the Tk GUI toolkit shipped with Python. Python with Tkinter is the fastest and easiest way to create GUI applications. Creating a GUI using Tkinter is an easy task.

Note: For more information, refer to [Python GUI – tkinter](#)

Checkbutton Widget

The Checkbutton widget is a standard Tkinter widget that is used to implement on/off selections. Checkbuttons can contain text or images. When the button is pressed, Tkinter calls that function or method.

Syntax:

The syntax to use the checkbutton is given below.

```
w = Checkbutton ( master, options)
```

Parameters:

- **master:** This parameter is used to represents the parent window.
- **options:** There are many options which are available and they can be used as key-value pairs separated by commas.

Options:

Following are commonly used Option can be used with this widget :-

- **activebackground:** This option used to represent the background color when the checkbutton is under the cursor.
- **activeforeground:** This option used to represent the foreground color when the checkbutton is under the cursor.
- **bg:** This option used to represent the normal background color displayed behind the label and indicator.
- **bitmap:** This option used to display a monochrome image on a button.
- **bd:** This option used to represent the size of the border around the indicator and the default value is 2 pixels.
- **command:** This option is associated with a function to be called when the state of the checkbutton is changed.
- **cursor:** By using this option, the mouse cursor will change to that pattern when it is over the checkbutton.
- **disabledforeground:** The foreground color used to render the text of a disabled checkbutton. The default is a stippled version of the default foreground color.
- **font:** This option used to represent the font used for the text.
- **fg:** This option used to represent the color used to render the text.
- **height:** This option used to represent the number of lines of text on the checkbutton and it's default value is 1.
- **highlightcolor:** This option used to represent the color of the focus highlight when the checkbutton has the focus.
- **image:** This option used to display a graphic image on the button.
- **justify:** This option used to control how the text is justified: CENTER, LEFT, or RIGHT.
- **offvalue:** The associated control variable is set to 0 by default if the button is unchecked. We can change the state of an unchecked variable to some other one.
- **onvalue:** The associated control variable is set to 1 by default if the button is checked. We can change the state of the checked variable to some other one.
- **padx:** This option used to represent how much space to leave to the left and right of the checkbutton and text. It's default value is 1 pixel.

- **pady:** This option used to represent how much space to leave above and below the checkbox and text. It's default value is 1 pixel.
- **relief:** The type of the border of the checkbox. It's default value is set to FLAT.
- **selectcolor:** This option used to represent the color of the checkbox when it is set. The Default is selectcolor="red".
- **selectimage:** The image is shown on the checkbox when it is set.
- **state:** It represents the state of the checkbox. By default, it is set to normal. We can change it to DISABLED to make the checkbox unresponsive. The state of the checkbox is ACTIVE when it is under focus.
- **text:** This option used use newlines ("\\n") to display multiple lines of text.
- **underline:** This option used to represent the index of the character in the text which is to be underlined. The indexing starts with zero in the text.
- **variable:** This option used to represents the associated variable that tracks the state of the checkbox.
- **width:** This option used to represents the width of the checkbox. and also represented in the number of characters that are represented in the form of texts.
- **wraplength:** This option will be broken text into the number of pieces.

Methods:

Methods used in this widgets are as follows:

- **deselect():** This method is called to turn off the checkbox.
- **flash():** The checkbox is flashed between the active and normal colors.
- **invoke():** This method will invoke the method associated with the checkbox.
- **select():** This method is called to turn on the checkbox.
- **toggle():** This method is used to toggle between the different Checkbuttons.

Example:

```
from tkinter import *

root = Tk()
root.geometry("300x200")

w = Label(root, text ='GeeksForGeeks', font = "50")
w.pack()

Checkbutton1 = IntVar()

Checkbutton2 = IntVar()

Checkbutton3 = IntVar()

Button1 = Checkbutton(root, text = "Tutorial",

                        variable = Checkbutton1,

                        onvalue = 1,

                        offvalue = 0,

                        height = 2,

                        width = 10)

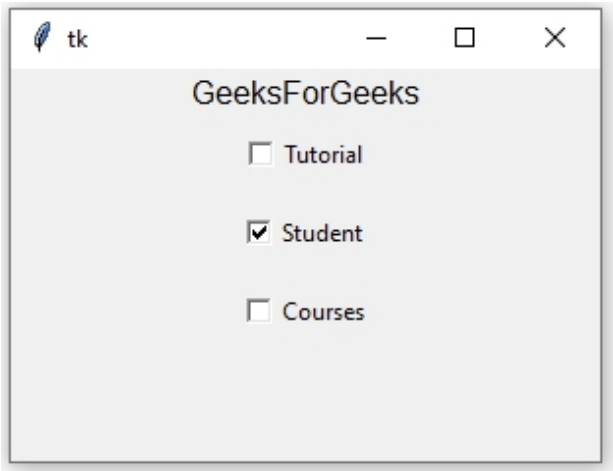
Button2 = Checkbutton(root, text = "Student",

                        variable = Checkbutton2,

                        onvalue = 1,
```

```
        offvalue = 0,  
        height = 2,  
        width = 10)  
  
Button3 = Checkbutton(root, text = "Courses",  
        variable = Checkbutton3,  
        onvalue = 1,  
        offvalue = 0,  
        height = 2,  
        width = 10)  
  
Button1.pack()  
Button2.pack()  
Button3.pack()  
  
mainloop()
```

Output:



CHAPTER 7: Python Tkinter – Canvas Widget

Tkinter is a GUI toolkit used in python to make user-friendly GUIs. Tkinter is the most commonly used and the most basic GUI framework available in python. Tkinter uses an object-oriented approach to make GUIs.

Note: For more information, refer to Python GUI – tkinter

Canvas widget

The Canvas widget lets us display various graphics on the application. It can be used to draw simple shapes to complicated graphs. We can also display various kinds of custom widgets according to our needs.

Syntax:

```
C = Canvas(root, height, width, bd, bg, ..)
```

Optional parameters:

- **root** = root window.
- **height** = height of the canvas widget.
- **width** = width of the canvas widget.
- **bg** = background colour for canvas.
- **bd** = border of the canvas window.
- **scrollregion** (w, n, e, s) tuple defined as a region for scrolling left, top, bottom and right.
- **highlightcolor** colour shown in the focus highlight.
- **cursor** It can be defined as a cursor for the canvas which can be a circle, a dot, an arrow etc.
- **confine** decides if canvas can be accessed outside the scroll region.
- **relief** type of the border which can be SUNKEN, RAISED, GROOVE and RIDGE.

Some common drawing methods:

- **Creating an Oval**

```
oval = C.create_oval(x0, y0, x1, y1, options)
```

- **Creating an arc**

```
arc = C.create_arc(20, 50, 190, 240, start=0, extent=110, fill="red")
```

- **Creating a Line**

```
line = C.create_line(x0, y0, x1, y1, ..., xn, yn, options)
```

- **Creating a polygon**

```
oval = C.create_polygon(x0, y0, x1, y1, ...xn, yn, options)
```

Example 1: Simple Shapes Drawing

- Python3

```
from tkinter import *

root = Tk()

C = Canvas(root, bg="yellow",
            height=250, width=300)

line = C.create_line(108, 120,
                     320, 40,
                     fill="green")

arc = C.create_arc(180, 150, 80,
                  210, start=0,
                  extent=220,
                  fill="red")

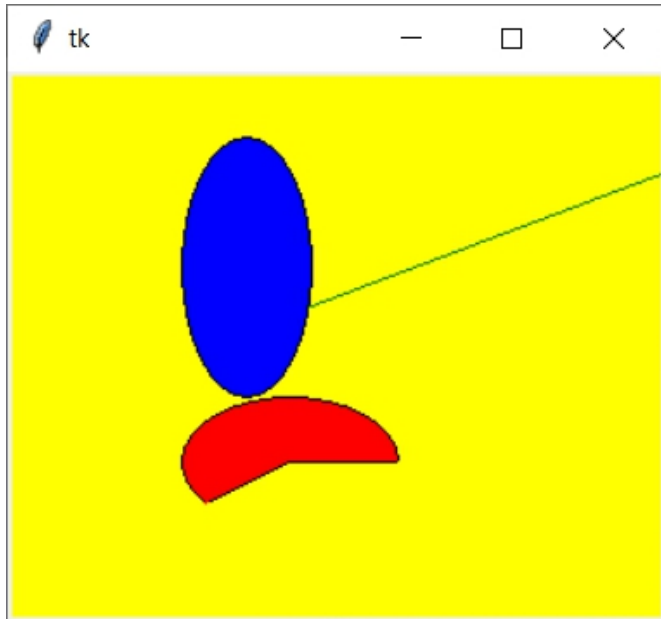
oval = C.create_oval(80, 30, 140,
                    150,
                    fill="blue")

C.pack()
```



```
mainloop()
```

Output:



Example 2: Simple Paint App

- Python3

```
from tkinter import *  
  
root = Tk()  
  
# Create Title  
root.title( "Paint App ")
```

```
# specify size

root.geometry("500x350")


# define function when

# mouse double click is enabled

def paint( event ):


    # Co-ordinates.

    x1, y1, x2, y2 = ( event.x - 3 ),( event.y - 3 ), ( event.x + 3 ),( event.y + 3 )


    # Colour

    Colour = "#000fff000"


    # specify type of display

    w.create_line( x1, y1, x2,

                  y2, fill = Colour )


# create canvas widget.

w = Canvas(root, width = 400, height = 250)


# call function when double

# click is enabled.
```

```
w.bind( "<B1-Motion>", paint )

# create label.

l = Label( root, text = "Double Click and Drag to draw." )

l.pack()

w.pack()

mainloop()
```

Output:



CHAPTER 8: Create different shapes using Canvas class

In Tkinter, **Canvas class** is used to create different shapes with the help of some functions which are defined under Canvas class. Any shape that Canvas class creates requires a canvas, so before creating any shapes a Canvas object is required and needs to be packed to the main window.

Canvas Methods for shapes:

Canvas.create_oval(x1, y1, x2, y2, options = ...): It is used to create a oval, pieslice and chord.

Canvas.create_rectangle(x1, y1, x2, y2, options = ...): It is used to create rectangle and square.

Canvas.create_arc(x1, y1, x2, y2, options = ...) This is used to create an arc.

Canvas.create_polygon(coordinates, options = ...) THis is used to create any valid shapes.

We are using a class to show the working of functions that helps to creates different shapes.

Class parameters –

Data members used: master, canvas

Member functions used: create() method

Widgets used: Canvas

Tkinter method used:

Canvas.create_oval()

Canvas.create_rectangle()

Canvas.create_arc()

Canvas.create_polygon()

pack()

title()

geometry()

Below is the Python code –

- Python3

```
# Imports each and every method and class
# of module tkinter and tkinter.ttk

from tkinter import * from tkinter.ttk import * class Shape:

    def __init__(self, master = None):

        self.master = master

        # Calls create method of class Shape

        self.create()

    def create(self):

        # Creates a object of class canvas

        # with the help of this we can create different shapes

        self.canvas = Canvas(self.master)

        # Creates a circle of diameter 80

        self.canvas.create_oval(10, 10, 80, 80,

                                outline = "black", fill = "white",

                                width = 2)
```

```
# Creates an ellipse with horizontal diameter
# of 210 and vertical diameter of 80
self.canvas.create_oval(110, 10, 210, 80,

                        outline = "red", fill = "green",

                        width = 2)

# Creates a rectangle of 50x60 (heightxwidth)
self.canvas.create_rectangle(230, 10, 290, 60,

                             outline = "black", fill = "blue",

                             width = 2)

# Creates an arc of 210 deg

self.canvas.create_arc(30, 200, 90, 100, start = 0,

                       extent = 210, outline = "green",

                       fill = "red", width = 2)

points = [150, 100, 200, 120, 240, 180,

          210, 200, 150, 150, 100, 200]

# Creates a polygon

self.canvas.create_polygon(points, outline = "blue",

                           fill = "orange", width = 2)

# Pack the canvas to the main window and make it expandable

self.canvas.pack(fill = BOTH, expand = 1)
```

```
if __name__ == "__main__":

    # object of class Tk, responsible for creating
    # a tkinter toplevel window

    master = Tk()

    shape = Shape(master)

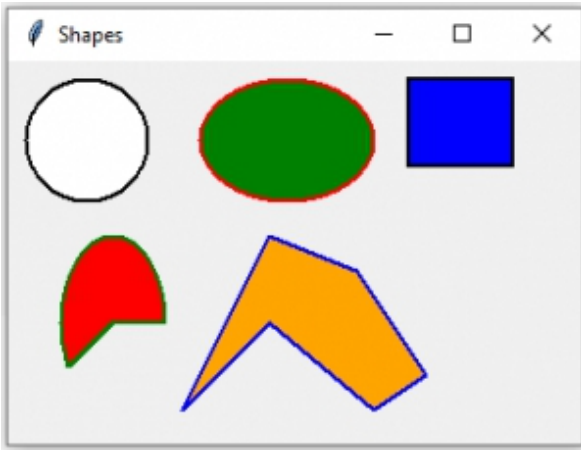
    # Sets the title to Shapes
    master.title("Shapes")

    # Sets the geometry and position
    # of window on the screen

    master.geometry("330x220 + 300 + 300")

    # Infinite loop breaks only by interrupt
    mainloop()
```

Output:



CHAPTER 9: Create different type of lines using Canvas class

In Tkinter, `Canvas.create_line()` method is used to create lines in any canvas. These lines can only be seen on canvas so first, you need to create a Canvas object and later pack it into the main window.

Syntax:

```
Canvas.create_line(x1, y1, x2, y2, ..., options = ...)
```

Note: Minimum of 4 points are required to create a line, but you can also add multiple points to create different drawings.

Class parameters:

Data members used:

master

canvas

Member functions used for the given class: *create()* method

Widgets used: *Canvas*

Tkinter method used:

canvas.create_line()
pack()
title()
geometry()

Below is the Python code –

- Python3

```
# Imports each and every method and class
# of module tkinter and tkinter.ttk

from tkinter import *
from tkinter.ttk import *

class GFG:

    def __init__(self, master = None):

        self.master = master

        # Calls create method of class GFG
        self.create()

    def create(self):

        # This creates a object of class canvas

        self.canvas = Canvas(self.master)
```

```
# This creates a line of length 200 (straight horizontal line)
```

```
self.canvas.create_line(15, 25, 200, 25)
```

```
# This creates a lines of 300 (straight vertical dashed line)
```

```
self.canvas.create_line(300, 35, 300, 200, dash = (5, 2))
```

```
# This creates a triangle (triangle can be created by other methods also)
```

```
self.canvas.create_line(55, 85, 155, 85, 105, 180, 55, 85)
```

```
# This pack the canvas to the main window and make it expandable
```

```
self.canvas.pack(fill = BOTH, expand = True)
```

```
if __name__ == "__main__":
```

```
# object of class Tk, responsible for creating
```

```
# a tkinter toplevel window
```

```
master = Tk()
```

```
geeks = GFG(master)
```

```
# This sets the title to Lines
```

```
master.title("Lines")
```

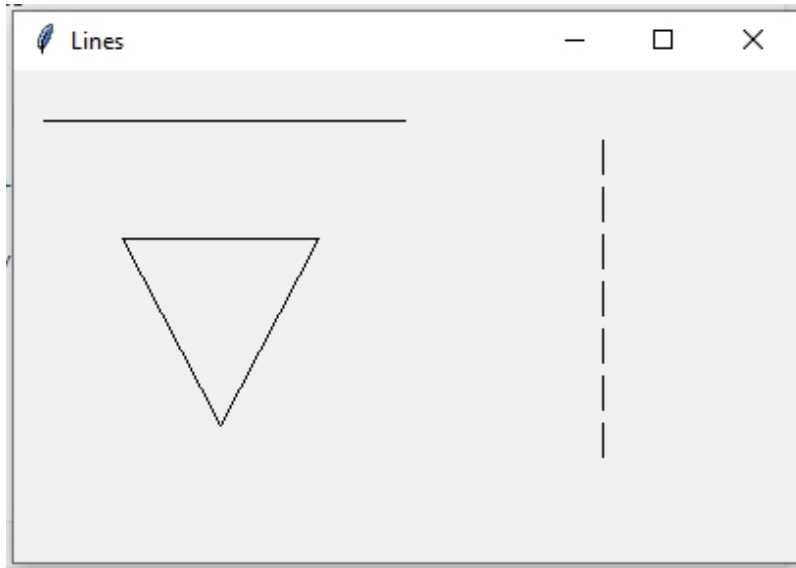
```
# This sets the geometry and position of window
```

```
# on the screen
```

```
master.geometry("400x250 + 300 + 300")
```

```
# Infinite loop breaks only by interrupt  
master.mainloop()
```

Output:



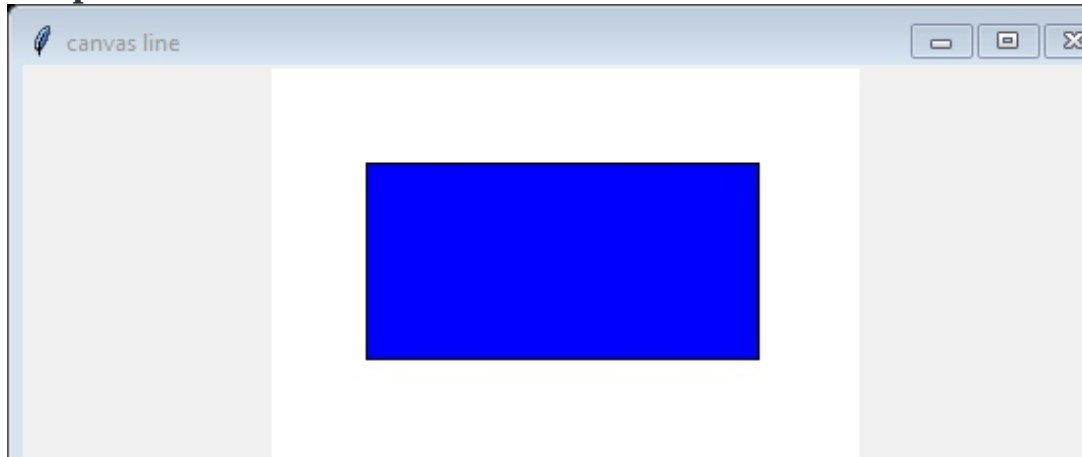
Example 2: For filling color in the shapes.

- Python3

```
from tkinter import *  
root=Tk()  
root.title("canvas line")  
root.geometry("555x555")  
our_canvas=Canvas(root,width=300,height=200,bg="white")  
our_canvas.pack()  
#creating rectangle  
our_canvas.create_rectangle(50,150,250,50,fill="blue")
```

```
root.mainloop()
```

Output:



CHAPTER 10: Moving objects using Canvas.move() method

The **Canvas class** of Tkinter supports functions that are used to move objects from one position to another in any canvas or Tkinter top-level.

Syntax: `Canvas.move(canvas_object, x, y)`

Parameters:

canvas_object is any valid image or drawing created with the help of Canvas class. To know how to create object using Canvas class take reference of [this](#).

x is horizontal distance from upper-left corner.

y is vertical distance from upper-left corner.

We will use class to see the working of the move() method.

Class parameters-

Data members used:

master

x

y

canvas

rectangle

Member functions used:

movement()

left()

right()

up()

down()

Widgets used: Canvas

Tkinter method used:

Canvas.create_rectangle()

pack()

Canvas.move()

after()

bind()

Below is the Python implementation:

Python3

```
import tkinter as tk
```

```
class MoveCanvas(tk.Canvas):
```

```
    def __init__(self, *args, **kwargs):
```

```
        super().__init__(*args, **kwargs)
```

```
        self.dx = 0
```

```
        self.dy = 0
```

```
        self.box = self.create_rectangle(0, 0, 10, 10, fill="black")

        self.dt = 25

        self.tick()

    def tick(self):

        self.move(self.box, self.dx, self.dy)

        self.after(self.dt, self.tick)

    def change_heading(self, dx, dy):

        self.dx = dx

        self.dy = dy

if __name__ == "__main__":

    root = tk.Tk()

    root.geometry("300x300")

    cvs = MoveCanvas(root)

    cvs.pack(fill="both", expand=True)

    ds = 3
```

```
root.bind("<KeyPress-Left>", lambda _: cvs.change_heading(-ds, 0))

root.bind("<KeyPress-Right>", lambda _: cvs.change_heading(ds, 0))

root.bind("<KeyPress-Up>", lambda _: cvs.change_heading(0, -ds))

root.bind("<KeyPress-Down>", lambda _: cvs.change_heading(0, ds))


root.mainloop()
```

CHAPTER 11: Combobox Widget in tkinter

Python provides a variety of GUI (Graphic User Interface) types such as PyQt, Tkinter, Kivy, WxPython, and PySide. Among them, `tkinter` is the most commonly used GUI module in Python since it is simple and easy to understand. The word Tkinter comes from the Tk interface. The `tkinter` module is available in Python standard library which has to be imported while writing a program in Python to generate a GUI.

Note: Tkinter (capital T) is different from the `tkinter`. Tkinter is used in Python 2.x and is changed to `tkinter` in Python 3.x.

Combobox is a combination of Listbox and an entry field. It is one of the Tkinter widgets where it contains a down arrow to select from a list of options. It helps the users to select according to the list of options displayed. When the user clicks on the drop-down arrow on the entry field, a pop up of the scrolled Listbox is displayed down the entry field. The selected option will be displayed in the entry field only when an option from the Listbox is selected.

Syntax:

```
combobox = ttk.Combobox(master, option=value, ...)
```

Example 1: Combobox widget without setting a default value.

```
# python program demonstrating
# Combobox widget using tkinter

import tkinter as tk

from tkinter import ttk

# Creating tkinter window

window = tk.Tk()

window.title('Combobox')

window.geometry('500x250')

# label text for title

ttk.Label(window, text = "GFG Combobox Widget",

          background = 'green', foreground ="white",

          font = ("Times New Roman", 15)).grid(row = 0, column = 1)

# label

ttk.Label(window, text = "Select the Month :",

          font = ("Times New Roman", 10)).grid(column = 0,

          row = 5, padx = 10, pady = 25)

# Combobox creation
```



```
n = tk.StringVar()

monthchoosen = ttk.Combobox(window, width = 27, textvariable = n)

# Adding combobox drop down list

monthchoosen['values'] = (' January',
                           ' February',
                           ' March',
                           ' April',
                           ' May',
                           ' June',
                           ' July',
                           ' August',
                           ' September',
                           ' October',
                           ' November',
                           ' December')

monthchoosen.grid(column = 1, row = 5)

monthchoosen.current()

window.mainloop()
```

Output:



Example 2: Combobox with initial default values.

We can also set the initial default values in the Combobox widget as shown in the below sample code.

```
import tkinter as tk

from tkinter import ttk

# Creating tkinter window

window = tk.Tk()

window.geometry('350x250')

# Label

ttk.Label(window, text = "Select the Month :",

          font = ("Times New Roman", 10)).grid(column = 0,

          row = 15, padx = 10, pady = 25)

n = tk.StringVar()

monthchoosen = ttk.Combobox(window, width = 27,

                             textvariable = n)
```

```
# Adding combobox drop down list

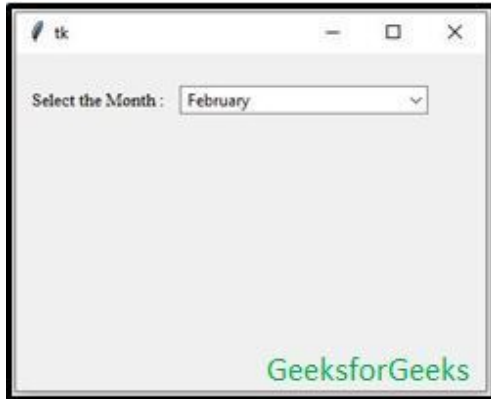
monthchoosen['values'] = (' January',
                           ' February',
                           ' March',
                           ' April',
                           ' May',
                           ' June',
                           ' July',
                           ' August',
                           ' September',
                           ' October',
                           ' November',
                           ' December')

monthchoosen.grid(column = 1, row = 15)

# Shows february as a default value
monthchoosen.current(1)

window.mainloop()
```

Output:



CHAPTER 12: maxsize() method in Tkinter

This method is used to set the **maximum size** of the root window (maximum size a window can be expanded). User will still be able to shrink the size of the window to the minimum possible. **Syntax :**
`master.maxsize(height, width)`

Here, height and width are in pixels. **Code #1:**

Python3

```
# importing only those functions
# which are needed

from tkinter import *

from tkinter.ttk import *

from time import strftime
```

```
# creating tkinter window

root = Tk()

# Adding widgets to the root window

Label(root, text = 'GeeksforGeeks',

      font =('Verdana', 15)).pack(side = TOP, pady = 10)

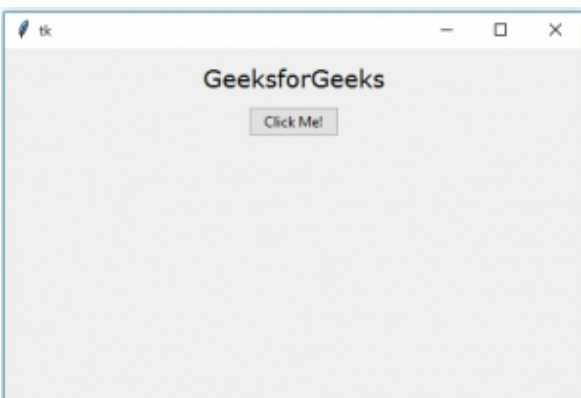
Button(root, text = 'Click Me !').pack(side = TOP)

mainloop()
```

Output : Initial size of the window (maximum size of the window is not set)



Expanded size of the window (this window can be expanded till the size of the screen because size is not fixed).



Code #2: Fixing maximum size of the root window

Python3

```
# importing only those functions
# which are needed

from tkinter import *

from tkinter.ttk import *

from time import strftime


# creating tkinter window

root = Tk()


# Fixing the size of the root window.
# No one can now expand the size of the
# root window than the specified one.
root.maxsize(200, 200)


# Adding widgets to the root window

Label(root, text = 'GeeksforGeeks',

      font=('Verdana', 15)).pack(side = TOP, pady = 10)


Button(root, text = 'Click Me !').pack(side = TOP)


mainloop()
```

Output : Maximum expanded size of the window



Note: [Tkinter](#) also offers a [minsize\(\)](#) method which is used to set the minimum size of the window.

CHAPTER 13: minsize() method in Tkinter

In Tkinter, **minsize()** method is used to set the minimum size of the [Tkinter](#) window. Using this method user can set window's initialized size to its minimum size, and still be able to maximize and scale the window larger.

Syntax:

```
master.minsize(width, height)
```

Here, height and width are in pixels.

Code #1: Root window without minimum size that means you can shrink window as much you want.

- Python3

```
# importing only those functions
```

```
# which are needed

from tkinter import *

from tkinter.ttk import *

from time import strftime

# creating tkinter window

root = Tk()

# Adding widgets to the root window

Label(root, text = 'GeeksforGeeks',

      font = ('Verdana', 15)).pack(side = TOP, pady = 10)

Button(root, text = 'Click Me !').pack(side = TOP)

mainloop()
```

Output: Initial root window without alteration in size



Root window after shrunk down, see the window is completely shrunk because it has no minimum [geometry](#).



Code #2: Root window with minimum size.

- Python3

```
# importing only those functions
# which are needed

from tkinter import *

from tkinter.ttk import *

from time import strftime

# creating tkinter window

root = Tk()

# setting the minimum size of the root window

root.minsize(150, 100)

# Adding widgets to the root window

Label(root, text = 'GeeksforGeeks',

      font=('Verdana', 15)).pack(side = TOP, pady = 10)

Button(root, text = 'Click Me !').pack(side = TOP)

mainloop()
```

Output: Initial window



Expanded window (we can expand window as much as we want because we haven't set the maximum size of the window).



Window shrunk to its minimum size (one cannot shrink it any further).



CHAPTER 14: `resizable()` method in Tkinter

`resizable()` method is used to allow [Tkinter](#) root window to change its size according to the users need as well we can prohibit resizing of the [Tkinter](#) window.

So, basically, if user wants to create a fixed size window, this method can be used.

How to use:

-> import tkinter

-> root = Tk()

-> root.resizable(height = None, width = None)

Arguments to be passed:

-> *In resizable() method user can pass either positive integer or True, to make the window resizable.*

-> *To make window non-resizable user can pass 0 or False.*

Code #1: Allowing root window to change it's size

```
# importing only those functions
```

```
# which are needed
```

```
from tkinter import *
```

```
from tkinter.ttk import *
```

```
from time import strftime
```

```
# creating tkinter window
```

```
root = Tk()
```

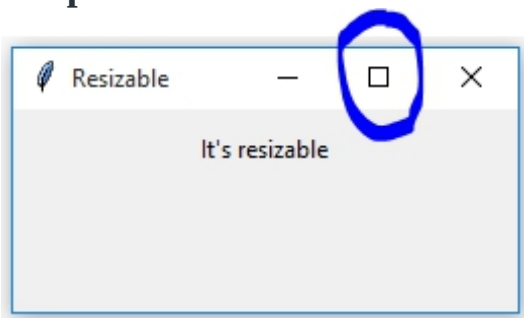
```
root.title('Resizable')
```

```
root.geometry('250x100')
```

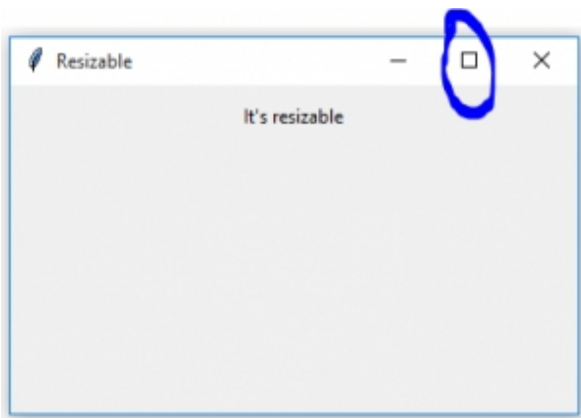
```
Label(root, text = 'It\'s resizable').pack(side = TOP, pady = 10)
```

```
# Allowing root window to change  
# it's size according to user's need  
root.resizable(True, True)  
  
mainloop()
```

Output:



Initial size – You may observe that the part inside blue circle is enabled i.e window is resizable and can be expanded. After resizing still the part inside the blue is enabled so you can still change the size of the window.



Code #2: Restricting root window to change it's size (fixed size window).

```
# importing only those functions  
# which are needed
```

```
from tkinter import *

from tkinter.ttk import *

from time import strftime

# creating tkinter window

root = Tk()

root.title('Resizable')

root.geometry('250x100')

Label(root, text = 'It's non-resizable').pack(side = TOP, pady = 10)

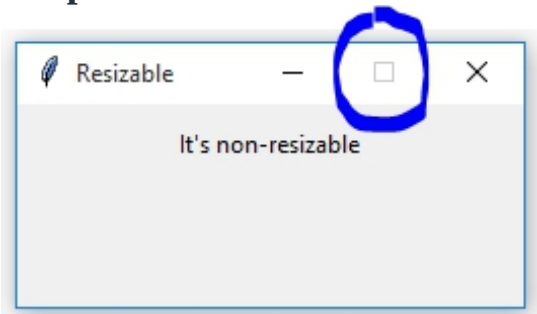
# Restricting root window to change

# it's size according to user's need

root.resizable(0, 0)

mainloop()
```

Output:



You may observe that the part inside the blue circle is disabled i.e size of the window cannot be altered.

CHAPTER 15: Python Tkinter – Entry Widget

Python offers multiple options for developing a GUI (Graphical User Interface). Out of all the GUI methods, Tkinter is the most commonly used method. Python with Tkinter is the fastest and easiest way to create GUI applications. Creating a GUI using Tkinter is an easy task. In Python3 Tkinter is come preinstalled But you can also install it by using the command:

```
pip install tkinter
```

Example: Now let's create a simple window using Tkinter

- Python3

```
# creating a simple tkinter window  
  
# if you are using python2  
  
# use import Tkinter as tk
```

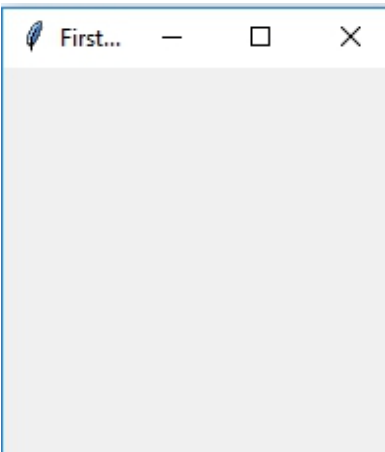
```
import tkinter as tk

root = tk.Tk()

root.title("First Tkinter Window")

root.mainloop()
```

Output :



The Entry Widget

The Entry Widget is a Tkinter Widget used to Enter or display a single line of text.

Syntax :

```
entry = tk.Entry(parent, options)
```

Parameters:

- 1) Parent:** The Parent window or frame in which the widget to display.
- 2) Options:** The various options provided by the entry widget are:

- **bg** : The normal background color displayed behind the label and indicator.
- **bd** : The size of the border around the indicator. Default is 2 pixels.
- **font** : The font used for the text.
- **fg** : The color used to render the text.
- **justify** : If the text contains multiple lines, this option controls how the text is justified: CENTER, LEFT, or RIGHT.
- **relief** : With the default value, relief=FLAT. You may set this option to any of the other styles like : SUNKEN, RIGID, RAISED, GROOVE
- **show** : Normally, the characters that the user types appear in the entry. To make a .password. entry that echoes each character as an asterisk, set show="*".
- **textvariable** : In order to be able to retrieve the current text from your entry widget, you must set this option to an instance of the StringVar class.

Methods: The various methods provided by the entry widget are:

- **get()** : Returns the entry's current text as a string.
- **delete()** : Deletes characters from the widget
- **insert (index, 'name')** : Inserts string 'name' before the character at the given index.

Example:

- Python3

```
# Program to make a simple
```

```
# login screen
```

```
import tkinter as tk
```



```
root=tk.Tk()

# setting the windows size
root.geometry("600x400")

# declaring string variable
# for storing name and password
name_var=tk.StringVar()
passw_var=tk.StringVar()

# defining a function that will
# get the name and password and
# print them on the screen
def submit():

    name=name_var.get()
    password=passw_var.get()

    print("The name is : " + name)

    print("The password is : " + password)

    name_var.set("")
    passw_var.set("")
```

```
# creating a label for
# name using widget Label

name_label = tk.Label(root, text = 'Username', font=('calibre',10, 'bold'))

# creating a entry for input
# name using widget Entry

name_entry = tk.Entry(root,textvariable = name_var, font=('calibre',10,'normal'))

# creating a label for password

passw_label = tk.Label(root, text = 'Password', font = ('calibre',10,'bold'))

# creating a entry for password

passw_entry=tk.Entry(root, textvariable = passw_var, font = ('calibre',10,'normal'), show = '*')

# creating a button using the widget
# Button that will call the submit function

sub_btn=tk.Button(root,text = 'Submit', command = submit)

# placing the label and entry in
# the required position using grid
# method

name_label.grid(row=0,column=0)

name_entry.grid(row=0,column=1)

passw_label.grid(row=1,column=0)

passw_entry.grid(row=1,column=1)
```

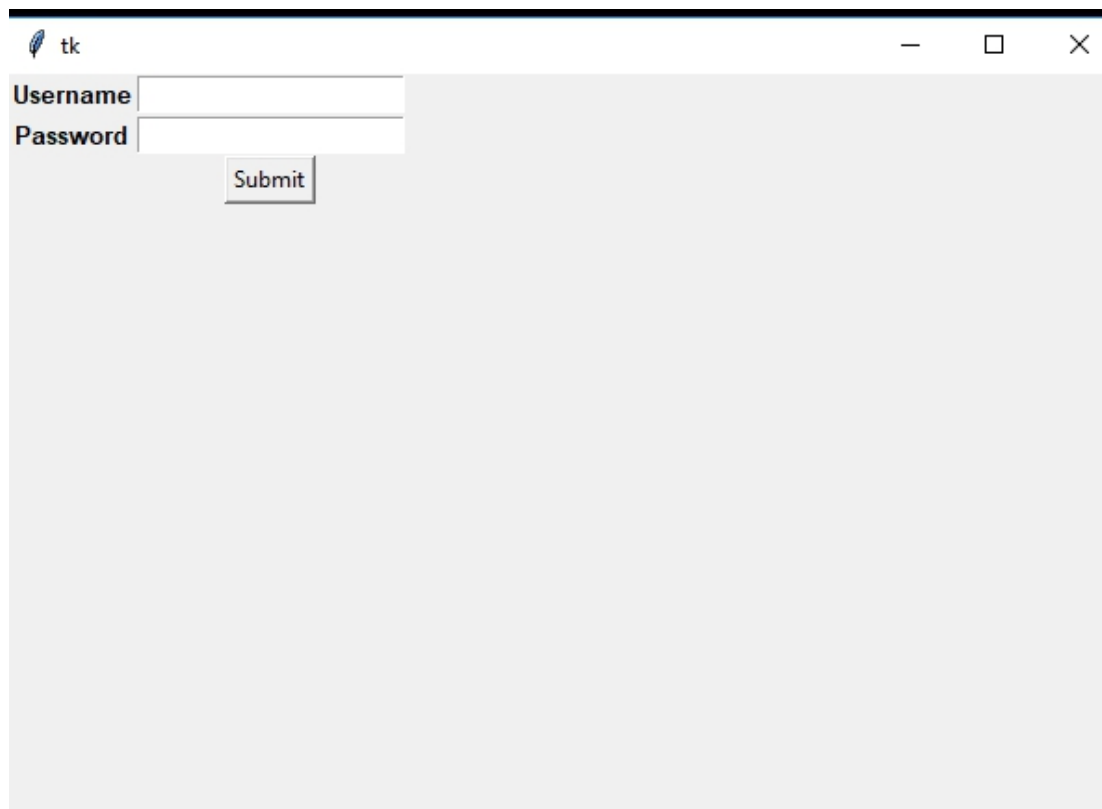
```
sub_btn.grid(row=2,column=1)
```

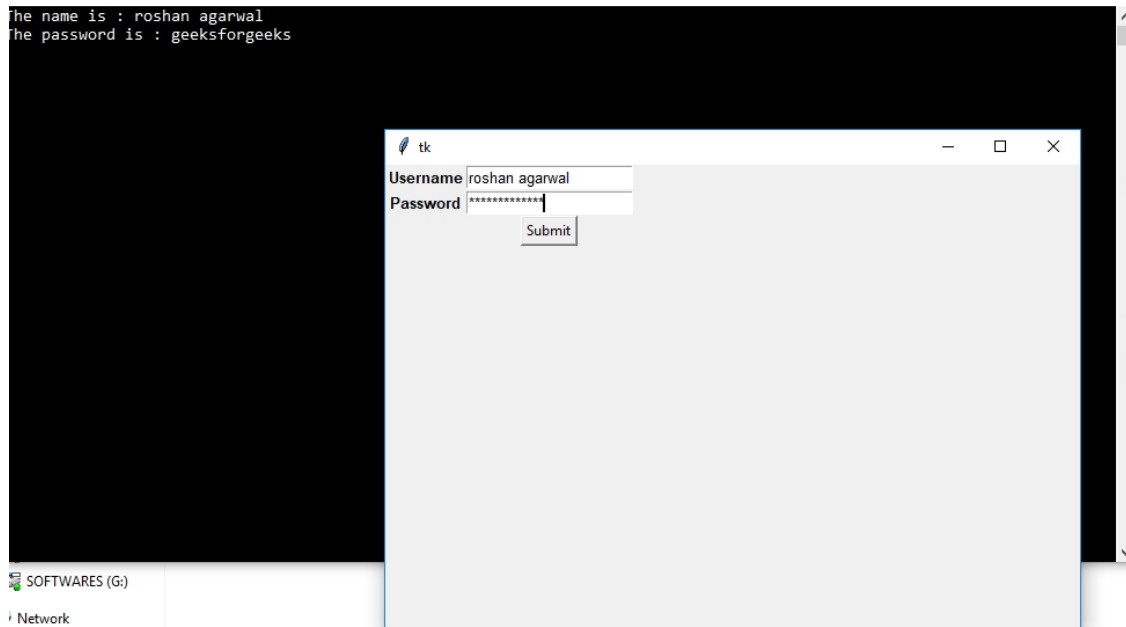
```
# performing an infinite loop
```

```
# for the window to display
```

```
root.mainloop()
```

Output :





CHAPTER 16: Tkinter – Read only Entry Widget

Python has a number of frameworks to develop GUI applications like PyQt, Kivy, Jython, WxPython, PyGUI, and Tkinter. Python [tkinter](#) module offers a variety of options to develop GUI based applications. Tkinter is an open-source and available under Python license. Tkinter provides the simplest and fastest way to develop a GUI application. Tkinter supports different widgets of which the Entry widget is used to accept input from the user. However, there are cases when a certain text needs to be in a read-only format to prevent alterations by the user. This can be achieved by the Entry widget and the various options available under the Entry widget.

In this article, we shall see the use of Tkinter variables. Python variables can be used Tkinter widgets but do not provide the flexibility as provided by Tkinter variables. [Tkinter variables](#) have a unique feature called ‘tracing’ that can be used to trace changes made to associated variables. This is useful to

track accidental changes made to a variable while working. The code below demonstrates the creation of a read-only Entry widget using Tkinter.

1. Import tkinter module

```
import tkinter
```

2. Import tkinter sub-module

```
from tkinter import *
```

3. Creating the parent widget

```
root = Tk()
```

Syntax: Tk(screenName=None, baseName=None, className='Tk', useTk=1)

Parameter: In this example, Tk class is instantiated without arguments.

Explanation:

This method creates a blank parent window with close, maximize, and minimize buttons on the top.

4. Creating Labels for the entry widgets and positioning the labels in the parent widget

```
5. L1 = Label(root, text="User Name")
```

```
6. L1.grid(row=0, column=0)
```

```
7. L2 = Label(root, text="Password")
```

```
L2.grid(row=1, column=0)
```

Syntax: Label(master, **options)

Parameter:

- **master:** The parent window (root) acts as the master.
- **options:** The options supported by Label() method are text, anchor, bg, bitmap, bd, cursor, font, fg, height, width, image, justify, relief, padx, pady, textvariable, underline and wraplength. Here the text option is used to display the name of the entry widget.

Explanation:

The Label widget is used to display text or images corresponding to a widget. The text displayed on the screen can further be formatted using the other options supported by the Label widget.

Syntax: grid(**options)

Parameter:

- **options:** The options available under the grid() method which can be used to alter the position of a widget in the parent widget are row, rowspan, column, columnspan, padx, pady, ipadx, ipady and sticky.

Explanation:

The grid() method splits the parent window into rows and columns just like a two dimensional table. Here grid() method specifies the position of the Label widget on the parent window.

8. Creating a Tkinter variable for the Entry widget

```
mystr = StringVar()
```

Syntax: StringVar()

Parameter: The constructor takes no argument. To set the value, set() method is used.

Explanation:

StringVar is one of the inbuilt variable classes in Tkinter. The default value of StringVar() is an empty string “” .

9. Setting the string value

```
mystr.set('username@xyz.com')
```

Syntax: set(string)

Parameter:

- **string:** Represents the text to be associated with the widget (here ‘entry’ widget)

Explanation:

Since StringVar class constructor accepts no argument during object creation, the set() method is used to change the value of the StringVar class variable. This method is called whenever the string value needs to be changed and the changed value is reflected automatically in the associated widget.

10.

Creating Entry widget

```
entry = Entry(textvariable=mystr, state=DISABLED).grid(row=0,  
column=1, padx=10, pady=10)  
mystr.set('username@xyz.com')  
passwd = Entry().grid(row=1, column=1, padx=10, pady=10)
```

Syntax: Entry(master, **options)

Parameter:

- **master:** Represents the parent widget (here root) .
- **options:** The options available under Entry widget are bg, bd, command, cursor, font, exportselection, justify, relief, highlightcolor, fg, selectbackground, selectforeground, selectborderwidth, show, xscrollcommand, state, textvariable and width.

Explanation:

This method creates a Entry widget on the parent widget. The Entry widget 'entry' is in disabled state which implies that it is in read-only mode and cannot be changed by the user. However, the Entry widget 'passwd' is in normal state and accepts input from user which can be changed if required. The grid() method positions the entry widget in the parent widget.

11.

Run the application

```
mainloop()
```

Syntax: `mainloop()`

Explanation:

The `mainloop()` basically acts like an infinite loop. It is used to run an application.

Complete Program

```
import tkinter

from tkinter import *

root = Tk()

L1 = Label(root, text="User Name")
L1.grid(row=0,column=0)

L2 = Label(root, text="Password")
L2.grid(row=1,column=0)

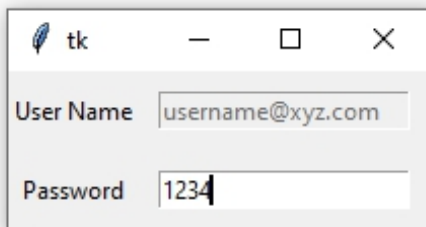
mystr = StringVar()
mystr.set('username@xyz.com')

entry = Entry(textvariable=mystr,
               state=DISABLED).grid(row=0,
                                     column=1,
```



```
        padx=10,  
        pady=10)  
  
passwd = Entry().grid(row=1,column=1,  
        padx=10,pady=10)  
  
mainloop()
```

Output



CHAPTER 17: Python Tkinter – Text Widget

Tkinter is a GUI toolkit used in python to make user-friendly GUIs. Tkinter is the most commonly used and the most basic GUI framework available in python. Tkinter uses an object-oriented approach to make GUIs.

Note: For more information, refer to [Python GUI – tkinter](#)

Text Widget

Text Widget is used where a user wants to insert multiline text fields. This widget can be used for a variety of applications where the multiline text is required such as messaging, sending information or displaying information and many other tasks. We can insert media files such as images and links also in the Textwidget.

Syntax:

T = Text(root, bg, fg, bd, height, width, font, ..)

Optional parameters

- **root** – root window.
- **bg** – background colour
- **fg** – foreground colour
- **bd** – border of widget.
- **height** – height of the widget.
- **width** – width of the widget.
- **font** – Font type of the text.
- **cursor** – The type of the cursor to be used.
- **insetofftime** – The time in milliseconds for which the cursor blink is off.
- **insertontime** – the time in milliseconds for which the cursor blink is on.
- **padx** – horizontal padding.
- **pady** – vertical padding.
- **state** – defines if the widget will be responsive to mouse or keyboards movements.
- **highlightthickness** – defines the thickness of the focus highlight.

- **insertionwidth** – defines the width of insertion character.
- **relief** – type of the border which can be SUNKEN, RAISED, GROOVE and RIDGE.
- **yscrollcommand** – to make the widget vertically scrollable.
- **xscrollcommand** – to make the widget horizontally scrollable.

Some Common methods

- **index(index)** – To get the specified index.
- **insert(index)** – To insert a string at a specified index.
- **see(index)** – Checks if a string is visible or not at a given index.
- **get(startindex, endindex)** – to get characters within a given range.
- **delete(startindex, endindex)** – deletes characters within specified range.

Tag handling methods

- **tag_delete(tagname)** – To delete a given tag.
- **tag_add(tagname, startindex, endindex)** – to tag the string in the specified range
- **tag_remove(tagname, startindex, endindex)** – to remove a tag from specified range

Mark handling methods

- **mark_names()** – to get all the marks in the given range.
- **index(mark)** – to get index of a mark.
- **mark_gravity()** – to get the gravity of a given mark.

Example 1:

Python3

```
import tkinter as tk

root = Tk()

# specify size of window.
root.geometry("250x170")

# Create text widget and specify size.
T = Text(root, height = 5, width = 52)

# Create label
l = Label(root, text = "Fact of the Day")
l.config(font =("Courier", 14))

Fact = """A man can be arrested in
Italy for wearing a skirt in public."""
```

```
# Create button for next text.

b1 = Button(root, text = "Next", )

# Create an Exit button.

b2 = Button(root, text = "Exit",

             command = root.destroy)

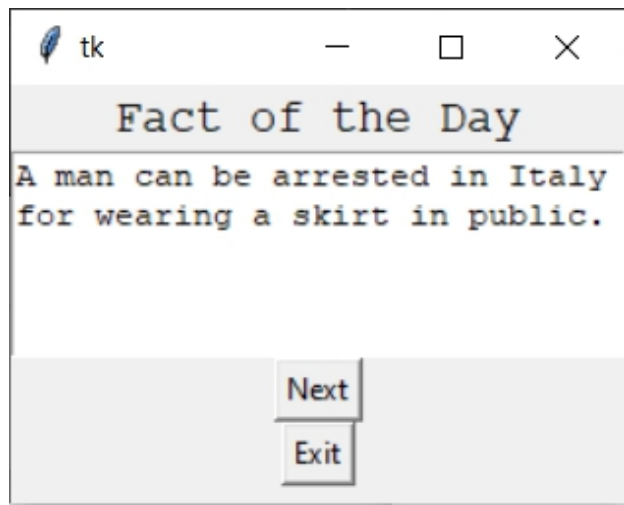
l.pack()
T.pack()
b1.pack()
b2.pack()

# Insert The Fact.

T.insert(tk.END, Fact)

tk.mainloop()
```

Output



Example 2: Saving Text and performing operations

Python3

```
from tkinter import *

root = Tk()

root.geometry("300x300")

root.title(" Q&A ")

def Take_input():

    INPUT = inputtxt.get("1.0", "end-1c")

    print(INPUT)

    if(INPUT == "120"):

        Output.insert(END, 'Correct')

    else:

        Output.insert(END, "Wrong answer")
```

```
l = Label(text = "What is 24 * 5 ? ")

inputtxt = Text(root, height = 10,

                 width = 25,

                 bg = "light yellow")

Output = Text(root, height = 5,

               width = 25,

               bg = "light cyan")

Display = Button(root, height = 2,

                 width = 20,

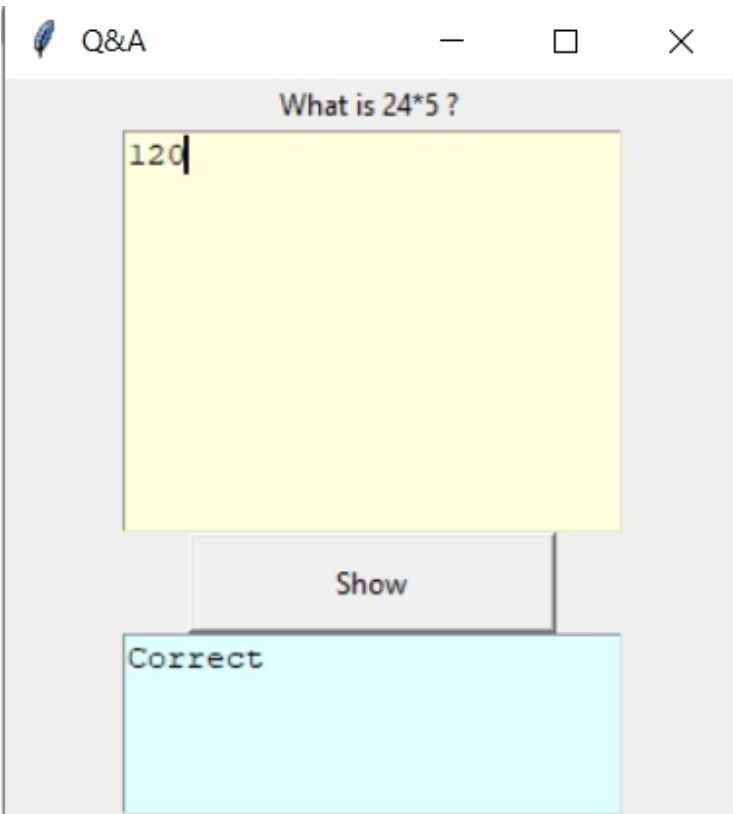
                 text = "Show",

                 command = lambda:Take_input())

l.pack()
inputtxt.pack()
Display.pack()
Output.pack()

mainloop()
```

Output



CHAPTER 18: Python Tkinter – Message

Python offers multiple options for developing a GUI (Graphical User Interface). Out of all the GUI methods, Tkinter is the most commonly used method. It is a standard Python interface to the Tk GUI toolkit shipped with Python. Python with Tkinter is the fastest and easiest way to create GUI applications. Creating a GUI using Tkinter is an easy task.

Note: For more information, refer to [Python GUI – tkinter](#)

Message widget

The Message widget is used to show the message to the user regarding the behavior of the python application. The message text contains more than one line.

Syntax:

The syntax to use the message is given below.

```
w = Message( master, options)
```

Parameters:

- **master:** This parameter is used to represents the parent window.
- **options:** There are many options which are available and they can be used as key-value pairs separated by commas.

Options:

Following are commonly used Option can be used with this widget :-

- **anchor:** This option is used to decide the exact position of the text within the space .Its default value is CENTER.
- **bg:** This option used to represent the normal background color.
- **bitmap:** This option used to display a monochrome image.
- **bd:** This option used to represent the size of the border and the default value is 2 pixels.
- **cursor:** By using this option, the mouse cursor will change to that pattern when it is over type.
- **font:** This option used to represent the font used for the text.
- **fg:** This option used to represent the color used to render the text.
- **height:** This option used to represent the number of lines of text on the message.
- **image:** This option used to display a graphic image on the widget.
- **justify:** This option used to control how the text is justified: CENTER, LEFT, or RIGHT.
- **padx:** This option used to represent how much space to leave to the left and right of the widget and text. It's default value is 1 pixel.
- **pady:** This option used to represent how much space to leave above and below the widget. It's default value is 1 pixel.

- **relief:** The type of the border of the widget. It's default value is set to FLAT.
- **text:** This option used use newlines (“\n”) to display multiple lines of text.
- **variable:** This option used to represents the associated variable that tracks the state of the widget.
- **width:** This option used to represents the width of the widget. and also represented in the number of characters that are represented in the form of texts.
- **wraplength:** This option will be broken text into the number of pieces.

Example:

```
from tkinter import *

root = Tk()
root.geometry("300x200")

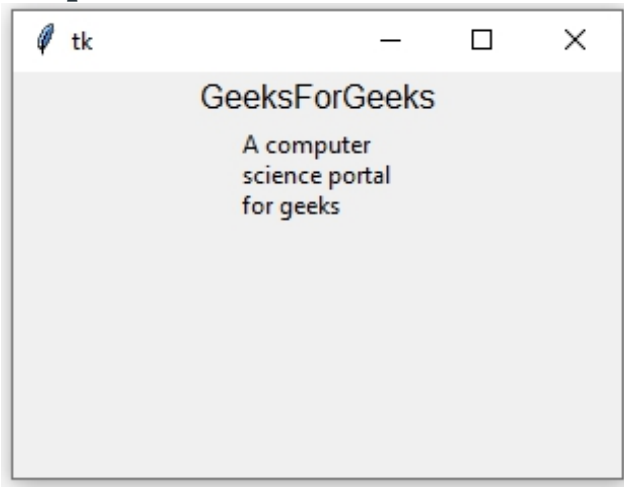
w = Label(root, text ='GeeksForGeeks', font = "50")
w.pack()

msg = Message( root, text = "A computer science portal for geeks")

msg.pack()

root.mainloop()
```

Output:



CHAPTER 19: Menu widget in Tkinter

[Tkinter](#) is Python's standard GUI (Graphical User Interface) package. It is one of the most commonly used package for GUI applications which comes with the Python itself.

Menus are the important part of any GUI. A common use of menus is to provide convenient access to various operations such as saving or opening a file, quitting a program, or manipulating data. Toplevel menus are displayed just under the title bar of the root or any other toplevel windows.

```
menu = Menu(master, **options)
```

Below is the implementation:

```
# importing only those functions  
# which are needed
```

```
from tkinter import *

from tkinter.ttk import *

from time import strftime

# creating tkinter window

root = Tk()

root.title('Menu Demonstration')

# Creating Menubar

menubar = Menu(root)

# Adding File Menu and commands

file = Menu(menubar, tearoff = 0)

menubar.add_cascade(label = 'File', menu = file)

file.add_command(label = 'New File', command = None)

file.add_command(label = 'Open...', command = None)

file.add_command(label = 'Save', command = None)

file.add_separator()

file.add_command(label = 'Exit', command = root.destroy)

# Adding Edit Menu and commands

edit = Menu(menubar, tearoff = 0)

menubar.add_cascade(label = 'Edit', menu = edit)
```

```
edit.add_command(label='Cut', command = None)

edit.add_command(label='Copy', command = None)

edit.add_command(label='Paste', command = None)

edit.add_command(label='Select All', command = None)

edit.add_separator()

edit.add_command(label='Find...', command = None)

edit.add_command(label='Find again', command = None)


# Adding Help Menu

help_ = Menu(menuubar, tearoff = 0)

menuubar.add_cascade(label='Help', menu = help_)

help_.add_command(label='Tk Help', command = None)

help_.add_command(label='Demo', command = None)

help_.add_separator()

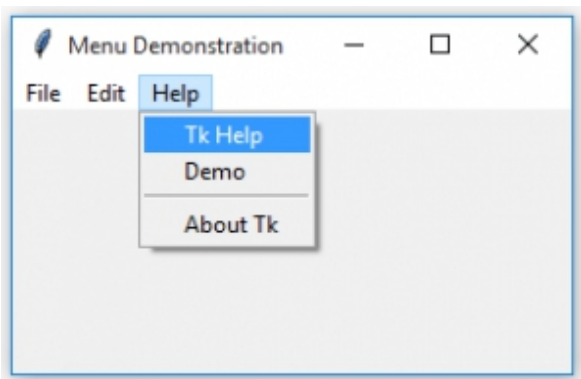
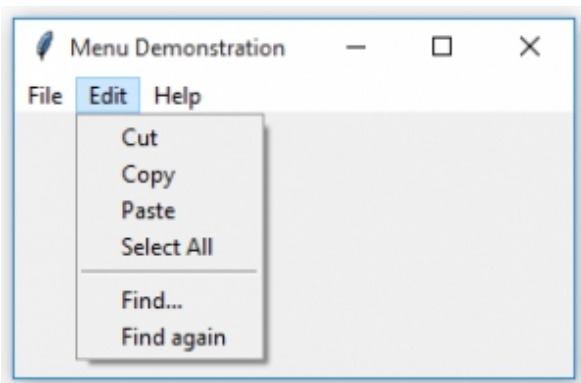
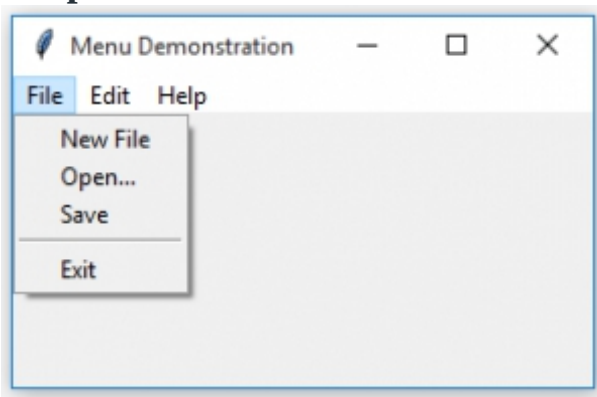
help_.add_command(label='About Tk', command = None)


# display Menu

root.config(menu = menuubar)

mainloop()
```

Output:



Note: In above application, commands are set to None but one may add different commands to different labels to perform the required task.

CHAPTER 20: Python Tkinter – Menubutton Widget

Python offers multiple options for developing GUI (Graphical User Interface). Out of all the GUI methods, tkinter is the most commonly used method. It is a standard Python interface to the Tk GUI toolkit shipped with Python. Python with tkinter is the fastest and easiest way to create the GUI applications. Creating a GUI using tkinter is an easy task.

Note: For more information, refer to [Python GUI – tkinter](#)

Menubutton widget

The Menubutton widget can be defined as the drop-down menu that is shown to the user all the time. The Menubutton is used to implement various types of menus in the python application.

Syntax:

```
w = Menubutton ( master, options )
```

Parameters:

- **master:** This parameter is used to represents the parent window.
- **options:** There are many options which are available and they can be used as key-value pairs separated by commas.

Options:

Following are commonly used Option can be used with this widget :-

- **activebackground:** This option used to represent the background color when the Menubutton is under the cursor.
- **activeforeground:** This option used to represent the foreground color when the Menubutton is under the cursor.
- **bg:** This option used to represent the normal background color displayed behind the label and indicator.
- **bitmap:** This option used to display a monochrome image on a button.

- **bd:** This option used to represent the size of the border around the indicator and the default value is 2 pixels.
- **anchor:** This option specifies the exact position of the widget content when the widget is assigned more space than needed.
- **cursor:** By using this option, the mouse cursor will change to that pattern when it is over the Menubutton.
- **disabledforeground:** The foreground color used to render the text of a disabled Menubutton. The default is a stippled version of the default foreground color.
- **direction:** Its direction can be specified so that menu can be displayed to the specified direction of the button.
- **fg:** This option used to represent the color used to render the text.
- **height:** This option used to represent the number of lines of text on the Menubutton and its default value is 1.
- **highlightcolor:** This option used to represent the color of the focus highlight when the Menubutton has the focus.
- **image:** This option used to display a graphic image on the button.
- **justify:** This option used to control how the text is justified: CENTER, LEFT, or RIGHT.
- **menu:** It represents the menu specified with the Menubutton.
- **padx:** This option used to represent how much space to leave to the left and right of the Menubutton and text. Its default value is 1 pixel.
- **pady:** This option used to represent how much space to leave above and below the Menubutton and text. Its default value is 1 pixel.
- **relief:** The type of the border of the Menubutton. Its default value is set to FLAT.
- **state:** It represents the state of the Menubutton. By default, it is set to normal. We can change it to DISABLED to make the Menubutton unresponsive. The state of the Menubutton is ACTIVE when it is under focus.
- **text:** This option used use newlines (“\n”) to display multiple lines of text.
- **underline:** This option used to represent the index of the character in the text which is to be underlined. The indexing starts with zero in the text.
- **textvariable:** This option used to represents the associated variable that tracks the state of the Menubutton.

- **width:** This option used to represents the width of the Menubutton. and also represented in the number of characters that are represented in the form of texts.
- **wraplength:** This option will be broken text into the number of pieces.

Example:

```
from tkinter import *

root = Tk()
root.geometry("300x200")

w = Label(root, text ='GeeksForGeeks', font = "50")
w.pack()

menubutton = Menubutton(root, text = "Menu")

menubutton.menu = Menu(menubutton)
menubutton["menu"]= menubutton.menu

var1 = IntVar()
var2 = IntVar()
var3 = IntVar()

menubutton.menu.add_checkbutton(label = "Courses",
```

```
variable = var1)

menubutton.menu.add_checkbutton(label = "Students",

variable = var2)

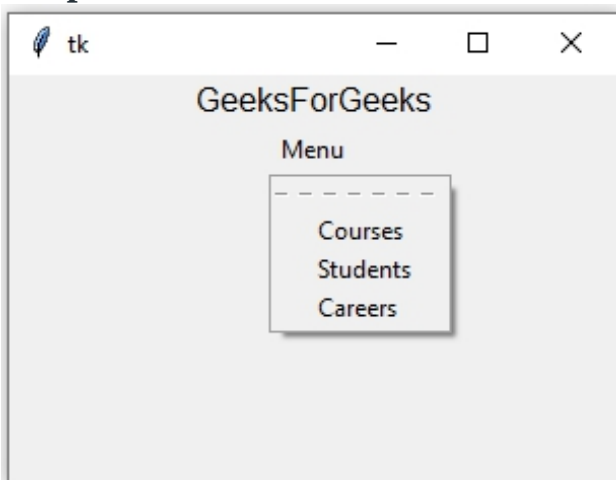
menubutton.menu.add_checkbutton(label = "Careers",

variable = var3)

menubutton.pack()

root.mainloop()
```

Output:



CHAPTER 21: Python Tkinter – SpinBox

Python offers multiple options for developing GUI (Graphical User Interface). Out of all the GUI methods, tkinter is the most commonly used method. It is a standard Python interface to the Tk GUI toolkit shipped with

Python. Python with tkinter is the fastest and easiest way to create the GUI applications. Creating a GUI using tkinter is an easy task.

Note: For more information, refer to [Python GUI – tkinter](#)

Spinbox widget

The Spinbox widget is used to select from a fixed number of values. It is an alternative Entry widget and provides the range of values to the user.

Syntax:

The syntax to use the Spinbox is given below.

```
w = Spinbox ( master, options)
```

Parameters:

- **master:** This parameter is used to represents the parent window.
- **options:** There are many options which are available and they can be used as key-value pairs separated by commas.

Options:

Following are commonly used Option can be used with this widget :-

- **activebackground:** This option used to represent the background color when the slider and arrowheads is under the cursor.
- **bg:** This option used to represent the normal background color displayed behind the label and indicator.
- **bd:** This option used to represent the size of the border around the indicator and the default value is 2 pixels.
- **command:** This option is associated with a function to be called when the state is changed.
- **cursor:** By using this option, the mouse cursor will change to that pattern when it is over the type.
- **disabledforeground:** This option used to represent the foreground color of the widget when it is disabled..
- **disabledbackground:** This option used to represent the background color of the widget when it is disabled..
- **font:** This option used to represent the font used for the text.

- **fg:** This option used to represent the color used to render the text.
- **format:** This option used to formatting the string and it's has no default value.
- **from_:** This option used to represent the minimum value.
- **justify:** This option used to control how the text is justified: CENTER, LEFT, or RIGHT.
- **relief:** This option used to represent the type of the border and It's default value is set to SUNKEN.
- **repeatdelay:** This option is used to control the button auto repeat and its default value is in milliseconds.
- **repeatinterval:** This option is similar to repeatdelay.
- **state:** This option used to represent the represents the state of the widget and its default value is NORMAL.
- **textvariable:** This option used to control the behaviour of the widget text.
- **to:** It specify the maximum limit of the widget value. The other is specified by the from_ option.
- **validate:** This option is used to control how the widget value is validated.
- **validatecommand:** This option is associated to the function callback which is used for the validation of the widget content.
- **values:** This option used to represent the tuple containing the values for this widget.
- **vcmd:** This option is same as validation command.
- **width:** This option is used to represents the width of the widget.
- **wrap:** This option wraps up the up and down button the Spinbox.
- **xscrollcommand:** This options is set to the set() method of scrollbar to make this widget horizontally scrollable.

Methods:

Methods used in this widgets are as follows:

- **delete(startindex, endindex):** This method is used to delete the characters present at the specified range.
- **get(startindex, endindex):** This method is used to get the characters present in the specified range.
- **identify(x, y):** This method is used to identify the widget's element within the specified range.

- **index(index):** This method is used to get the absolute value of the given index.
- **insert(index, string):** This method is used to insert the string at the specified index.
- **invoke(element):** This method is used to invoke the callback associated with the widget.

Example:

```
from tkinter import *

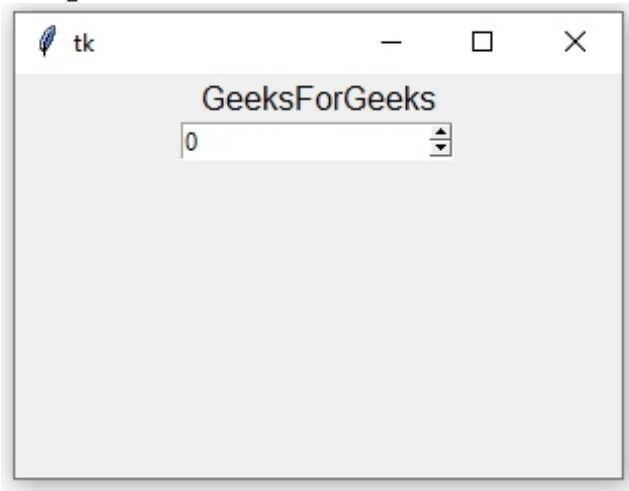
root = Tk()
root.geometry("300x200")

w = Label(root, text='GeeksForGeeks', font = "50")
w.pack()

sp = Spinbox(root, from_ = 0, to = 20)
sp.pack()

root.mainloop()
```

Output:



CHAPTER 22: Progressbar widget in Tkinter

The purpose of this widget is to reassure the user that something is happening. It can operate in one of two modes –

In **determinate mode**, the widget shows an indicator that moves from beginning to end under program control.

In **indeterminate mode**, the widget is animated so the user will believe that something is in progress. In this mode, the indicator bounces back and forth between the ends of the widget.

Syntax:

```
widget_object = Progressbar(parent, **options)
```

Code #1 In determinate mode

```
# importing tkinter module

from tkinter import * from tkinter.ttk import *
```

```
# creating tkinter window

root = Tk()


# Progress bar widget

progress = Progressbar(root, orient = HORIZONTAL,

                        length = 100, mode = 'determinate')


# Function responsible for the updation
# of the progress bar value

def bar():

    import time

    progress['value'] = 20

    root.update_idletasks()

    time.sleep(1)


    progress['value'] = 40

    root.update_idletasks()

    time.sleep(1)


    progress['value'] = 50

    root.update_idletasks()

    time.sleep(1)


    progress['value'] = 60
```

```
root.update_idletasks()

time.sleep(1)

progress['value'] = 80

root.update_idletasks()

time.sleep(1)

progress['value'] = 100

progress.pack(pady = 10)

# This button will initialize
# the progress bar

Button(root, text = 'Start', command = bar).pack(pady = 10)

# infinite loop

mainloop()
```

Code #2: In indeterminate mode

```
# importing tkinter module

from tkinter import * from tkinter.ttk import *

# creating tkinter window
```



```
root = Tk()

# Progress bar widget

progress = Progressbar(root, orient = HORIZONTAL,

                        length = 100, mode = 'indeterminate')

# Function responsible for the updation
# of the progress bar value

def bar():

    import time

    progress['value'] = 20

    root.update_idletasks()

    time.sleep(0.5)

    progress['value'] = 40

    root.update_idletasks()

    time.sleep(0.5)

    progress['value'] = 50

    root.update_idletasks()

    time.sleep(0.5)

    progress['value'] = 60

    root.update_idletasks()
```

```
time.sleep(0.5)
```

```
    progress['value'] = 80
```

```
    root.update_idletasks()
```

```
    time.sleep(0.5)
```

```
    progress['value'] = 100
```

```
    root.update_idletasks()
```

```
    time.sleep(0.5)
```

```
    progress['value'] = 80
```

```
    root.update_idletasks()
```

```
    time.sleep(0.5)
```

```
    progress['value'] = 60
```

```
    root.update_idletasks()
```

```
    time.sleep(0.5)
```

```
    progress['value'] = 50
```

```
    root.update_idletasks()
```

```
    time.sleep(0.5)
```

```
    progress['value'] = 40
```

```
    root.update_idletasks()
```

```
    time.sleep(0.5)
```

```
        progress['value'] = 20

    root.update_idletasks()

    time.sleep(0.5)

    progress['value'] = 0

progress.pack(pady = 10)

# This button will initialize
# the progress bar
Button(root, text = 'Start', command = bar).pack(pady = 10)

# infinite loop
mainloop()
```

CHAPTER 23: Python Tkinter - Scrollbar

Python offers multiple options for developing a GUI (Graphical User Interface). Out of all the GUI methods, Tkinter is the most commonly used method. It is a standard Python interface to the Tk GUI toolkit shipped with

Python. Python with Tkinter is the fastest and easiest way to create GUI applications. Creating a GUI using Tkinter is an easy task.

Note: For more information, refer to [Python GUI – tkinter](#)

Scrollbar Widget

The scrollbar widget is used to scroll down the content. We can also create the horizontal scrollbars to the Entry widget.

Syntax:

The syntax to use the Scrollbar widget is given below.

```
w = Scrollbar(master, options)
```

Parameters:

- **master:** This parameter is used to represents the parent window.
- **options:** There are many options which are available and they can be used as key-value pairs separated by commas.

Options:

Following are commonly used Option can be used with this widget :-

- **activebackground:** This option is used to represent the background color of the widget when it has the focus.
- **bg:** This option is used to represent the background color of the widget.
- **bd:** This option is used to represent the border width of the widget.
- **command:** This option can be set to the procedure associated with the list which can be called each time when the scrollbar is moved.
- **cursor:** In this option, the mouse pointer is changed to the cursor type set to this option which can be an arrow, dot, etc.
- **elementborderwidth:** This option is used to represent the border width around the arrow heads and slider. The default value is -1.
- **Highlightbackground:** This option is used to focus highlightcolor when the widget doesn't have the focus.
- **highlightcolor:** This option is used to focus highlightcolor when the widget has the focus.
- **highlightthickness:** This option is used to represent the thickness of the focus highlight.

- **jump:** This option is used to control the behavior of the scroll jump. If it set to 1, then the callback is called when the user releases the mouse button.
- **orient:** This option can be set to HORIZONTAL or VERTICAL depending upon the orientation of the scrollbar.
- **repeatdelay:** This option tells the duration up to which the button is to be pressed before the slider starts moving in that direction repeatedly. The default is 300 ms.
- **repeatinterval:** The default value of the repeat interval is 100.
- **takefocus:** You can tab the focus through a scrollbar widget
- **troughcolor:** This option is used to represent the color of the trough.
- **width:** This option is used to represent the width of the scrollbar.

Methods:

Methods used in this widgets are as follows:

- **get():** This method is used to returns the two numbers a and b which represents the current position of the scrollbar.
- **set(first, last):** This method is used to connect the scrollbar to the other widget w. The yscrollcommand or xscrollcommand of the other widget to this method.

Example:

```
from tkinter import *

root = Tk()

root.geometry("150x200")

w = Label(root, text ='GeeksForGeeks',

          font = "50")

w.pack()
```

```
scroll_bar = Scrollbar(root)

scroll_bar.pack( side = RIGHT,

                fill = Y )

mylist = Listbox(root,

                yscrollcommand = scroll_bar.set )

for line in range(1, 26):

    mylist.insert(END, "Geeks " + str(line))

mylist.pack( side = LEFT, fill = BOTH )

scroll_bar.config( command = mylist.yview )

root.mainloop()
```

Output:



CHAPTER 24: Python Tkinter – ScrolledText Widget

[Tkinter](#) is a built-in standard python library. With the help of Tkinter, many GUI applications can be created easily. There are various types of widgets available in Tkinter such as button, frame, label, menu, scrolledtext, canvas and many more. A widget is an element that provides various controls.

ScrolledText widget is a text widget with a scroll bar. The `tkinter.scrolledtext` module provides the text widget along with a scroll bar. This widget helps the user enter multiple lines of text with convenience. Instead of adding a Scroll bar to a text widget, we can make use of a scrolledtext widget that helps to enter any number of lines of text.

Example 1 : Python code displaying scrolledText widget.

```
# Python program demonstrating  
# ScrolledText widget in tkinter
```

[illegible]

15))

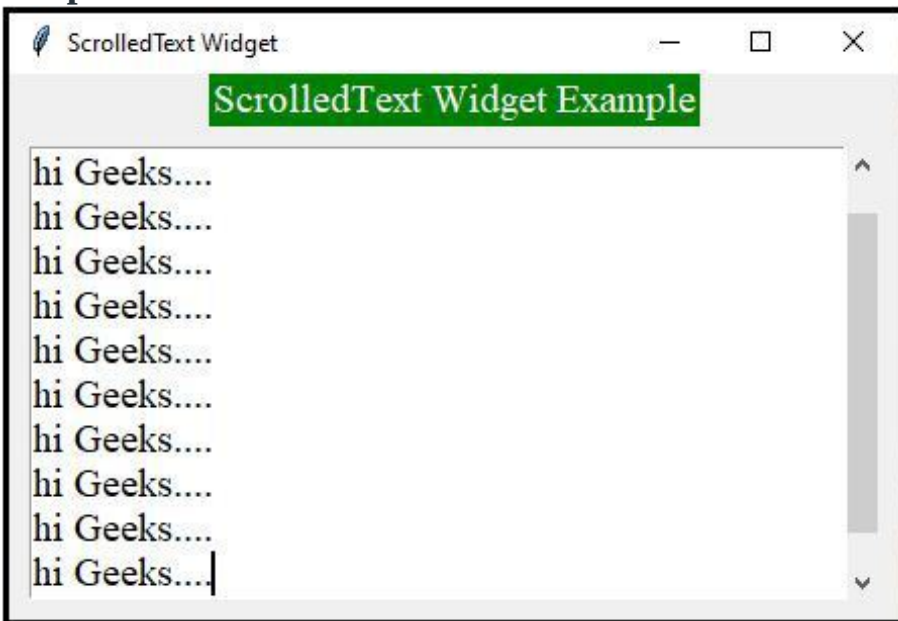
```
text_area.grid(column = 0, pady = 10, padx = 10)
```

```
# Placing cursor in the text area
```

```
text_area.focus()
```

```
win.mainloop()
```

Output :



Example 2 : ScrolledText widget making tkinter text Read only.

```
# Importing required modules
```

```
import tkinter as tk
```

```
import tkinter.scrolledtext as st
```

```
# Creating tkinter window

win = tk.Tk()

win.title("ScrolledText Widget")


# Title Label

tk.Label(win,

        text = "ScrolledText Widget Example",

        font = ("Times New Roman", 15),

        background = 'green',

        foreground = "white").grid(column = 0,

                                    row = 0)


# Creating scrolled text area

# widget with Read only by

# disabling the state

text_area = st.ScrolledText(win,

                             width = 30,

                             height = 8,

                             font = ("Times New Roman",

                                     15))

text_area.grid(column = 0, pady = 10, padx = 10)

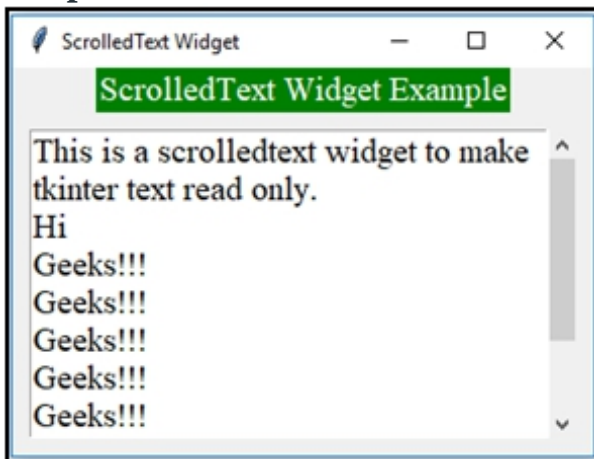

# Inserting Text which is read only
```

```
text_area.insert(tk.INSERT,
"""\
This is a scrolledtext widget to make tkinter text read only.
Hi
Geeks !!!
Geeks !!!
Geeks !!!
Geeks !!!
Geeks !!!
Geeks !!!
Geeks !!!
Geeks !!!
""")

# Making the text read only
text_area.configure(state='disabled')

win.mainloop()
```

Output :



In the first example, as you can see the cursor, the user can enter any number of lines of text. In the second example, the user can just read the text which is displayed in the text box and cannot edit/enter any lines of text. We may observe that the scroll bar disappears automatically if the text entered by the user is less than the size of the widget.

CHAPTER 25: Python Tkinter – ListBox Widget

Tkinter is a GUI toolkit used in python to make user-friendly GUIs. Tkinter is the most commonly used and the most basic GUI framework available in python. Tkinter uses an object-oriented approach to make GUIs.

Note: For more information, refer to [Python GUI – tkinter](#)

Listbox widget

The ListBox widget is used to display different types of items. These items must be of the same type of font and having the same font color. The items must also be of Text type. The user can select one or more items from the given list according to the requirement.

Syntax:

```
listbox = Listbox(root, bg, fg, bd, height, width, font, ..)
```

Optional parameters

- **root** – root window.
- **bg** – background colour
- **fg** – foreground colour
- **bd** – border
- **height** – height of the widget.
- **width** – width of the widget.

- **font** – Font type of the text.
- **highlightcolor** – The colour of the list items when focused.
- **yscrollcommand** – for scrolling vertically.
- **xscrollcommand** – for scrolling horizontally.
- **cursor** – The cursor on the widget which can be an arrow, a dot etc.

Common methods

- **yview** – allows the widget to be vertically scrollable.
- **xview** – allows the widget to be horizontally scrollable.
- **get()** – to get the list items in a given range.
- **activate(index)** – to select the lines with a specified index.
- **size()** – return the number of lines present.
- **delete(start, last)** – delete lines in the specified range.
- **nearest(y)** – returns the index of the nearest line.
- **curseselection()** – returns a tuple for all the line numbers that are being selected.

Example 1:

Python3

```
from tkinter import *

# create a root window.

top = Tk()

# create listbox object

listbox = Listbox(top, height = 10,

                  width = 15,

                  bg = "grey",

                  activestyle = 'dotbox',
```

```
        font = "Helvetica",

        fg = "yellow")

# Define the size of the window.

top.geometry("300x250")

# Define a label for the list.

label = Label(top, text = " FOOD ITEMS")

# insert elements by their
# index and names.

listbox.insert(1, "Nachos")

listbox.insert(2, "Sandwich")

listbox.insert(3, "Burger")

listbox.insert(4, "Pizza")

listbox.insert(5, "Burrito")

# pack the widgets

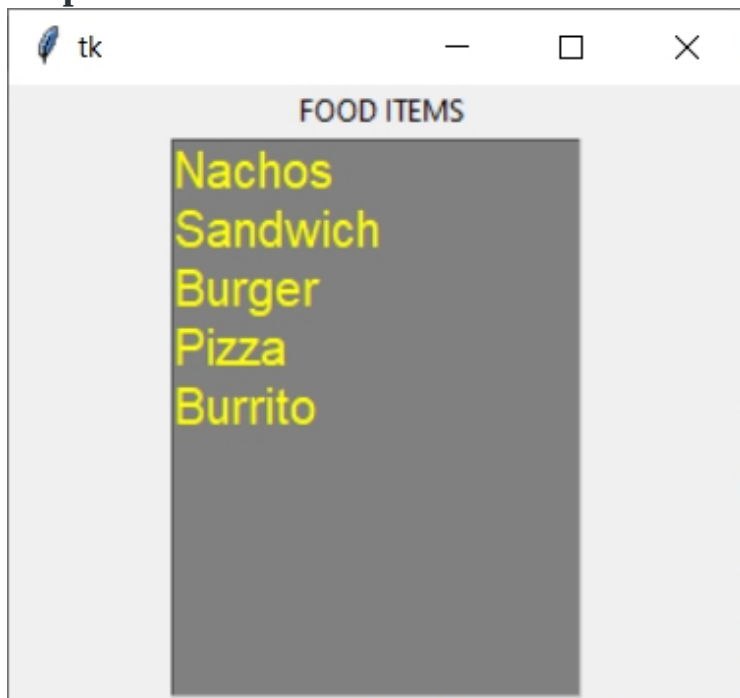
label.pack()

listbox.pack()

# Display until User
# exits themselves.

top.mainloop()
```

Output:

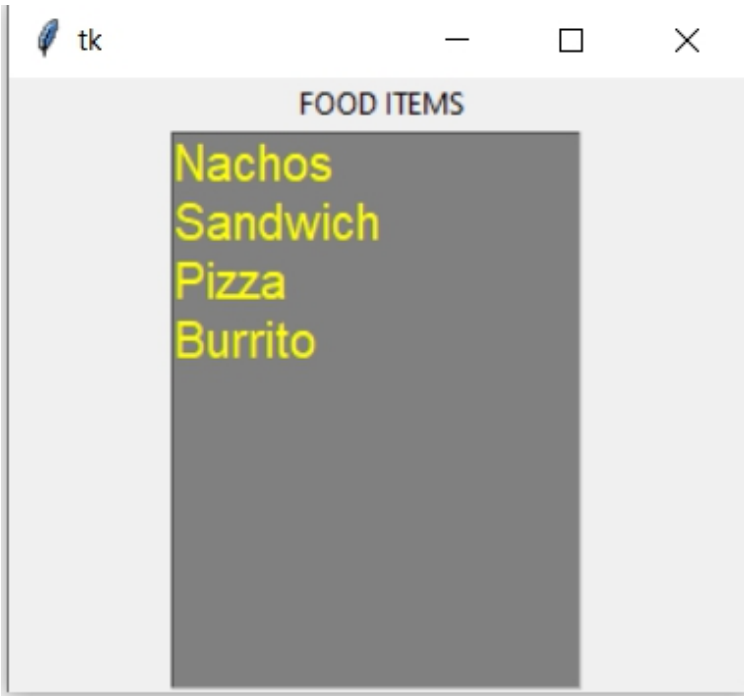


Example 2: Let's Delete the elements from the above created listbox

Python3

```
# Delete Items from the list  
  
# by specifying the index.  
  
listbox.delete(2)
```

Output:



CHAPTER 26: Scrollable ListBox in Python-tkinter

Tkinter is the standard GUI library for Python. Tkinter in Python comes with a lot of good widgets. Widgets are standard GUI elements, and the ListBox, Scrollbar will also come under this Widgets.

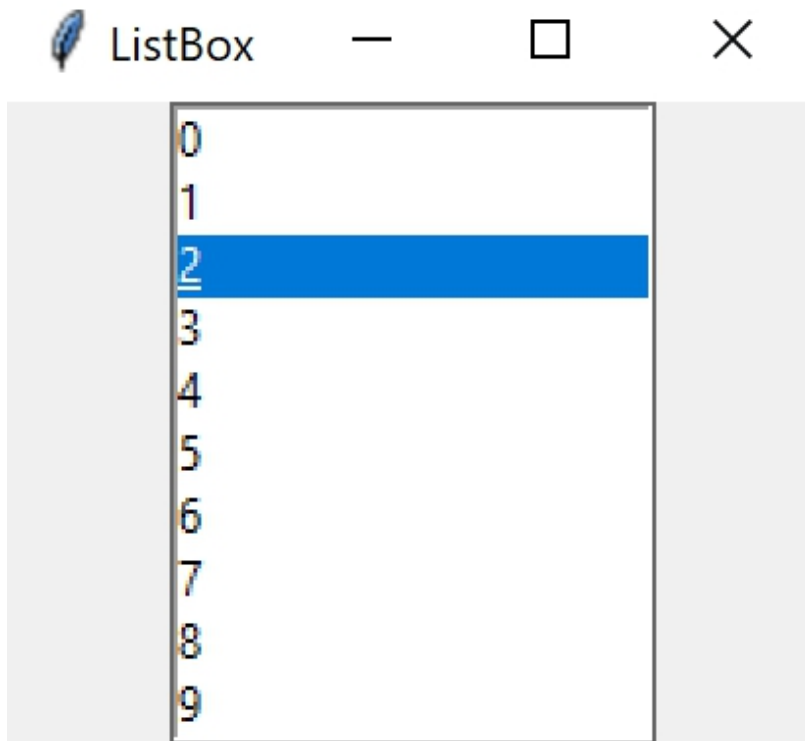
Note: For more information, refer to Python GUI – tkinter

ListBox

The ListBox widget is used to display different types of items. These items must be of the same type of font and having the same font color. The items must also be of Text type. The user can select one or more items from the given list according to the requirement.

Syntax:

`listbox = Listbox(root, bg, fg, bd, height, width, font, ..)`



Scrollbar

The scrollbar widget is used to scroll down the content. We can also create the horizontal scrollbars to the Entry widget.

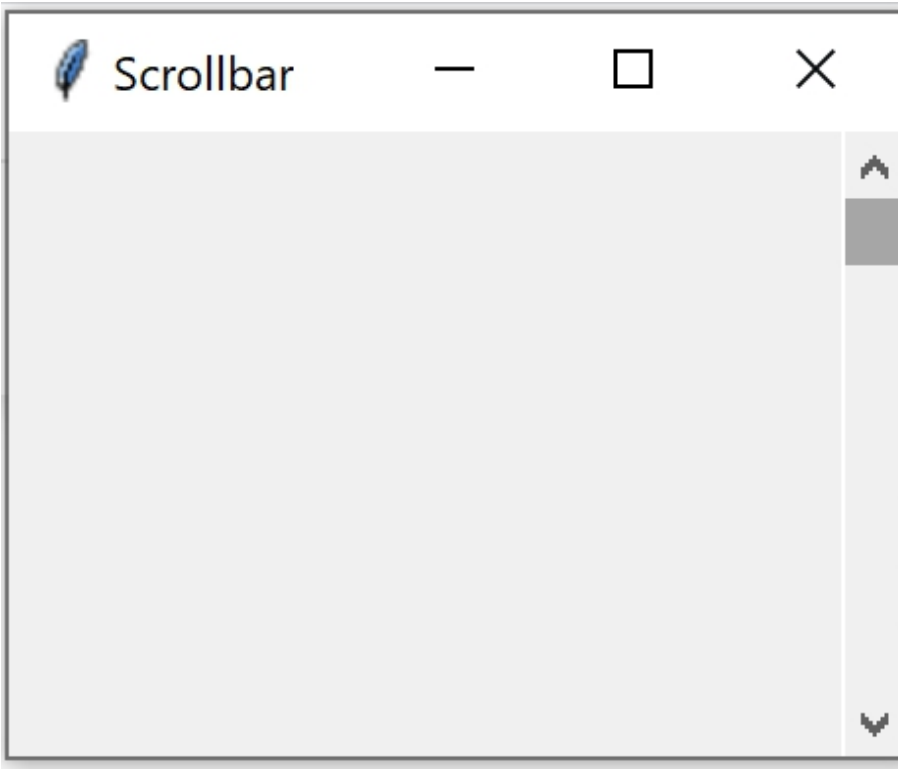
Syntax:

The syntax to use the Scrollbar widget is given below.

`w = Scrollbar(master, options)`

Parameters:

- **master:** This parameter is used to represents the parent window.
- **options:** There are many options which are available and they can be used as key-value pairs separated by commas.



Adding Scrollbar to ListBox

To do this we need to attach the scrollbar to Listbox, and to attach we use a function `listbox.config()` and set its `command` parameter to the scrollbar's `set` method then set the scrollbar's `command` parameter to point a method that would be called when the scroll bar position is changed

```
from tkinter import *

# Creating the root window

root = Tk()

# Creating a Listbox and
```

```
# attaching it to root window

listbox = Listbox(root)


# Adding Listbox to the left

# side of root window

listbox.pack(side = LEFT, fill = BOTH)


# Creating a Scrollbar and

# attaching it to root window

scrollbar = Scrollbar(root)


# Adding Scrollbar to the right

# side of root window

scrollbar.pack(side = RIGHT, fill = BOTH)


# Insert elements into the listbox

for values in range(100):

    listbox.insert(END, values)


# Attaching Listbox to Scrollbar

# Since we need to have a vertical

# scroll we use yscrollcommand

listbox.config(yscrollcommand = scrollbar.set)
```

```
# setting scrollbar command parameter

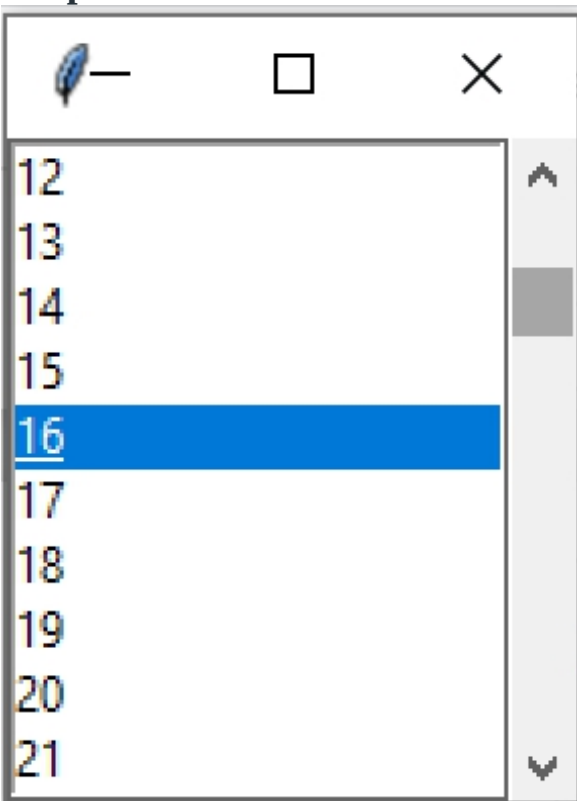
# to listbox.yview method its yview because

# we need to have a vertical view

scrollbar.config(command = listbox.yview)

root.mainloop()
```

Output



CHAPTER 27: Python Tkinter – Frame Widget

Python offers multiple options for developing GUI (Graphical User Interface). Out of all the GUI methods, tkinter is the most commonly used method. It is a standard Python interface to the Tk GUI toolkit shipped with Python. Python with tkinter is the fastest and easiest way to create the GUI applications. Creating a GUI using tkinter is an easy task.

Note: For more information, refer to [Python GUI – tkinter](#)

Frame

A frame is a rectangular region on the screen. A frame can also be used as a foundation class to implement complex widgets. It is used to organize a group of widgets.

Syntax:

The syntax to use the frame widget is given below.

```
w = frame( master, options)
```

Parameters:

- **master:** This parameter is used to represents the parent window.
- **options:** There are many options which are available and they can be used as key-value pairs separated by commas.

Options:

Following are commonly used Option can be used with this widget :-

- **bg:** This option used to represent the normal background color displayed behind the label and indicator.
- **bd:** This option used to represent the size of the border around the indicator and the default value is 2 pixels.
- **cursor:** By using this option, the mouse cursor will change to that pattern when it is over the frame.

- **height:** The vertical dimension of the new frame.
- **highlightcolor:** This option used to represent the color of the focus highlight when the frame has the focus.
- **highlightthickness:** This option used to represent the color of the focus highlight when the frame does not have focus.
- **highlightbackground:** This option used to represent the thickness of the focus highlight..
- **relief:** The type of the border of the frame. It's default value is set to FLAT.
- **width:** This option used to represents the width of the frame.

Example:

```
from tkinter import * root = Tk()

root.geometry("300x150")


w = Label(root, text ='GeeksForGeeks', font = "50")
w.pack()


frame = Frame(root)
frame.pack()


bottomframe = Frame(root)
bottomframe.pack( side = BOTTOM )


b1_button = Button(frame, text ="Geeks1", fg ="red")
b1_button.pack( side = LEFT)
```

```
b2_button = Button(frame, text ="Geeks2", fg ="brown")

b2_button.pack( side = LEFT )


b3_button = Button(frame, text ="Geeks3", fg ="blue")

b3_button.pack( side = LEFT )


b4_button = Button(bottomframe, text ="Geeks4", fg ="green")

b4_button.pack( side = BOTTOM)


b5_button = Button(bottomframe, text ="Geeks5", fg ="green")

b5_button.pack( side = BOTTOM)


b6_button = Button(bottomframe, text ="Geeks6", fg ="green")

b6_button.pack( side = BOTTOM)


root.mainloop()
```

Output:



CHAPTER 28: Scrollable Frames in Tkinter

A scrollbar is a widget that is useful to scroll the text in another widget. For example, the text in Text, Canvas Frame or Listbox can be scrolled from top to bottom or left to right using scrollbars. There are two types of scrollbars. They are horizontal and vertical. The horizontal scrollbar is useful to view the text from left to right. The vertical scrollbar is useful to scroll the text from top to bottom.

Now let us see how we can create a scrollbar. To create a scrollbar we have to create a scrollbar class object as:

```
h = Scrollbar(root, orient='horizontal')
```

Here h represents the scrollbar object which is created as a child to root window. Here orient indicates *horizontal* for horizontal scrollbar and *vertical* indicates vertical scroll bars. Similarly to create a vertical scrollbar we can write:

```
v = Scrollbar(root, orient='vertical')
```

On attaching the scrollbar to a widget like text box, list box, etc we then have to add a command *xview* for horizontal scrollbar and *yview* for the vertical scrollbar.

```
h.config(command=t.xview) #for horizontal scrollbar
```

```
v.config(command=t.yview) #for vertical scrollbar
```

Now let us look at the code for attaching a vertical and horizontal scrollbar to a Text Widget.

- Python3

```
# Python Program to make a scrollable frame
```

```
# using Tkinter
```



```
from tkinter import *

class ScrollBar:

    # constructor

    def __init__(self):

        # create root window

        root = Tk()

        # create a horizontal scrollbar by

        # setting orient to horizontal

        h = Scrollbar(root, orient = 'horizontal')

        # attach Scrollbar to root window at

        # the bottom

        h.pack(side = BOTTOM, fill = X)

        # create a vertical scrollbar-no need

        # to write orient as it is by

        # default vertical

        v = Scrollbar(root)

        # attach Scrollbar to root window on

        # the side
```

```

v.pack(side = RIGHT, fill = Y)

# create a Text widget with 15 chars
# width and 15 lines height
# here xscrollcommand is used to attach Text
# widget to the horizontal scrollbar
# here yscrollcommand is used to attach Text
# widget to the vertical scrollbar

t = Text(root, width = 15, height = 15, wrap = NONE,

          xscrollcommand = h.set,

          yscrollcommand = v.set)

# insert some text into the text widget

for i in range(20):

    t.insert(END,"this is some text\n")

# attach Text widget to root window at top
t.pack(side=TOP, fill=X)

# here command represents the method to
# be executed xview is executed on
# object 't' Here t may represent any
# widget
h.config(command=t.xview)

```

```
# here command represents the method to  
# be executed yview is executed on  
# object 't' Here t may represent any  
# widget  
v.config(command=t.yview)  
  
# the root window handles the mouse  
# click event  
root.mainloop()  
  
# create an object to Scrollbar class  
s = ScrollBar()
```

CHAPTER 29: Make a proper double scrollbar frame in Tkinter

Tkinter is a Python binding to the Tk GUI(Graphical User Interface) Toolkit. It is a thin-object oriented layer on top of Tcl/Tk. When combined with Python, it helps create fast and efficient GUI applications.

Note: For more information refer, Python GUI-tkinter

Steps to Create a double scrollbar frame in Tkinter

1) Firstly, the module Tkinter will be imported as:

```
import tkinter as tk
```

So, **tkinter** is abbreviated here as **tk** so as to make the code look cleaner and efficient.

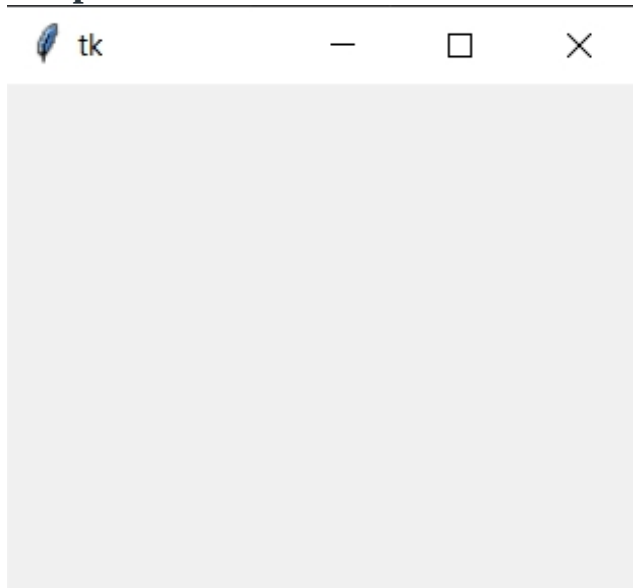
Now, a window will be created to display:

```
import tkinter as tk

window = tk.Tk()

window.geometry("250x200")
```

Output:



Functions to understand:

- **geometry():** This method is used to set the dimensions of the Tkinter window as well as it is used to set the position of the main window on the user's desktop.

2) The next code is to assign to the scrollbars for horizontal and vertical.

```
SVBar = tk.Scrollbar(window)

SVBar.pack (side = tk.RIGHT,

            fill = "y")

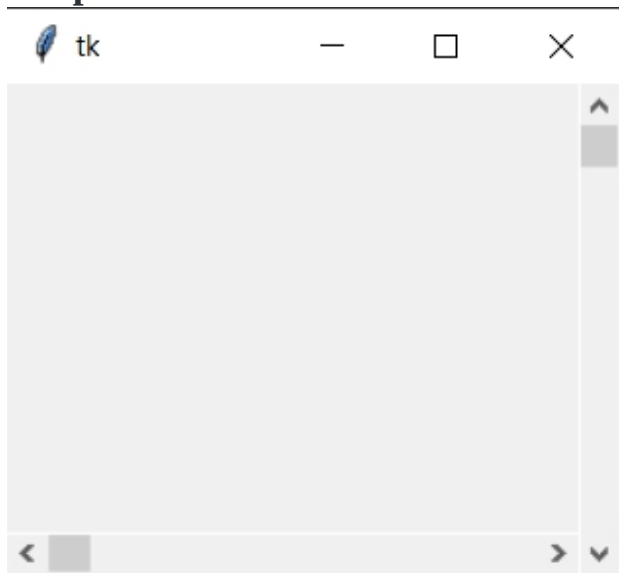
SHBar = tk.Scrollbar(window,

                    orient = tk.HORIZONTAL)

SHBar.pack (side = tk.BOTTOM,

            fill = "x")
```

Output:



Functions to understand:

- **Scrollbar()** = It is the scrollbar that is allotted to the sides of the window.
- **pack()** method: It organizes the widgets in blocks before placing in the parent widget.

3) Now, make a text box for the window:

```
TBox = tk.Text(window,  
  
    height = 500,  
  
    width = 500,  
  
    yscrollcommand = SVBar.set,  
  
    xscrollcommand = SHBar.set,  
  
    wrap = "none")
```

```
TBox = tk.Text(window,  
  
    height = 500,  
  
    width = 500,  
  
    yscrollcommand = SVBar.set,  
  
    xscrollcommand = SHBar.set,  
  
    wrap = "none")
```

```
TBox.pack(expand = 0, fill = tk.BOTH)
```

Functions to understand:

- **Text()** = It is a textbox widget of a standard Tkinter widget used to display the text.
- **pack()** = It is a geometry manager for organizing the widgets in blocks before placing them in the parent widget. Options like fill, expand and side are used in the function.

```
SHBar.config(command = TBox.xview)
```

```
SVBar.config(command = TBox.yview)
```

Here, within the arguments of the function `config()`, the scrollbars are assigned at their specific x-axis and y-axis and are able to function.

Now, insert some amount of text to display:

```
Num_Vertical =
```

```
("A\nB\nC\nD\nE\nF\nG\nH\nI\nJ\nK\nL\nM\nN\nO\nP\nQ\nR\nS\nT\nU\nV\nW\nX\nY\nZ")
```

```
Num_Horizontal = ("A B C D E F G H I J K L M N O P Q R S T U V W X Y Z")
```

To insert the text in the window for the display, the following code is done:

```
TBox.insert(tk.END, Num_Horizontal)
```

```
TBox.insert(tk.END, Num_Vertical)
```

Complete Code:

```
import tkinter as tk
```

```
Num_Vertical = ("A\nB\nC\nD\nE\nF\nG\nH\nI\nJ\nK\nL\nM\nN\nO\nP\nQ\nR\nS\nT\nU\nV\nW\nX\nY\nZ")
```

```
Num_Horizontal = ("A B C D E F G H I J K L M N O P Q R S T U V W X Y Z")
```

```
U\nV\nW\nX\nY\nZ")
```

```
Num_Horizontal = ("A B C D E F G H \
```

```
I J K L M N O P Q R S T U V \
```

```
W X Y Z")
```

```
window = tk.Tk()
```

```
window.geometry("250x200")
```

```
SVBar = tk.Scrollbar(window)
```

```
SVBar.pack (side = tk.RIGHT,
```

```
fill = "y")
```

```
SHBar = tk.Scrollbar(window,
```

```
orient = tk.HORIZONTAL)
```

```
SHBar.pack (side = tk.BOTTOM,
```

```
fill = "x")
```

```
TBox = tk.Text(window,
```

```
height = 500,
```

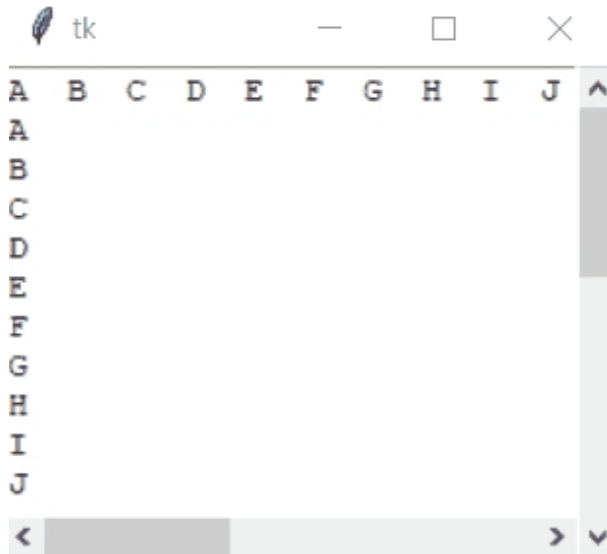
```
width = 500,
```

```
yscrollcommand = SVBar.set,
```



```
xscrollcommand = SHBar.set,  
  
wrap = "none")  
  
TBox = tk.Text(window,  
  
    height = 500,  
  
    width = 500,  
  
    yscrollcommand = SVBar.set,  
  
    xscrollcommand = SHBar.set,  
  
    wrap = "none")  
  
TBox.pack(expand = 0, fill = tk.BOTH)  
  
TBox.insert(tk.END, Num_Horizontal)  
TBox.insert(tk.END, Num_Vertical)  
  
SHBar.config(command = TBox.xview)  
  
SVBar.config(command = TBox.yview)  
  
window.mainloop()
```

Output:



CHAPTER 30: Python Tkinter – Scale Widget

Tkinter is a GUI toolkit used in python to make user-friendly GUIs. Tkinter is the most commonly used and the most basic GUI framework available in python. Tkinter uses an object-oriented approach to make GUIs.

Note: For more information, refer to [Python GUI – tkinter](#)

Scale widget

The Scale widget is used whenever we want to select a specific value from a range of values. It provides a sliding bar through which we can select the values by sliding from left to right or top to bottom depending upon the orientation of our sliding bar.

Syntax:

```
S = Scale(root, bg, fg, bd, command, orient, from_, to, ..)
```

Optional parameters

- **root** – root window.
- **bg** – background colour
- **fg** – foreground colour
- **bd** – border
- **orient** – orientation(vertical or horizontal)
- **from_** – starting value
- **to** – ending value
- **troughcolor** – set colour for trough.
- **state** – decides if the widget will be responsive or unresponsive.
- **sliderlength** – decides the length of the slider.
- **label** – to display label in the widget.
- **highlightbackground** – the colour of the focus when widget is not focused.
- **cursor** – The cursor on the widget which could be arrow, circle, dot etc.

Methods

- **set(value)** – set the value for scale.
- **get()** – get the value of scale.

Example 1: Creating a horizontal bar

```
# Python program to demonstrate  
# scale widget  
  
from tkinter import *  
  
root = Tk()  
  
root.geometry("400x300")
```

```
v1 = DoubleVar()

def show1():

    sel = "Horizontal Scale Value = " + str(v1.get())

    l1.config(text = sel, font = ("Courier", 14))


s1 = Scale( root, variable = v1,

            from_ = 1, to = 100,

            orient = HORIZONTAL)

l3 = Label(root, text = "Horizontal Scaler")

b1 = Button(root, text = "Display Horizontal",

            command = show1,

            bg = "yellow")

l1 = Label(root)


s1.pack(anchor = CENTER)

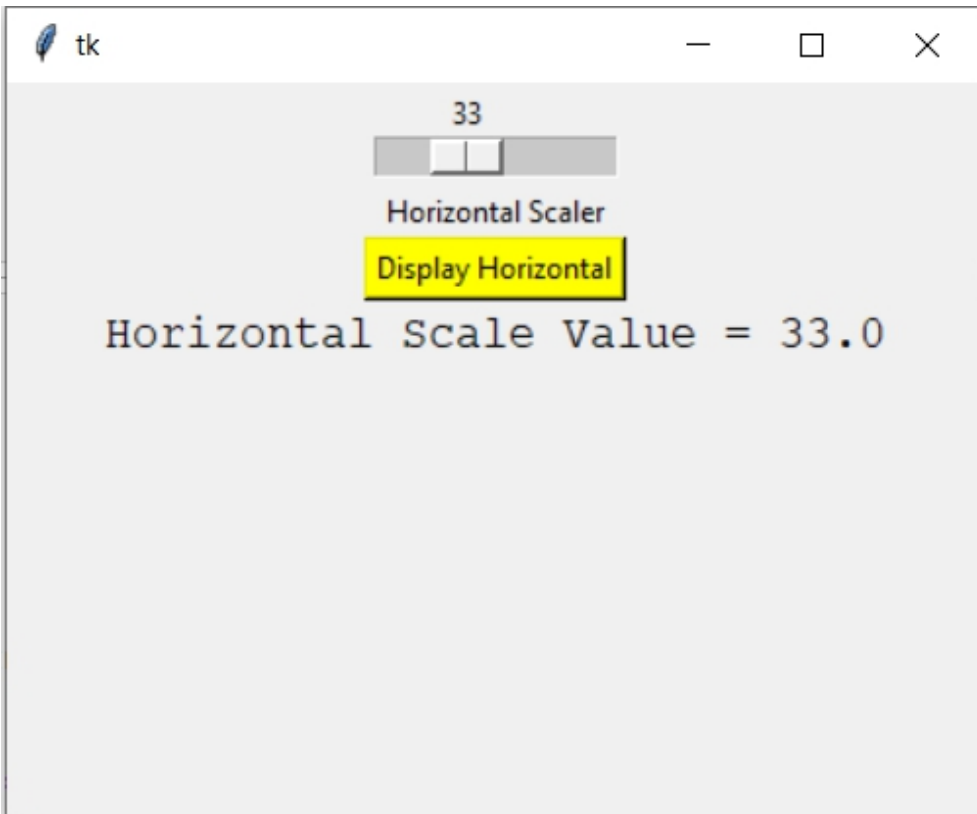
l3.pack()

b1.pack(anchor = CENTER)
```

```
l1.pack()
```

```
root.mainloop()
```

Output:



Example 2: Creating a vertical slider

```
from tkinter import *
```

```
root = Tk()
```

```
root.geometry("400x300")
```

```
v2 = DoubleVar()

def show2():

    sel = "Vertical Scale Value = " + str(v2.get())

    l2.config(text = sel, font = ("Courier", 14))

s2 = Scale( root, variable = v2,

            from_ = 50, to = 1,

            orient = VERTICAL)

l4 = Label(root, text = "Vertical Scaler")

b2 = Button(root, text = "Display Vertical",

            command = show2,

            bg = "purple",

            fg = "white")

l2 = Label(root)

s2.pack(anchor = CENTER)

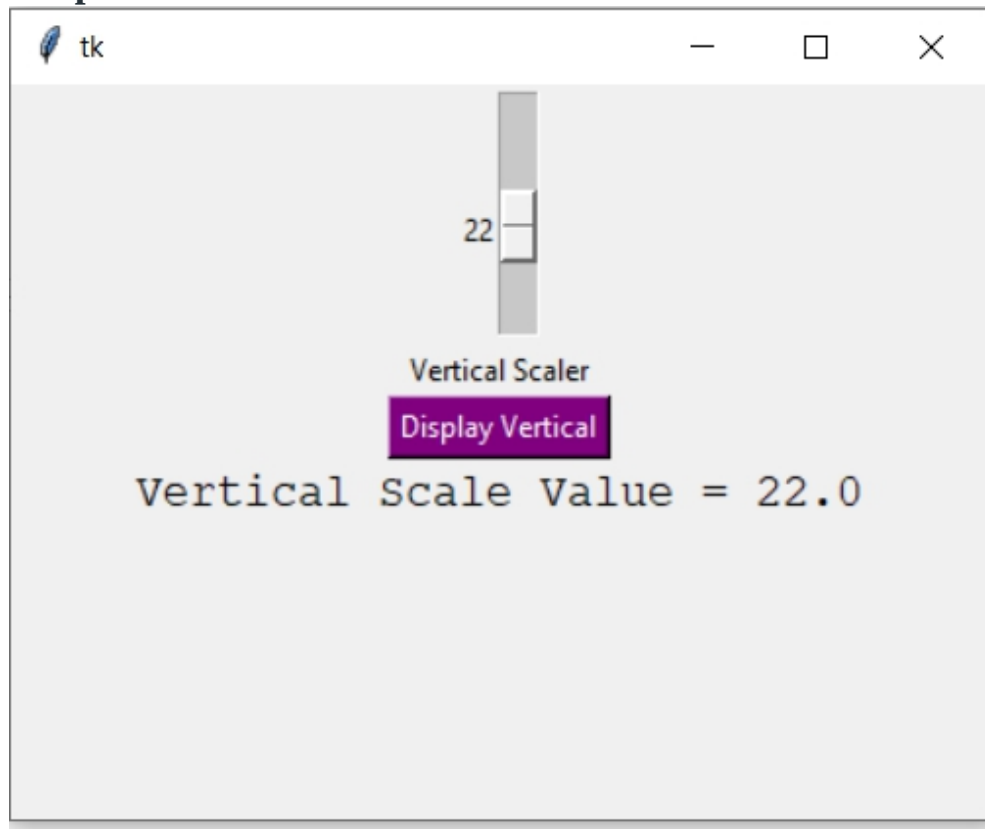
l4.pack()

b2.pack()
```

```
l2.pack()
```

```
root.mainloop()
```

Output:



CHAPTER 31: Hierarchical treeview in Python GUI application

Python uses different GUI applications that are helpful for the users while interacting with the applications they are using. There are basically three GUI(s) that python uses namely *Tkinter*, *wxPython*, and *PyQt*. All of these

can operate with windows, Linux, and mac-OS. However, these GUI applications have many widgets i.e, controls that are helpful for the user interaction with the application. Some of the widgets are buttons, list boxes, scrollbar, treeview, etc.

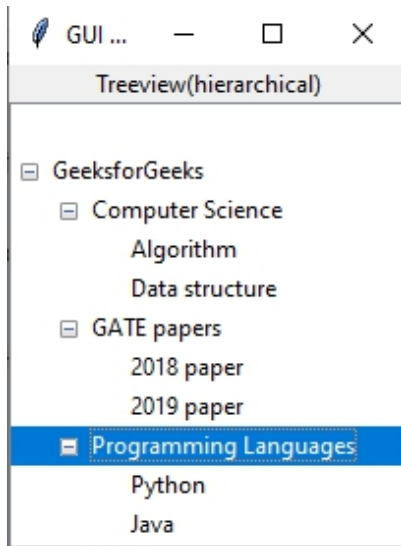
Note: For more information, refer [to Python GUI – tkinter](#)

Treeview widgets

This widget is helpful in visualizing and permitting navigation over a hierarchy of items. It can display more than one feature of every item in the hierarchy. It can build a tree view as a user interface like in Windows explorer. Therefore, here we will use Tkinter in order to construct a hierarchical treeview in the Python GUI application.

*Let's see an example of constructing a **hierarchical treeview in Python GUI application**.*

The GUI would look like below:



Example:

- Python

```
# Python program to illustrate the usage
```



```
# of hierarchical treeview in python GUI

# application using tkinter


# Importing tkinter

from tkinter import *


# Importing ttk from tkinter

from tkinter import ttk


# Creating app window

app = Tk()


# Defining title of the app

app.title("GUI Application of Python")


# Defining label of the app and calling a geometry
# management method i.e, pack in order to organize
# widgets in form of blocks before locating them
# in the parent widget

ttk.Label(app, text = "Treeview(hierarchical)").pack()


# Creating treeview window

treeview = ttk.Treeview(app)


# Calling pack method on the treeview
```

```
treeview.pack()

# Inserting items to the treeview

# Inserting parent
treeview.insert("", '0', 'item1',
               text ='GeeksforGeeks')

# Inserting child
treeview.insert("", '1', 'item2',
               text ='Computer Science')
treeview.insert("", '2', 'item3',
               text ='GATE papers')
treeview.insert("", 'end', 'item4',
               text ='Programming Languages')

# Inserting more than one attribute of an item
treeview.insert('item2', 'end', 'Algorithm',
               text ='Algorithm')
treeview.insert('item2', 'end', 'Data structure',
               text ='Data structure')
treeview.insert('item3', 'end', '2018 paper',
               text ='2018 paper')
treeview.insert('item3', 'end', '2019 paper',
               text ='2019 paper')
treeview.insert('item4', 'end', 'Python',
               text ='Python')
```

```
treeview.insert('item4', 'end', 'Java',
               text ='Java')

# Placing each child items in parent widget

treeview.move('item2', 'item1', 'end')

treeview.move('item3', 'item1', 'end')

treeview.move('item4', 'item1', 'end')

# Calling main()

app.mainloop()
```

CHAPTER 32: Tkinter Treeview scrollbar

Python has several options for constructing GUI and [python tkinter](#) is one of them. It is the standard *GUI* library for Python, which helps in making GUI applications easily. It provides an efficient object-oriented interface to the *tk* GUI toolkit. It also has multiple controls called widgets like text boxes, scrollbars, buttons, etc. Moreover, Tkinter has some geometry management methods namely, *pack()*, *grid()*, and *place()* which are helpful in organizing widgets.

Note: For more information, refer to [Python GUI – tkinter](#)

Treeview scrollbar

When a scrollbar uses *treeview* widgets, then that type of scrollbar is called as *treeview scrollbar*. Where, a treeview widget is helpful in displaying more than one feature of every item listed in the tree to the right side of the tree in the form of columns. However, it can be implemented using tkinter in python with the help of some widgets and geometry management methods as supported by tkinter.

Below example illustrates the usage of **Treeview Scrollbar** using Python-tkinter:

Example 1:

- Python

```
# Python program to illustrate the usage of
# treeview scrollbars using tkinter

from tkinter import ttk

import tkinter as tk

# Creating tkinter window

window = tk.Tk()

window.resizable(width = 1, height = 1)

# Using treeview widget

treev = ttk.Treeview(window, selectmode ='browse')

# Calling pack method w.r.to treeview
```

```
treev.pack(side ='right')

# Constructing vertical scrollbar

# with treeview

verscrlbar = ttk.Scrollbar(window,

                           orient ="vertical",

                           command = treev.yview)

# Calling pack method w.r.to vertical

# scrollbar

verscrlbar.pack(side ='right', fill ='x')

# Configuring treeview

treev.configure(xscrollcommand = verscrlbar.set)

# Defining number of columns

treev["columns"] = ("1", "2", "3")

# Defining heading

treev['show'] = 'headings'

# Assigning the width and anchor to the

# respective columns

treev.column("1", width = 90, anchor ='c')

treev.column("2", width = 90, anchor ='se')
```

```
treev.column("3", width = 90, anchor ='se')
```

```
# Assigning the heading names to the
```

```
# respective columns
```

```
treev.heading("1", text ="Name")
```

```
treev.heading("2", text ="Sex")
```

```
treev.heading("3", text ="Age")
```

```
# Inserting the items and their features to the
```

```
# columns built
```

```
treev.insert("", 'end', text ="L1",  
             values =("Nidhi", "F", "25"))
```

```
treev.insert("", 'end', text ="L2",  
             values =("Nisha", "F", "23"))
```

```
treev.insert("", 'end', text ="L3",  
             values =("Preeti", "F", "27"))
```

```
treev.insert("", 'end', text ="L4",  
             values =("Rahul", "M", "20"))
```

```
treev.insert("", 'end', text ="L5",  
             values =("Sonu", "F", "18"))
```

```
treev.insert("", 'end', text ="L6",  
             values =("Rohit", "M", "19"))
```

```
treev.insert("", 'end', text ="L7",  
             values =("Geeta", "F", "25"))
```

```
treev.insert("", 'end', text ="L8",  
             values =("Ankit", "M", "22"))
```

```
treev.insert("", 'end', text = "L10",  
              values = ("Mukul", "F", "25"))  
treev.insert("", 'end', text = "L11",  
              values = ("Mohit", "M", "16"))  
treev.insert("", 'end', text = "L12",  
              values = ("Vivek", "M", "22"))  
treev.insert("", 'end', text = "L13",  
              values = ("Suman", "F", "30"))  
  
# Calling mainloop  
window.mainloop()
```

PART 3: Toplevel Widgets

CHAPTER 1: Python Tkinter – Toplevel Widget

Tkinter is a GUI toolkit used in python to make user-friendly GUIs. Tkinter is the most commonly used and the most basic GUI framework available in Python. Tkinter uses an object-oriented approach to make GUIs.

Note: For more information, refer to [Python GUI – tkinter](#)

Toplevel widget

A Toplevel widget is used to create a window on top of all other windows. The Toplevel widget is used to provide some extra information to the user and also when our program deals with more than one application. These windows are directly organized and managed by the Window Manager and do not need to have any parent window associated with them every time.

Syntax:

```
toplevel = Toplevel(root, bg, fg, bd, height, width, font, ..)
```

Optional parameters

- **root** = root window(optional)
- **bg** = background colour
- **fg** = foreground colour
- **bd** = border
- **height** = height of the widget.
- **width** = width of the widget.

- **font** = Font type of the text.
- **cursor** = cursor that appears on the widget which can be an arrow, a dot etc.

Common methods

- **iconify** turns the windows into icon.
- **deiconify** turns back the icon into window.
- **state** returns the current state of window.
- **withdraw** removes the window from the screen.
- **title** defines title for window.
- **frame** returns a window identifier which is system specific.

Example 1:

- Python3

```
from tkinter import *

root = Tk()

root.geometry("200x300")

root.title("main")

l = Label(root, text = "This is root window")

top = Toplevel()

top.geometry("180x100")

top.title("toplevel")

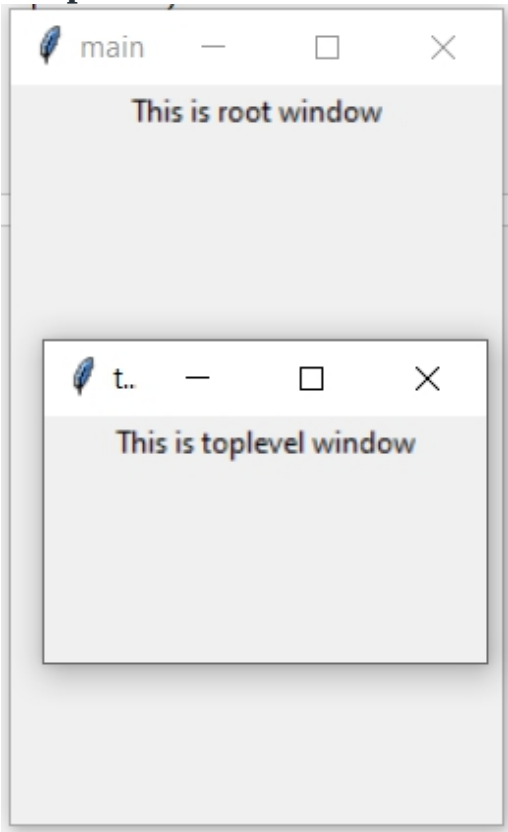
l2 = Label(top, text = "This is toplevel window")
```

```
l.pack()

l2.pack()

top.mainloop()
```

Output



Example 2: Creating Multiple toplevels over one another

- Python3

```
from tkinter import *
```

```
# Create the root window

# with specified size and title

root = Tk()

root.title("Root Window")

root.geometry("450x300")


# Create label for root window

label1 = Label(root, text = "This is the root window")


# define a function for 2nd toplevel

# window which is not associated with

# any parent window

def open_Toplevel2():

    # Create widget

    top2 = Toplevel()

    # define title for window

    top2.title("Toplevel2")

    # specify size

    top2.geometry("200x100")

    # Create label

    label = Label(top2,
```

```
        text = "This is a Toplevel2 window")

# Create exit button.

button = Button(top2, text = "Exit",

                command = top2.destroy)

label.pack()

button.pack()

# Display until closed manually.

top2.mainloop()

# define a function for 1st toplevel

# which is associated with root window.

def open_Toplevel1():

    # Create widget

    top1 = Toplevel(root)

    # Define title for window

    top1.title("Toplevel1")

    # specify size

    top1.geometry("200x200")
```

```
# Create label

label = Label(top1,

               text = "This is a Toplevel1 window")


# Create Exit button

button1 = Button(top1, text = "Exit",

                  command = top1.destroy)


# create button to open toplevel2

button2 = Button(top1, text = "open toplevel2",

                  command = open_Toplevel2)


label.pack()
button2.pack()
button1.pack()


# Display until closed manually

top1.mainloop()


# Create button to open toplevel1

button = Button(root, text = "open toplevel1",

                 command = open_Toplevel1)

label1.pack()

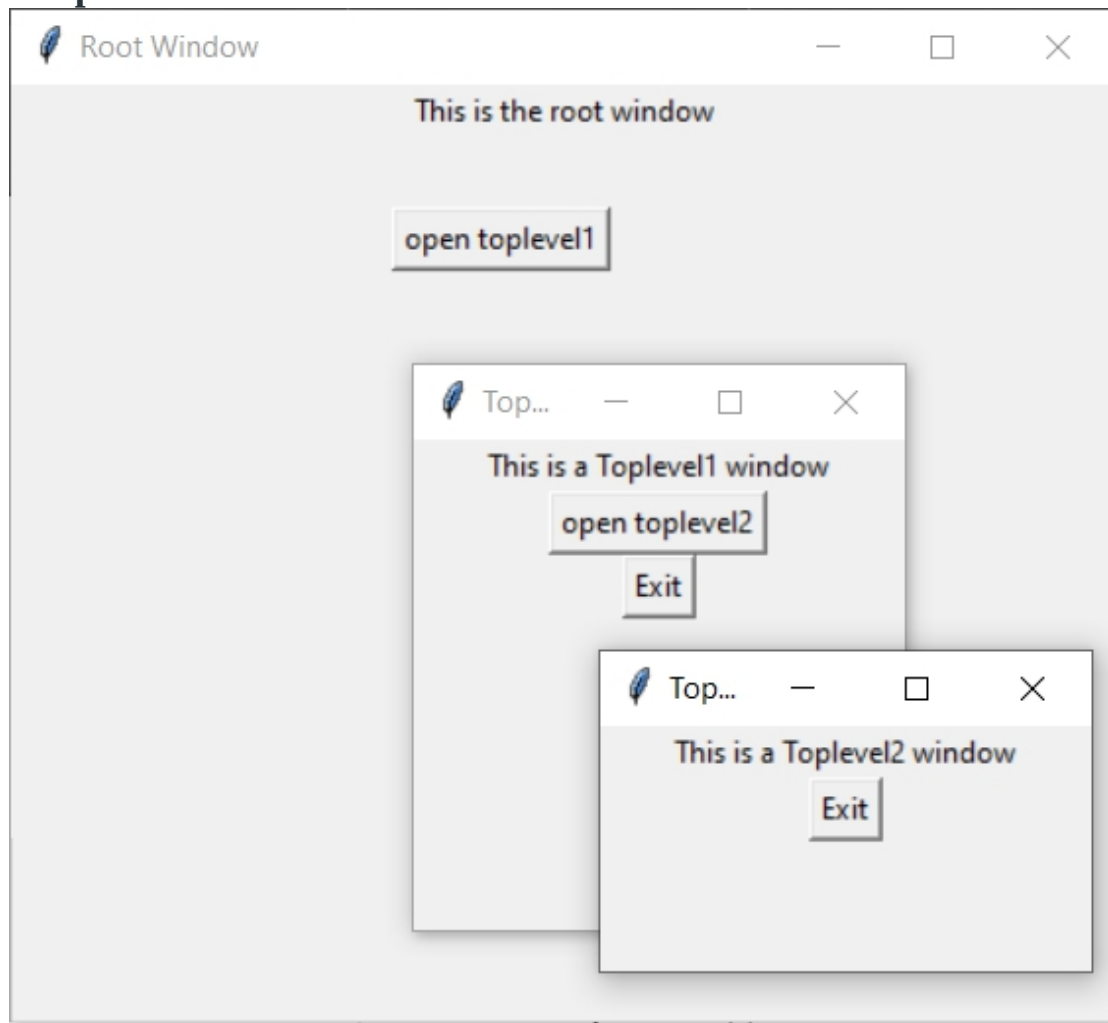

# position the button
```

```
button.place(x = 155, y = 50)
```

```
# Display until closed manually
```

```
root.mainloop()
```

Output



CHAPTER 2: askopenfile() function in Tkinter

While working with GUI one may need to open files and read data from it or may require to write data in that particular file. One can achieve this with the help of **open()** function (python built-in) but one may not be able to select any required file unless provides a path to that particular file in code.

With the help of GUI, you may not require to specify the path of any file but you can directly open a file and read it's content.

In order to use askopenfile() function you may require to follow these steps:

-> import tkinter -> from tkinter.filedialog import askopenfile ## Now you can use this function -> file = askopenfile(mode='r', filetypes=[('any name you want to display', 'extension of file type')])

We have to specify the mode in which you want to open the file like in above snippet, this will open a file in reading mode.

- Python3

```
# importing tkinter and tkinter.ttk

# and all their functions and classes

from tkinter import *

from tkinter.ttk import *


# importing askopenfile function

# from class filedialog

from tkinter.filedialog import askopenfile
```

```
root = Tk()

root.geometry('200x100')


# This function will be used to open
# file in read mode and only Python files
# will be opened

def open_file():

    file = askopenfile(mode='r', filetypes=[('Python Files', '*.py')])

    if file is not None:

        content = file.read()

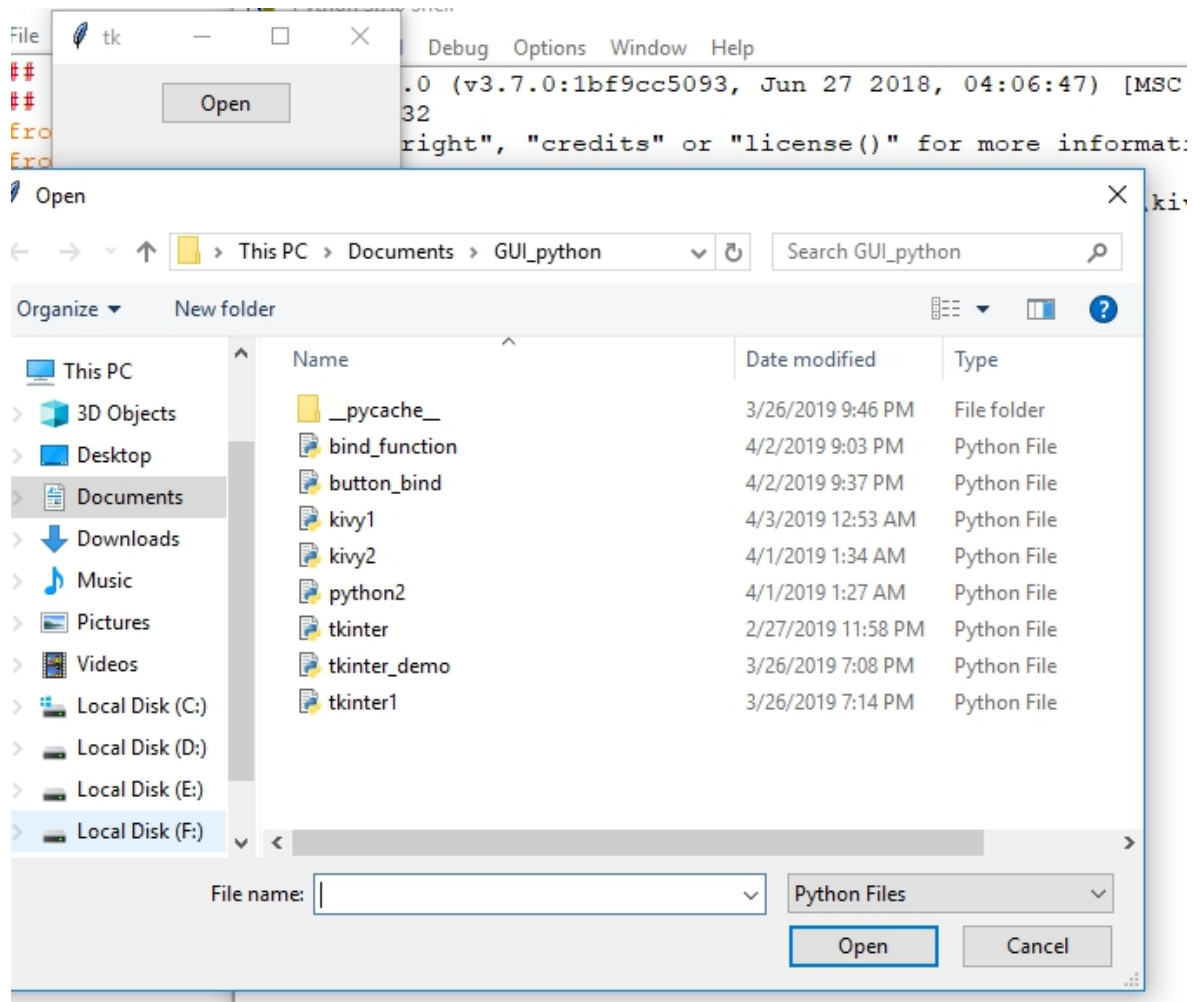
        print(content)


btn = Button(root, text='Open', command = lambda:open_file())

btn.pack(side = TOP, pady = 10)


mainloop()
```

Output:



Printed content of selected file –

```

from tkinter import *
from PIL import Image, ImageTk

root = Tk()
ox = root.winfo_screenwidth()/2
oy = root.winfo_screenheight()/2
root.geometry("=300x300+%d+%d" % (ox-400,oy-345) )
                                                    #sin bordes

can = Canvas(root,bg='red',width=800,height=700)
can.pack()
photo=ImageTk.PhotoImage(file="information.png")
can.create_image(20,20,image=photo)

boton = Button(can,image=photo,border=0)

boton.place(x=60,y=100)

root.mainloop()

```

Comparison of content of original file and printed content –

<pre> from tkinter import * from PIL import Image, ImageTk root = Tk() ox = root.winfo_screenwidth()/2 oy = root.winfo_screenheight()/2 root.geometry("=300x300+%d+%d" % (ox-400,oy-345)) #sin can = Canvas(root,bg='red',width=800,height=700) can.pack() photo=ImageTk.PhotoImage(file="information.png") can.create_image(20,20,image=photo) boton = Button(can,image=photo,border=0) boton.place(x=60,y=100) root.mainloop() </pre>	<pre> from tkinter import * from PIL import Image, ImageTk root = Tk() ox = root.winfo_screenwidth()/2 oy = root.winfo_screenheight()/2 root.geometry("=300x300+%d+%d" % (ox-400,oy-345)) can = Canvas(root,bg='red',width=800,height=700) can.pack() photo=ImageTk.PhotoImage(file="information.png") can.create_image(20,20,image=photo) boton = Button(can,image=photo,border=0) boton.place(x=60,y=100) root.mainloop() </pre>
---	--

Note: In above code only .py (python files) types files will be open. To open specified type of files, one has to mention it in the **filetypes** option along with it's extension as done in above code.

CHAPTER 3: asksaveasfile() function in Tkinter

Python provides a variety of modules with the help of which one may develop GUI (Graphical User Interface) applications. [Tkinter](#) is one of the easiest and fastest way to develop GUI applications.

While working with files one may need to open files, do operations on files and after that to save file. `asksaveasfile()` is the function which is used to save user's file (extension can be set explicitly or you can set default extensions also). This function comes under the `class filedialog`.

Below is the Code:

```
# importing all files from tkinter

from tkinter import *

from tkinter import ttk


# import only asksaveasfile from filedialog
# which is used to save file in any extension

from tkinter.filedialog import asksaveasfile


root = Tk()

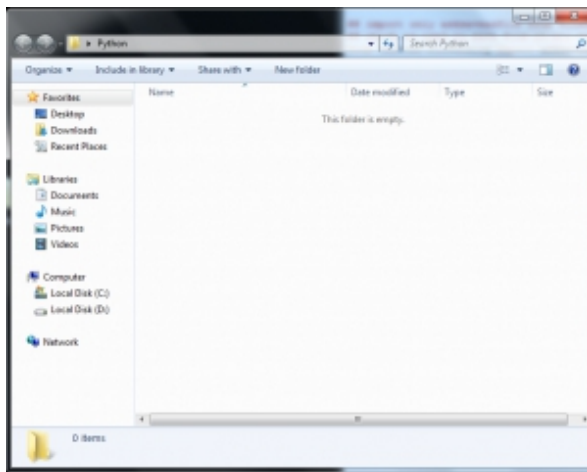
root.geometry('200x150')


# function to call when user press
# the save button, a filedialog will
# open and ask to save file

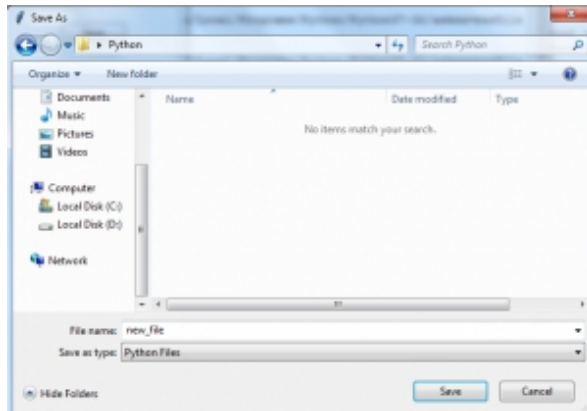
def save():
```

```
files = [('All Files', '*.*'),  
         ('Python Files', '*.py'),  
         ('Text Document', '*.txt')]  
  
file = asksaveasfile(filetypes = files, defaulttextextension = files)  
  
btn = ttk.Button(root, text = 'Save', command = lambda : save())  
  
btn.pack(side = TOP, pady = 20)  
  
mainloop()
```

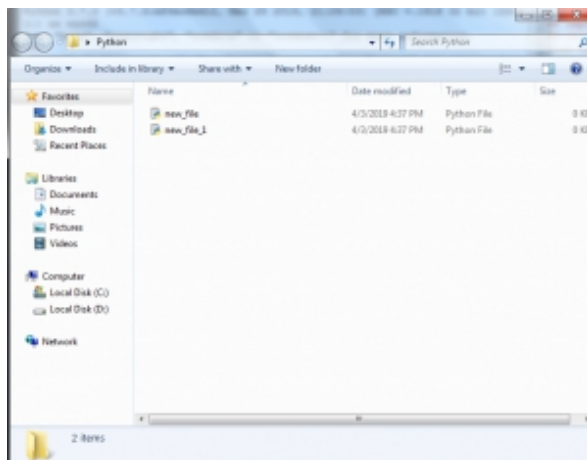
Output #1: Directory before saving any file (folder is initially empty)



Output #2: Dialogbox when user presses the save button (dialog box to save file is opened). You may see in the output Python file as default is selected.



Output #3: Directory after saving 2 Python files (one may also change the type of file)



CHAPTER 4: Tkinter askquestion Dialog

In Python, There Are Several Libraries for Graphical User Interface. [Tkinter](#) is one of them that is most useful. It is a standard interface. Tkinter is easy to use and provides several functions for building efficient applications. In Every Application, we need some Message to Display like “Do You Want To Close ” or showing any warning or Something information. For this Tkinter provide a library like **messagebox**. By using the message box library we can show several Information, Error, Warning, Cancellation ETC in the form of Message-Box. It has a Different message box for a different purpose.

1. **showinfo()** – To display some important information .
2. **showwarning()** – To display some type of Warning.
3. **showerror()** –To display some Error Message.
4. **askquestion()** – To display a dialog box that asks with two options YES or NO.
5. **askokcancel()** – To display a dialog box that asks with two options OK or CANCEL.
6. **askretrycancel()** – To display a dialog box that asks with two options RETRY or CANCEL.
7. **askyesnocancel()** – To display a dialog box that asks with three options YES or NO or CANCEL.

Syntax of the Message-Box Functions:

`messagebox.name_of_function(Title, Message, [, options])`

1. **name_of_function** – Function name that which we want to use .
2. **Title** – Message Box's Title.
3. **Message** – Message that you want to show in the dialog.
4. **Options** –To configure the options.

Askquestion()

This function is used to ask questions to the user. That has only two options YES or NO.

Application of this function:

1. We can use this to ask the user if the User want's to continue.
2. We can use this to ask the user if the User wanted to Submit or not.

Syntax:

`messagebox.askfunction((Title, Message, [, options])`

Example:

- Python3

```
from tkinter import *
```

```
from tkinter import messagebox

# object of TK()

main = Tk()

# function to use the
# askquestion() function

def Submit():

    messagebox.askquestion("Form",

                           "Do you want to Submit")

# setting geometry of window

# instance

main.geometry("100x100")

# creating Window

B1 = Button(main, text = "Submit", command = Submit)

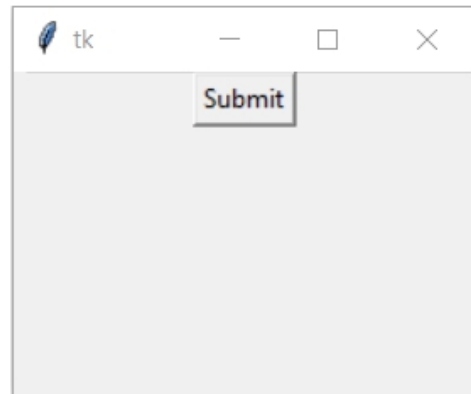
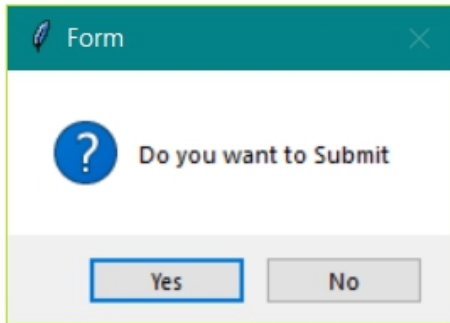
# Button positioning

B1.pack()

# infinite loop till close

main.mainloop()
```

Output:



1. Importing the Libraries

To use the GUI functionality in python we have to import the libraries. In the first line, we are importing Tkinter, and second-line we're importing messagebox library

```
from tkinter import *  
from tkinter import messagebox
```

2. Main window instance

We have to create an instance or object for the window to TK(); Tk() is a function of Tkinter that create a window that can be referred from the main variable

```
main = Tk()
```

3. Set dimension

we set the dimension of the window we can set it in various ways.in this we are setting is by geometry() function of size "100X100".

```
top.geometry("100x100")
```

4. Applying other widget and function

In our example, we create a method named Submit and call the askquestion() and Creating Button and setting it by Pack() function

```
def Submit():  
    messagebox.askquestion("Form", "Do you want to Submit")  
main.geometry("100x100")  
B1 = Button(main, text = "Submit", command = Submit)  
B1.pack()
```

5. mainloop()

This method can be used when all the code is ready to execute. It runs the INFINITE Loop used to run the application. A window will open until the close button is pressed.

ICONS that We can use in Options

1. Error
2. Info
3. Warning
4. Question

We can change the icon of the dialog box. The type of icon that we want to use just depends on the application's need. we have four icons.

Error

```
messagebox.function_name(Title, Message, icon='error')
```

Example-

- Python3

```
# illustration of icon - Error  
  
from tkinter import *  
  
from tkinter import messagebox
```

```
main = Tk()

def check():

    messagebox.askquestion("Form",

                           "Is your name correct?",

                           icon='error')

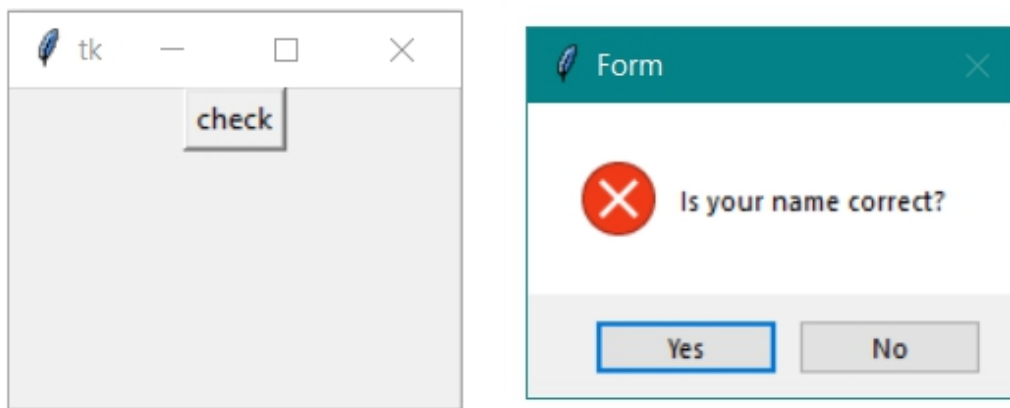
main.geometry("100x100")

B1 = Button(main, text = "check", command = check)

B1.pack()

main.mainloop()
```

Output:



info

messagebox.function_name(Title, Message, icon='info')

Example-

- Python3

```
# illustration of icon - Info

from tkinter import *

from tkinter import messagebox

main = Tk()

def check():

    messagebox.askquestion("Form",

                           "do you want to continue",

                           icon ='info')

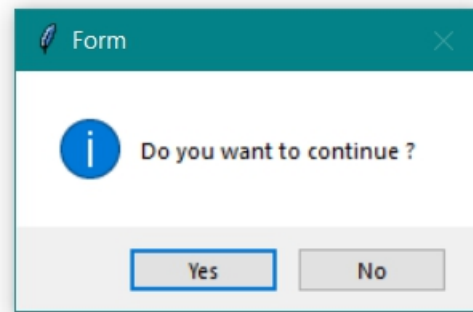
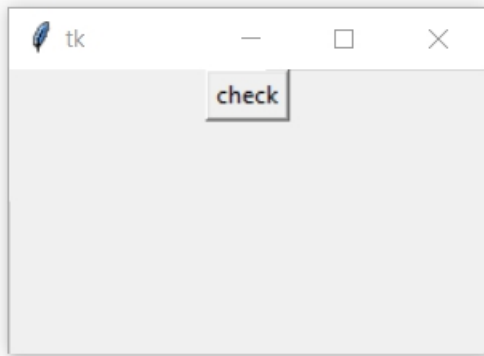
main.geometry("100x100")

B1 = Button(main, text = "check", command = check)

B1.pack()

main.mainloop()
```

Output:



Question

`messagebox.function_name(Title, Message, icon='question')`

Example-

- Python3

```
# illustration of icon - question

from tkinter import *

from tkinter import messagebox

main = Tk()

def check():

    messagebox.askquestion("Form",

                           "are you 18+",

                           icon ='question')

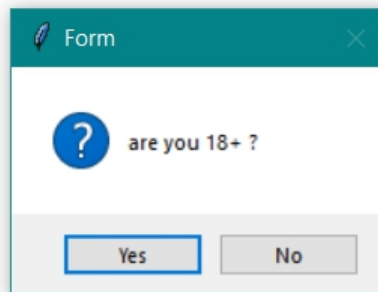
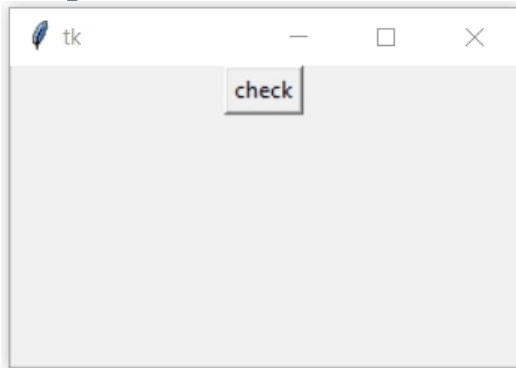
main.geometry("100x100")
```

```
B1 = Button(main, text = "check", command = check)
```

```
B1.pack()
```

```
main.mainloop()
```

Output:



Warning

```
messagebox.function_name(Title, Message, icon='warning')
```

Example-

- Python3

```
# illustration of icon - Warning
```

```
from tkinter import *
```

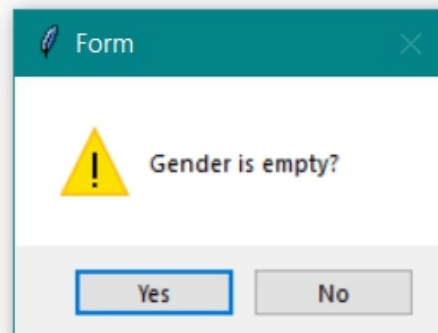
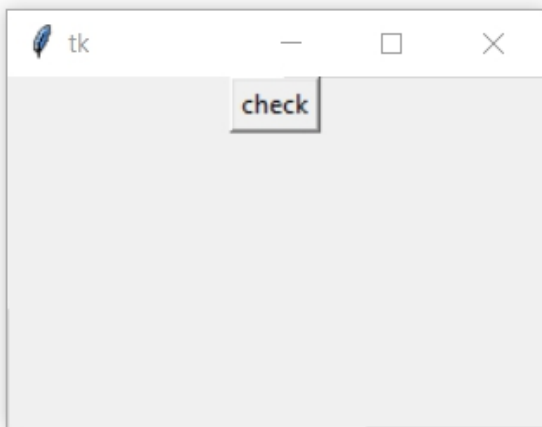
```
from tkinter import messagebox
```

```
main = Tk()
```

```
def check():
```

```
messagebox.askquestion("Form",  
                        "Gender is empty?",  
                        icon='warning')  
  
main.geometry("100x100")  
  
B1 = Button(main, text = "check", command = check)  
B1.pack()  
  
main.mainloop()
```

Output:



CHAPTER 5: Python Tkinter – MessageBox Widget

Python offers multiple options for developing GUI (Graphical User Interface). Out of all the GUI methods, tkinter is the most commonly used method. It is a standard Python interface to the Tk GUI toolkit shipped with Python. Python with tkinter is the fastest and easiest way to create the GUI applications. Creating a GUI using tkinter is an easy task.

Note: For more information, refer to [Python GUI – tkinter](#)

MessageBox Widget

Python Tkinter – MessageBox Widget is used to display the message boxes in the python applications. This module is used to display a message using provides a number of functions.

Syntax:

`messagebox.Function_Name(title, message [, options])`

Parameters:

There are various parameters :

- **Function_Name:** This parameter is used to represents an appropriate message box function.
- **title:** This parameter is a string which is shown as a title of a message box.
- **message:** This parameter is the string to be displayed as a message on the message box.
- **options:** There are two options that can be used are:
 1. **default:** This option is used to specify the default button like ABORT, RETRY, or IGNORE in the message box.
 2. **parent:** This option is used to specify the window on top of which the message box is to be displayed.

Function_Name:

There are functions or methods available in the messagebox widget.

1. **showinfo():** Show some relevant information to the user.
2. **showwarning():** Display the warning to the user.
3. **showerror():** Display the error message to the user.

4. **askquestion():** Ask question and user has to answered in yes or no.
5. **askokcancel():** Confirm the user's action regarding some application activity.
6. **askyesno():** User can answer in yes or no for some action.
7. **askretrycancel():** Ask the user about doing a particular task again or not.

Example:

```
from tkinter import *

from tkinter import messagebox

root = Tk()
root.geometry("300x200")

w = Label(root, text ='GeeksForGeeks', font = "50")
w.pack()

messagebox.showinfo("showinfo", "Information")

messagebox.showwarning("showwarning", "Warning")

messagebox.showerror("showerror", "Error")

messagebox.askquestion("askquestion", "Are you sure?")

messagebox.askokcancel("askokcancel", "Want to continue?")
```

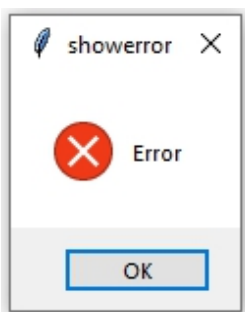
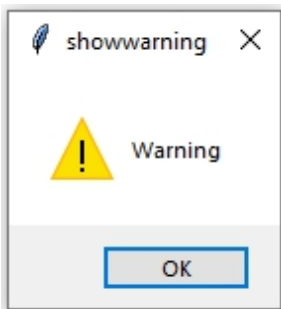
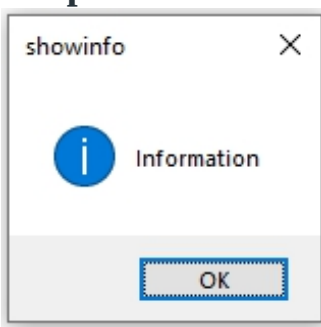


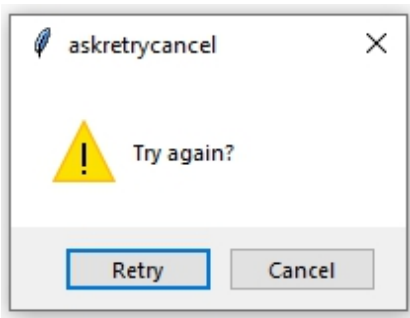
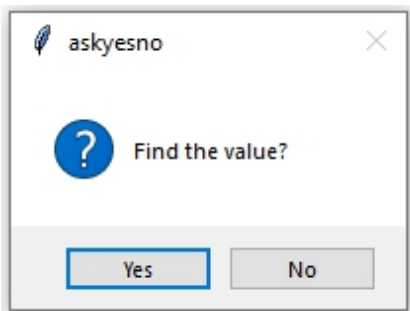
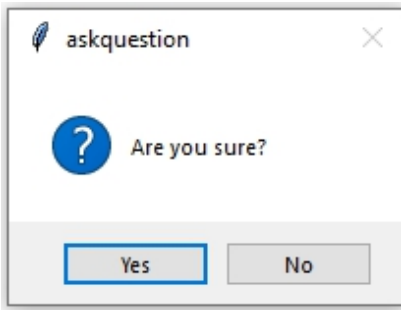
```
messagebox.askyesno("askyesno", "Find the value?")
```

```
messagebox.askretrycancel("askretrycancel", "Try again?")
```

```
root.mainloop()
```

Output:





CHAPTER 6: Create a Yes/No Message Box in Python using tkinter

Python offers a number Graphical User Interface(GUI) framework but Tk interface or [tkinter](#) is the most widely used framework. It is cross-platform

which allows the same code to be run irrespective of the OS platform (Windows, Linux or macOS). Tkinter is lightweight, faster and simple to work with. Tkinter provides a variety of widgets that can be customized using standard attributes and geometry management methods. The Tkinter message box can be used to ask questions or display messages to the user.

Note: For more information, refer to [Python GUI – tkinter](#)

Steps to create a tkinter message box :

1. Import tkinter module

2. import tkinter as tk

from tkinter import *

Note: Name of the module in Python 2.x is 'Tkinter' and in Python 3.x it is 'tkinter'. Python 3.x is used here.

3. Import tkinter messagebox widget

from tkinter import messagebox as mb

4. Create the method that is called to display the Yes/No Message Box

5. def call():

6. res=mb.askquestion('Exit Application', 'Do you really want to exit')

7. if res == 'yes' :

8. root.destroy()

9. else :

10.

 mb.showinfo('Return', 'Returning to main application')

Explanation:

Syntax:

askquestion(title=None, message=None, **options)

Parameter

- **title:** used to give a name which is displayed in as header of the dialog box.
- **message:** question for the user.

Return value: Returns 'yes' when the yes option is clicked and 'no' when the no option is clicked.

Syntax:

```
showinfo(title=None, message=None, **options)
```

Parameter

- **title:** used to give a name which is displayed in as header of the dialog box.
- **message:** information for the user.

Syntax:

```
destroy()
```

This method destroys a widget.

11.

Create the canvas for the button will be placed

12.

```
root=tk.Tk()
```

13.

```
canvas=tk.Canvas(root, width=200, height=200)
```

14.

```
canvas.pack()
```

Explanation:

Syntax:

```
Tk(screenName=None, baseName=None, className='Tk', useTk=1)
```

Used to create the parent window. Tk class is instantiated without any arguments. The name of the parent window can be changed to desired one by changing the value of className argument. Here 'root' is the parent window.

Syntax:

```
Canvas(master, option=value)
```

Parameter:

- **master:** used to represent the parent window. Here 'root' is the master.

- **option:** used to specify border, background color, height, width etc .

Return Value: The method returns a string (.!canvas) .

Syntax:

pack(**options)

Organizes the widgets in blocks before placing in the parent widget. The options can be used to expand, fill and specify side(left, right, top, bottom)

15.

Create the button and place it inside the canvas

16.

b=Button(root, text='Quit Application', command=call)

17.

canvas.create_window(100, 100, window=b)

Explanation:

Syntax:

Button(master=None, options)

Parameter:

- **master:** Here root is the parent window.
- **options:** There are a number of supported options. The options used in this case are text and command.
 - **text:** button text
 - **command:** the action or method that is to be invoked when the button is pressed.

Return Value: The method returns a string (.!button) .

Syntax:

create_window(x, y, **options)

Parameter:

- **x, y:** Specifies the position of the widget(button) within the canvas.
- **options:** There are a variety of options supported like anchor, height, width, state, tags, window. The option used here is

window.

- **window:** window=b where b is the widget(button) to be placed on the canvas.

Return Value: Returns the object ID for the window object.

18.

Call the mainloop() method

19.

root.mainloop()

Explanation:

Syntax:

mainloop()

It is an infinite loop that is called when the program is ready to be run. It waits for an event(mouse clicks) to occur and as soon as the event is received the event is processed. The mainloop() runs as long as the parent window is not destroyed.

The complete program is as follows:

```
# Python program to create
# yes/no message box

import tkinter as tk

from tkinter import *

from tkinter import messagebox as mb

def call():

    res = mb.askquestion('Exit Application',
```

```
'Do you really want to exit')
```

```
if res == 'yes' :
```

```
    root.destroy()
```

```
else :
```

```
    mb.showinfo('Return', 'Returning to main application')
```

```
# Driver's code
```

```
root = tk.Tk()
```

```
canvas = tk.Canvas(root,
```

```
                    width = 200,
```

```
                    height = 200)
```

```
canvas.pack()
```

```
b = Button(root,
```

```
           text ='Quit Application',
```

```
           command = call)
```

```
canvas.create_window(100, 100,
```

```
                    window = b)
```

```
root.mainloop()
```

CHAPTER 7: Change the size of MessageBox – Tkinter

Python have many libraries for GUI. [Tkinter](#) is one of the libraries that provides a Graphical user interface. For the Short Message, we can use the MessageBox Library. It has many functions for the effective interface. In this MessageBox library provide different type of functions to Display **Message Box**.

1. **showinfo()**:- To display some usual Information
2. **showwarning()**:- To display warning to the User
3. **showerror()**:- To display different type of Error
4. **askquestion()**:- to Ask Query to the User

Example:

- Python3

```
# MessageBox Illustration of showinfo() function

from tkinter import *

from tkinter import messagebox

# creating window object

top = Tk()

def Button_1():

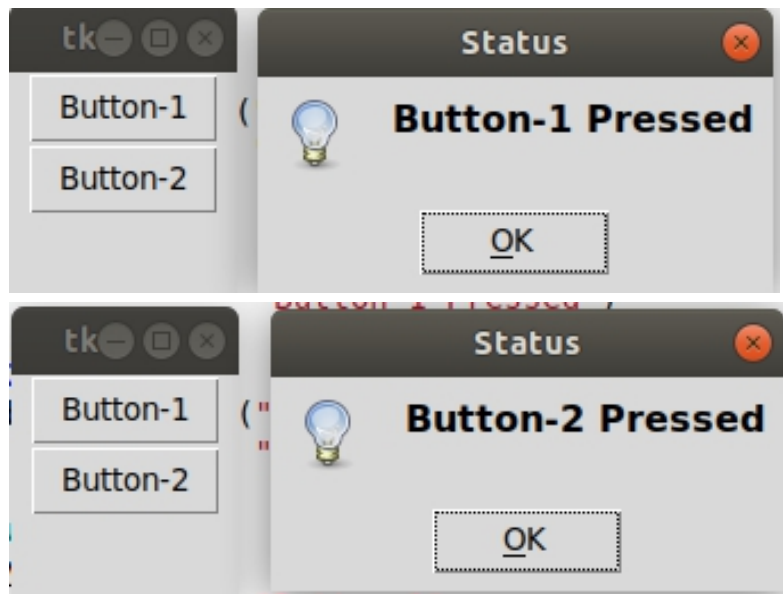
    messagebox.showinfo("Status",

                        "Button-1 Pressed")
```



```
def Button_2():  
    messagebox.showinfo("Status",  
                        "Button-2 Pressed")  
  
# size for window  
top.geometry("100x100")  
  
B1 = Button(top, text = "Button-1",  
            command = Button_1)  
  
B2 = Button(top, text = "Button-2",  
            command = Button_2)  
  
B1.pack()  
B2.pack()  
top.mainloop()
```

Output:



By default, the size of the message box is Fix. **We can't change the size of that Message Box.** Different Boxes have different sizes. However, we can use Different alternative methods for this purpose

- Message Widget
- By Changing ReadMe File

1. Message Widget

MessageBox library doesn't provide the functions to change the configuration of the box. We can use the other function. The message can also be used to display the information. The size of the message is the size of the window so that we can set the size of the message by geometry, pack.

- Python3

```
from tkinter import *
```

```
main = Tk()
```

```
# variable for text
```

```
str_var = StringVar()

# Message Function

label = Message( main, textvariable=str_var,

                  relief=RAISED )

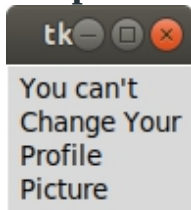
# The size of the text determines
# the size of the messagebox

str_var.set("You can't Change Your Profile Picture ")

label.pack()

main.mainloop()
```

Output:



2. By Changing ReadMe File

This is another alternative option of the Message Box. In this, We are Opening the file readme.txt the length of the content of the readme file determines the size of the messagebox.

Input File:

Welcome

Geeks

to

Geeks

for

Geeks

world

|

- Python3

```
from tkinter import *  
  
from tkinter import messagebox  
  
top = Tk()  
  
def helpfile(filetype):  
  
    if filetype==1:  
  
        with open("read.txt") as f:  
  
            # reading file  
  
            readme = f.read()
```

```
# Display whole message

messagebox.showinfo(title="Title",

                    message = str(readme))

# Driver code

helpfile(1)

top.mainloop()
```

Output:



CHAPTER 8: Different messages in Tkinter

[Tkinter](#) provides a **messagebox** class which can be used to show variety of messages so that user can respond according to those messages. Messages like confirmation message, error message, warning message etc.

In order to use this class one must import this class as shown below:

```
# import all the functions and constants of this class.
```

```
from tkinter.messagebox import *
```

Syntax and uses of different functions of this class –

```
# Ask if operation should proceed;
```

```
# return true if the answer is ok.
```

```
# Ask a question.
```

```
# Ask if operation should be retried;
```

```
# return true if the answer is yes.
```

```
# Ask a question; return true
```

```
# if the answer is yes.
```

```
# Ask a question; return true
```

```
# if the answer is yes, None if cancelled.
```

```
# Show an error message.
```

```
# Show an info message.
```

```
# Show a warning message.
```

Program to demonstrate various messages:

- Python3

```
# importing messagebox class

from tkinter.messagebox import *

# Showing various messages

print(askokcancel("askokcancel", "Ok or Cancel"))

print(askquestion("askquestion", "Question?"))

print(askretrycancel("askretrycancel", "Retry or Cancel"))

print(askyesno("askyesno", "Yes or No"))

print(askyesnocancel("askyesnocancel", "Yes or No or Cancel"))

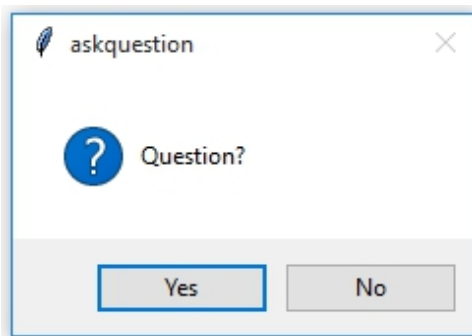
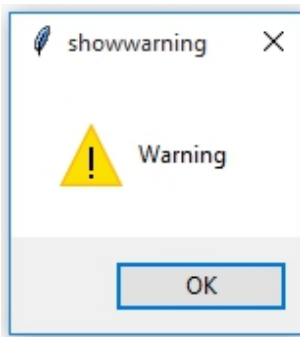
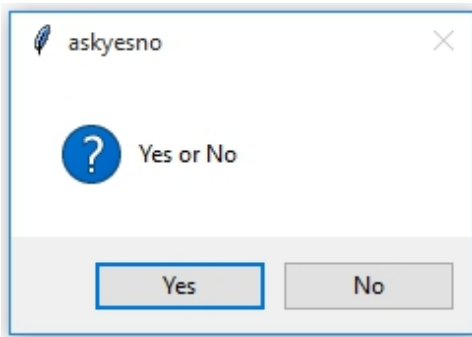
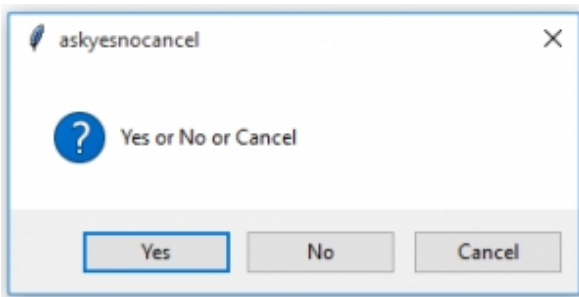
print(showerror("showerror", "Error"))

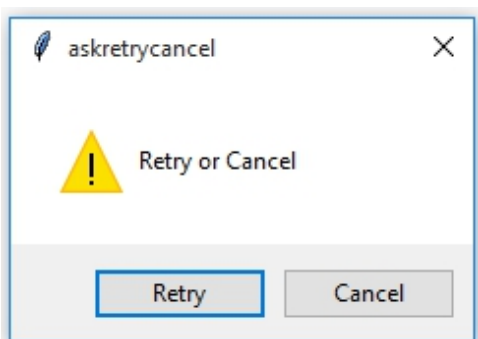
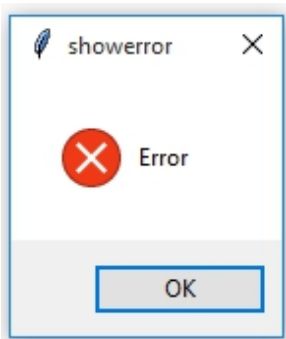
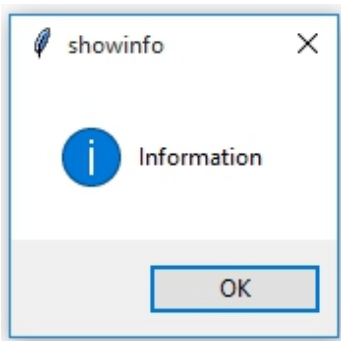
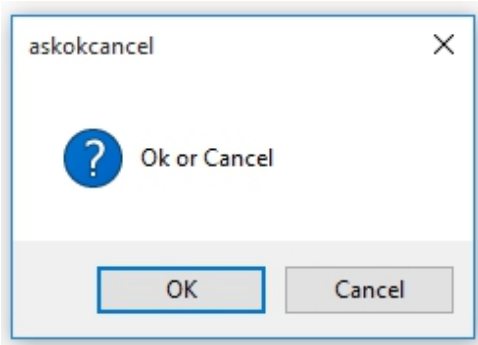
print(showinfo("showinfo", "Information"))

print(showwarning("showwarning", "Warning"))

# print statement is used so that we can
# print the returned value by the function
```

Output:





Note: Note that in above program we don not have to import [Tkinter](#) module only **messagebox** module/class is sufficient because definitions of these

functions is in **messagebox** class.

CHAPTER 9: Change Icon for Tkinter MessageBox

We know many modules and one of them is [Tkinter](#). Tkinter is a module which is a standard interface of Python to the Tk GUI toolkit. This interface Tk and the Tkinter modules, both of them are available on most of the Unix platforms. it is also available on Windows OS and many others. But it is usually a shared library or DLL file, for some cases, it is statically linked with the Python interpreter.

Default icon of MessageBox

Whenever we create a message box using the Tkinter, we always see a similar icon in the message box. Let us see it again with an example. **Example:**

- Python3

```
import tkinter as tk
```

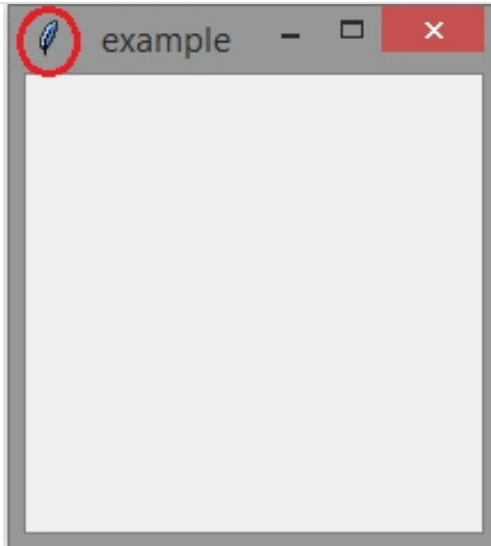
```
win = tk.Tk()
```

```
# as it does not have any mentions
```

```
# of the icon we get a default icon.
```

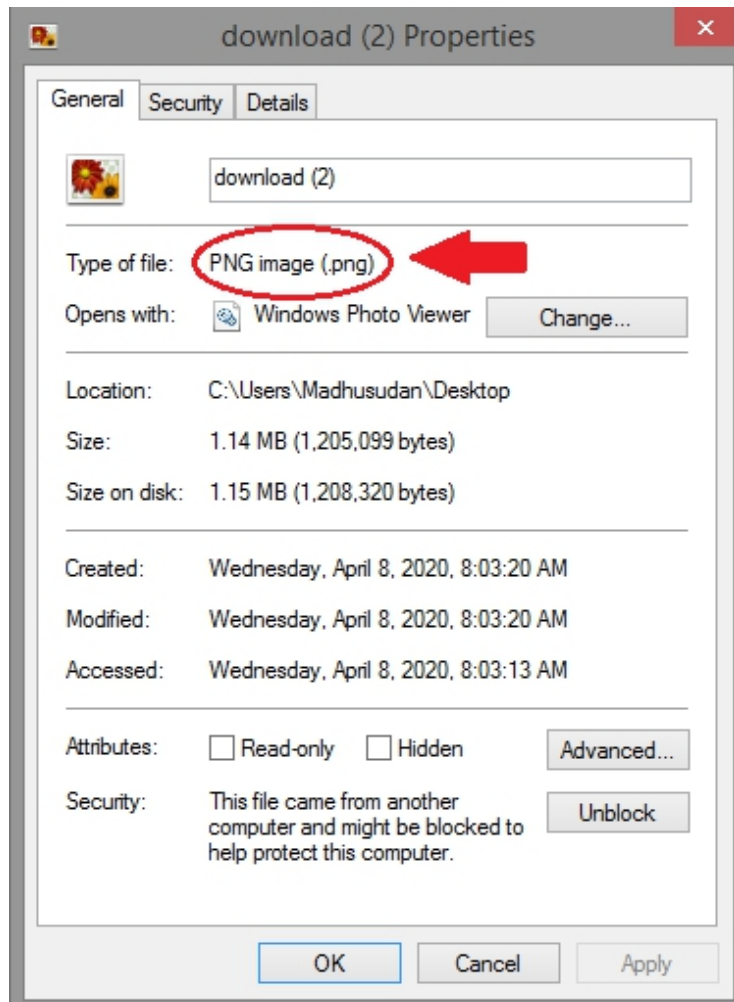
```
win.title("example")
```

```
win.mainloop()
```



Output :

In the above example, within the circle that is the default icon we get, when it is not mentioned in the code. And yes, we can change it according to our wishes. Select the photo you want to keep it as an icon. Select and then [Right click-> Properties]. Type of File you see will be in the format of (.png).



But we need it in the format of **(.ico)**. There might be a question that why we need to convert .png to .ico? Why can't we use it in the format of .png? For that, the answer will be the icon bitmap function (depending on the programming language) should be used to set a bitmap image to the window when the window is iconified. For that just search online for **online iconconverter**, go there, and simply convert the image you want to convert it to **(.ico)**. So we got our image ready to the desired format, next get back to the coding part and let's learn how to change the default icon to our selected icon follow the steps. **Step 1:** Add a line, defining the icon bitmap i.e. `win.iconbitmap(r'')`

import tkinter as to

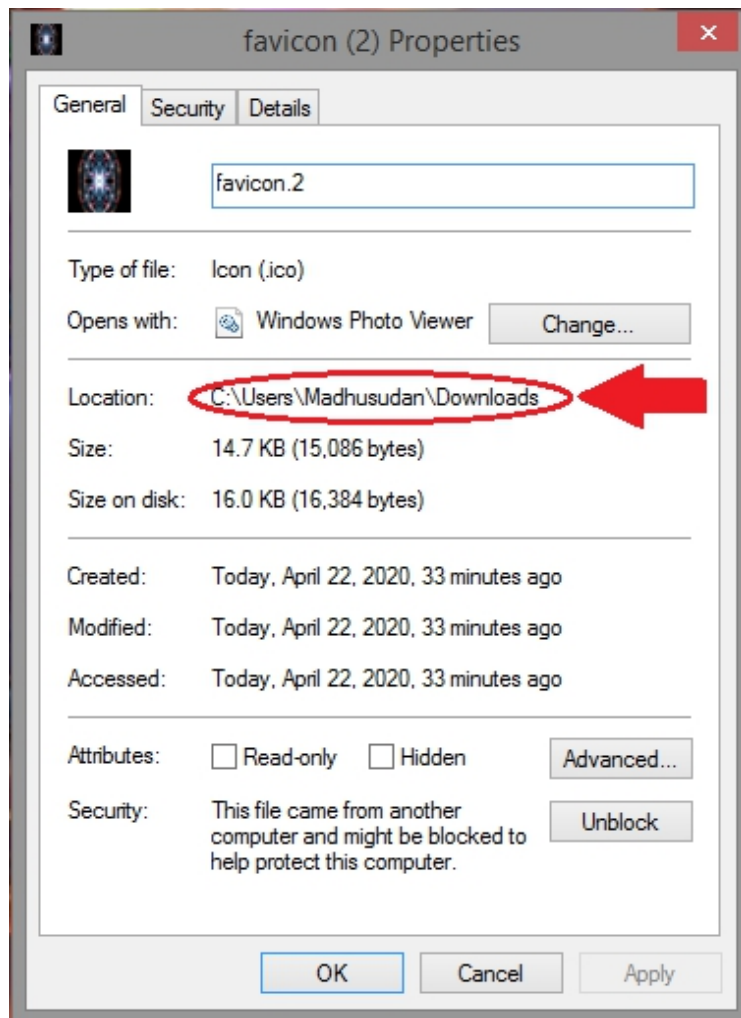
win = tk.Tk()

```
win.title("example")
```

```
win.iconbitmap(r")
```

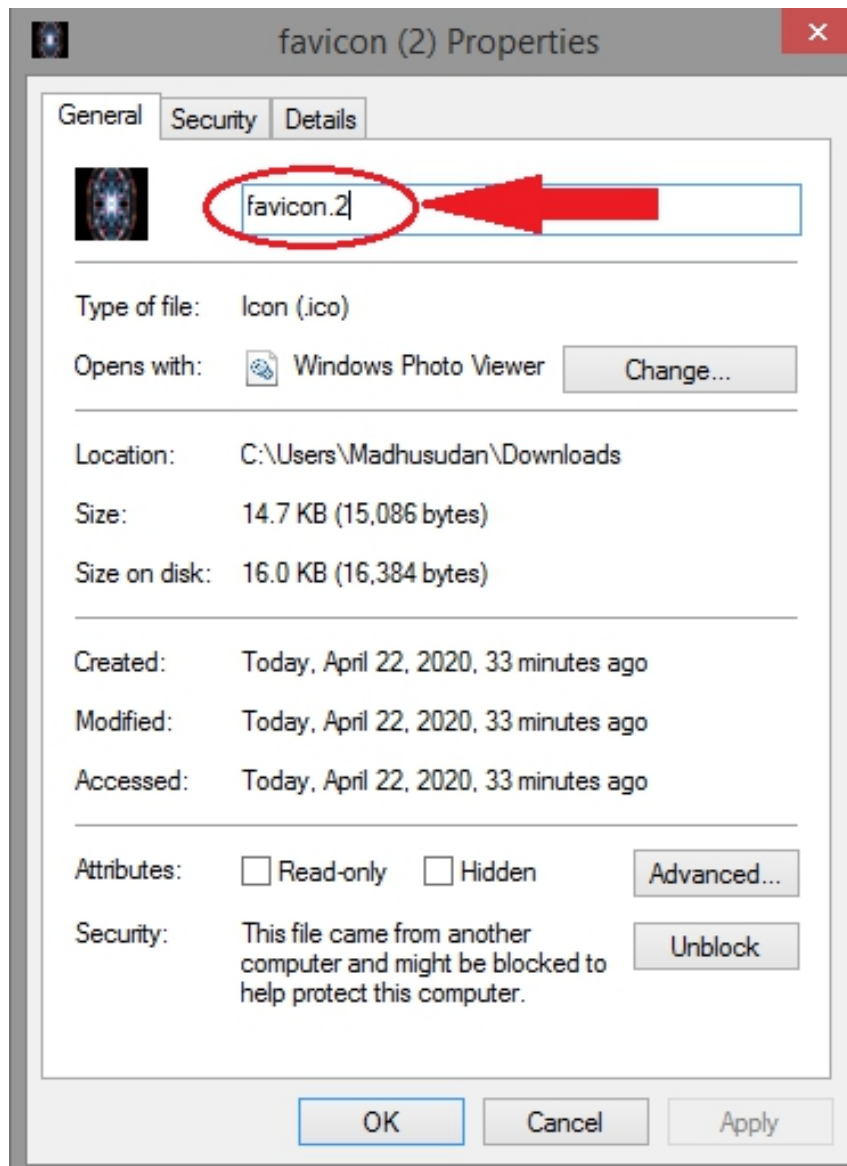
```
win.mainloop()
```

Step 2: Mentioning the file path of the image we want as an icon. Copy the File location and paste it inside “win.iconbitmap(r”)”.



```
win.iconbitmap(r'C:\Users\Madhusudan\Downloads\')
```

Step 3: Mention the file name. Copy the file name and paste it after “\” where you mentioned the file location.



`win.iconbitmap(r'C:\Users\Madhusudan\Downloads\favicon(2).ico')`

Finally, we get the complete code for how we can change the icon. Let's put it together.

- Python3

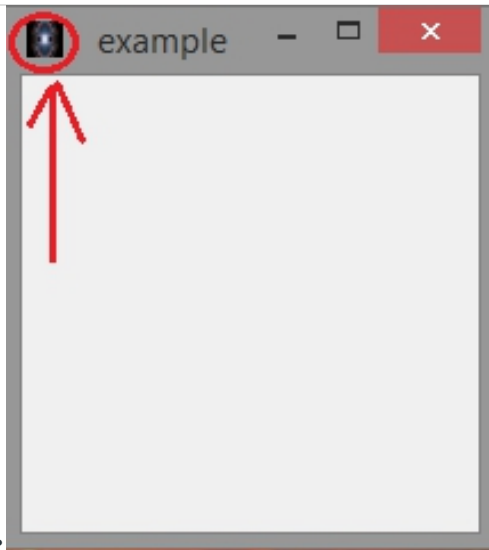
```
import tkinter as tk
```

```
win = tk.Tk()

win.title("example")

win.iconbitmap(r'C:\Users\Madhusudan\Downloads\favicon(2).ico')

win.mainloop()
```



Output :

CHAPTER 10: Tkinter Choose color Dialog

Python provides many options for GUI (Graphical User Interface) development. tkinter is the most commonly used method options apart from all other available alternatives. It is a standard method for developing GUI applications using Tk GUI toolkit.

The steps involved in developing a basic Tkinter app are:

1. Importing the Tkinter module.
2. Creating the main window or the container.
3. Insert as many widgets as you want to the main window.
4. Apply the event Trigger on all the widgets.

The process of importing the Tkinter module is the same as importing any other module in Python.

```
import tkinter
```

Creating choose color dialog box using Tkinter

The Tkinter module has a package in it named **colorchooser**. This package of the Tkinter module helps in developing the color chooser dialog box. This package has a function named **askcolor()** that plays a major role.

askcolor()

This function belongs to the *colorchooser* package of Tkinter module. The function helps in creating a color chooser dialog box. As soon as the function is called, it makes the color chooser dialogue box pop up. The function returns the hexadecimal code of the color selected by the user.

Syntax:

```
colorchooser.askcolor()
```

Example:

- Python3

```
# Python program to create color chooser dialog box
```



```
# importing tkinter module

from tkinter import *

# importing the choosecolor package

from tkinter import colorchooser

# Function that will be invoked when the
# button will be clicked in the main window

def choose_color():

    # variable to store hexadecimal code of color

    color_code = colorchooser.askcolor(title="Choose color")

    print(color_code)

root = Tk()

button = Button(root, text = "Select color",

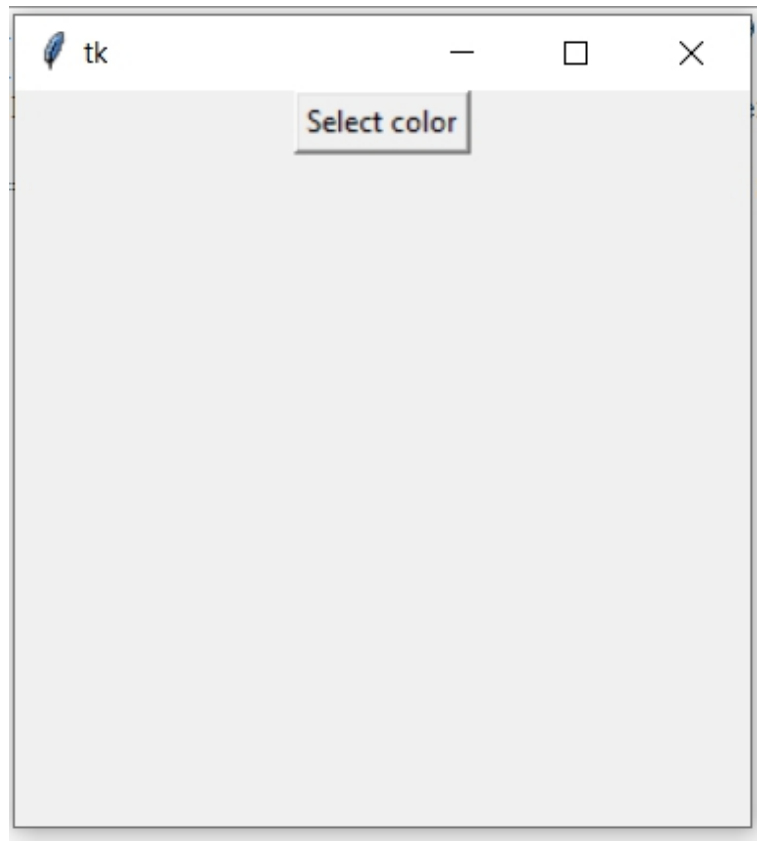
                command = choose_color)

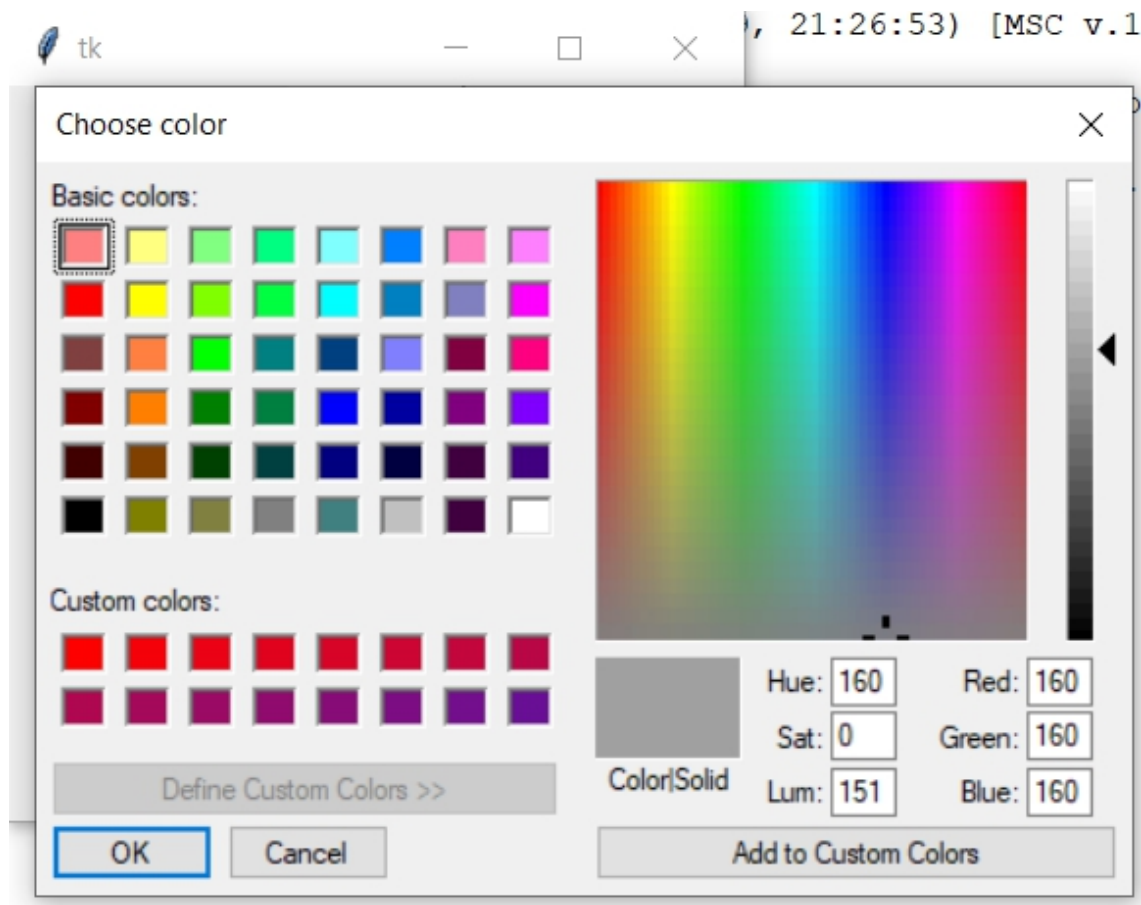
button.pack()

root.geometry("300x300")

root.mainloop()
```

Output:





Note: The color chooser dialog box may vary for different operating systems.

CHAPTER 11: Popup Menu in Tkinter

Tkinter is Python's standard GUI (Graphical User Interface) package. It is one of the most commonly used packages for GUI applications which comes with the Python itself.

Note: For more information, refer to Python GUI – tkinter

Menu Widget

Menus are an important part of any GUI. A common use of menus is to provide convenient access to various operations such as saving or opening a

file, quitting a program, or manipulating data. Toplevel menus are displayed just under the title bar of the root or any other top-level windows.

Syntax:

```
menu = Menu(master, **options)
```

Note: For more information, refer to [Python | Menu widget in Tkinter](#)

Popup Menu

Popup menus are context menus that appear on user interaction. This menu can be shown anywhere on the client window. below is the python code to create a popup menu using Tkinter library.

```
#creating popup menu in tkinter

import tkinter

class A:

    #creates parent window

    def __init__(self):

        self.root = tkinter.Tk()

        self.root.geometry('500x500')

        self.frame1 = tkinter.Label(self.root,

                                     width = 400,

                                     height = 400,

                                     bg = '#AAAAAA')
```

```
self.frame1.pack()
```

```
#create menu
```

```
def popup(self):
```

```
self.popup_menu = tkinter.Menu(self.root,
```

tearoff = 0)

```
self.popup_menu.add_command(label = "say hi",
```

```
command = lambda:self.hey("hi"))
```

```
self.popup_menu.add_command(label = "say hello",
```

```
command = lambda:self.hey("hello"))
```

```
self.popup_menu.add_separator()
```

```
self.popup_menu.add_command(label = "say bye",
```

```
command = lambda:self.hey("bye"))
```

```
#display menu on right click
```

```
def do_popup(self,event):
```

try:

```
self.popup_menu.tk_popup(event.x_root,
```

event.y_root)

finally:

```
        self.popup_menu.grab_release()

def hey(self,s):

    self.frame1.configure(text = s)

def run(self):

    self.popup()

    self.root.bind("<Button-3>",self.do_popup)

    tkinter.mainloop()

a = A()

a.run()
```

Functions

- **Menu(root):** creates the menu.
- **add_command(label, command):** adds the commands on the menu, the command argument calls the function hey() when that option is clicked.
- **add_separator():** adds a separator.
- **tk_popup(x, y):** posts the menu at the position given as arguments
- **grab_release():** releases the event grab
- **bind(key, event):** binds the mouse event.

PART 4: Geometry Management=====

CHAPTER 1: place() method in Tkinter

The **Place** geometry manager is the simplest of the three general geometry managers provided in Tkinter. It allows you explicitly set the position and size of a window, either in absolute terms, or relative to another window. You can access the **place** manager through the **place()** method which is available for all standard widgets. It is usually not a good idea to use **place()** for ordinary window and dialog layouts; its simply too much work to get things working as they should. Use the **pack()** or **grid()** managers for such purposes. **Syntax:**

```
widget.place(relx = 0.5, rely = 0.5, anchor = CENTER)
```

Note : place() method can be used with **grid()** method as well as with **pack()** method. **Code #1:**

Python3

```
# Importing tkinter module

from tkinter import * from tkinter.ttk import *


# creating Tk window

master = Tk()


# setting geometry of tk window

master.geometry("200x200")
```

```
# button widget
```

```
b1 = Button(master, text = "Click me !")
```

```
b1.place(relx = 1, x = -2, y = 2, anchor = NE)
```

```
# label widget
```

```
l = Label(master, text = "I'm a Label")
```

```
l.place(anchor = NW)
```

```
# button widget
```

```
b2 = Button(master, text = "GFG")
```

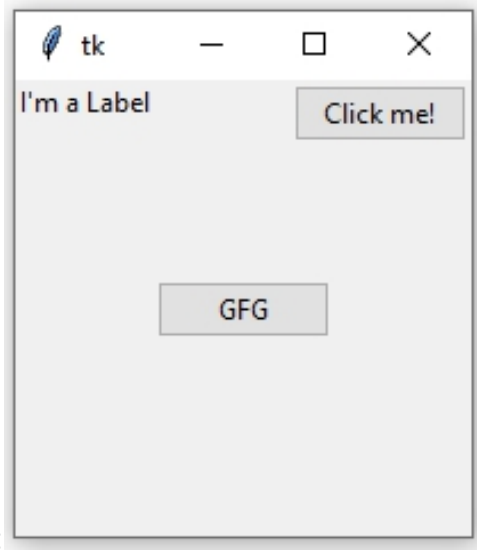
```
b2.place(relx = 0.5, rely = 0.5, anchor = CENTER)
```

```
# infinite loop which is required to
```

```
# run tkinter program infinitely
```

```
# until an interrupt occurs
```

```
mainloop()
```

Output:

When we use **pack()** or **grid()** managers, then it is very easy to put two different widgets separate to each other but putting one of them inside other is a bit difficult. But this can easily be achieved by **place()** method. In **place()** method, we can use **in_** option to put one widget inside other. **Code #2:**

Python3

```
# Importing tkinter module

from tkinter import * from tkinter.ttk import *

# creating Tk window

master = Tk()

# setting geometry of tk window

master.geometry("200x200")

# button widget

b2 = Button(master, text = "GFG")
```

```
b2.pack(fill = X, expand = True, ipady = 10)

# button widget

b1 = Button(master, text = "Click me !")

# This is where b1 is placed inside b2 with in_ option

b1.place(in_ = b2, relx = 0.5, rely = 0.5, anchor = CENTER)

# label widget

l = Label(master, text = "I'm a Label")

l.place(anchor = NW)

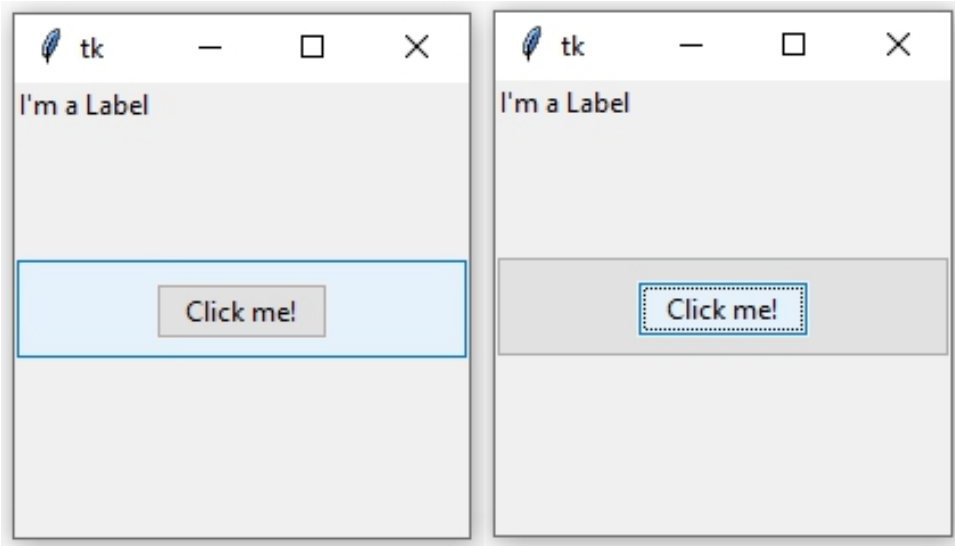
# infinite loop which is required to

# run tkinter program infinitely

# until an interrupt occurs

mainloop()
```

Output: In below images notice that one button is placed inside the other.



CHAPTER 2: grid() method in Tkinter

The **Grid** geometry manager puts the widgets in a 2-dimensional table. The master widget is split into a number of rows and columns, and each “cell” in the resulting table can hold a widget. The **grid** manager is the most flexible of the geometry managers in [Tkinter](#). If you don’t want to learn how and when to use all three managers, you should at least make sure to learn this one. Consider the following example –

<label 1>	<entry 2>	<image>	
<label 1>	<entry 2>		
<checkboxbutton>		<button 1>	<button 2>

Creating this layout using the **pack** manager is possible, but it takes a number of extra frame widgets, and a lot of work to make things look good. If you use the **grid** manager instead, you only need one call per widget to get everything laid out properly. Using the **grid** manager is easy. Just create the widgets, and use the **grid** method to tell the manager in which row and column to place them. You don’t have to specify the size of the grid

beforehand; the manager automatically determines that from the widgets in it.

Code #1:

- Python3

```
# import tkinter module

from tkinter import * from tkinter.ttk import *

# creating main tkinter window/toplevel

master = Tk()

# this will create a label widget

l1 = Label(master, text = "First:")

l2 = Label(master, text = "Second:")

# grid method to arrange labels in respective
# rows and columns as specified

l1.grid(row = 0, column = 0, sticky = W, pady = 2)

l2.grid(row = 1, column = 0, sticky = W, pady = 2)

# entry widgets, used to take entry from user

e1 = Entry(master)

e2 = Entry(master)
```

```
# this will arrange entry widgets

e1.grid(row = 0, column = 1, pady = 2)

e2.grid(row = 1, column = 1, pady = 2)


# infinite loop which can be terminated by keyboard

# or mouse interrupt

mainloop()
```

Output:



Code #2: Creating the layout which is shown above.

- Python3

```
# import tkinter module

from tkinter import * from tkinter.ttk import *


# creating main tkinter window/toplevel

master = Tk()


# this will create a label widget

l1 = Label(master, text = "Height")

l2 = Label(master, text = "Width")
```

```
# grid method to arrange labels in respective
# rows and columns as specified

l1.grid(row = 0, column = 0, sticky = W, pady = 2)

l2.grid(row = 1, column = 0, sticky = W, pady = 2)


# entry widgets, used to take entry from user

e1 = Entry(master)

e2 = Entry(master)


# this will arrange entry widgets

e1.grid(row = 0, column = 1, pady = 2)

e2.grid(row = 1, column = 1, pady = 2)


# checkbutton widget

c1 = Checkbutton(master, text = "Preserve")

c1.grid(row = 2, column = 0, sticky = W, columnspan = 2)


# adding image (remember image should be PNG and not JPG)

img = PhotoImage(file = r"C:\Users\Admin\Pictures\capture1.png")

img1 = img.subsample(2, 2)


# setting image with the help of label

Label(master, image = img1).grid(row = 0, column = 2,

                                columnspan = 2, rowspan = 2, padx = 5, pady = 5)
```

```
# button widget

b1 = Button(master, text = "Zoom in")

b2 = Button(master, text = "Zoom out")


# arranging button widgets

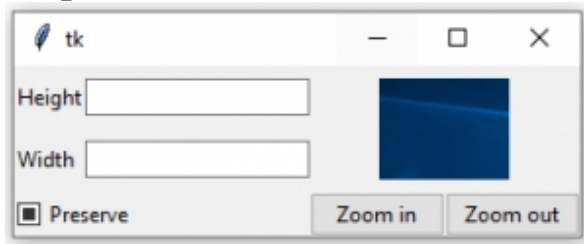
b1.grid(row = 2, column = 2, sticky = E)

b2.grid(row = 2, column = 3, sticky = E)


# infinite loop which can be terminated
# by keyboard or mouse interrupt

mainloop()
```

Output:



Warning: Never mix **grid()** and **pack()** in the same master window.

CHAPTER 3: `grid_location()` and `grid_size()` method

Tkinter is used to develop GUI (Graphical User Interface) applications. It supports a variety of widgets as well as a variety of widget methods or universal widget methods.

grid_location() method –

This method returns a tuple containing (column, row) of any specified widget. Since this method is a widget method you cannot use it with master object (or Tk() object). In order to use this method, you should first create a Frame and treat it as a parent (or master).

Syntax: `widget.grid_location(x, y)`

Parameters:

x and y are the positions, relative to the upper left corner of the widget (parent widget).

In below example, `grid_location()` is used to get the location of the widget in the Frame widget.

- Python3

```
# This imports all functions in tkinter module

from tkinter import * from tkinter.ttk import *

# creating master window

master = Tk()

# This method is used to get the position
```



```
# of the desired widget available in any
# other widget

def click(event):

    # Here retrieving the size of the parent
    # widget relative to master widget

    x = event.x_root - f.winfo_rootx()

    y = event.y_root - f.winfo_rooty()

    # Here grid_location() method is used to
    # retrieve the relative position on the
    # parent widget

    z = f.grid_location(x, y)

    # printing position
    print(z)

# Frame widget, will work as
# parent for buttons widget

f = Frame(master)
f.pack()

# Button widgets

b = Button(f, text = "Button")

b.grid(row = 2, column = 3)
```

```
c = Button(f, text = "Button2")

c.grid(row = 1, column = 0)


# Here binding click method with mouse

master.bind("<Button-1>", click)


# infinite loop

mainloop()
```

grid_size() method –

This method is used to get the total number of grids present in any parent widget. This is a widget method so one cannot use it with master object. One has to create a Frame widget.

Syntax: (columns, rows) = widget.grid_size()

Return Value: It returns total numbers of columns and rows (grids).

Below is the Python code-

- Python3

```
# This imports all functions in tkinter module

from tkinter import * from tkinter.ttk import *


# creating master window
```

```
master = Tk()

# This method is used to get the size
# of the desired widget i.e number of grids
# available in the widget

def grids(event):

    # Here, grid_size() method is used to get
    # the total number grids available in frame
    # widget

    x = f.grid_size()

    # printing (columns, rows)
    print(x)

# Frame widget, will work as
# parent for buttons widget

f = Frame(master)
f.pack()

# Button widgets

b = Button(f, text = "Button")

b.grid(row = 1, column = 2)

c = Button(f, text = "Button2")
```

```
c.grid(row = 1, column = 0)

# Here binding click method with mouse

master.bind("<Button-1>", grids)

# infinite loop

mainloop()
```

Output:

Every time you click the mouse button it will return the same value until more widgets are not added OR number of rows and columns are not increased.

(3, 2)

pack() method in Tkinter

The Pack geometry manager packs widgets relative to the earlier widget. Tkinter literally packs all the widgets one after the other in a window. We can use options like **fill**, **expand**, and **side** to control this geometry manager. Compared to the **grid** manager, the **pack** manager is somewhat limited, but it's much easier to use in a few, but quite common situations:

- Put a widget inside a frame (or any other container widget), and have it fill the entire frame
- Place a number of widgets on top of each other

- Place a number of widgets side by side

Code #1: Putting a widget inside frame and filling entire frame. We can do this with the help of **expand** and **fill** options.

- Python3

```
# Importing tkinter module

from tkinter import * from tkinter.ttk import *

# creating Tk window

master = Tk()

# creating a Frame which can expand according
# to the size of the window

pane = Frame(master)

pane.pack(fill = BOTH, expand = True)

# button widgets which can also expand and fill
# in the parent widget entirely

# Button 1

b1 = Button(pane, text = "Click me !")

b1.pack(fill = BOTH, expand = True)

# Button 2
```

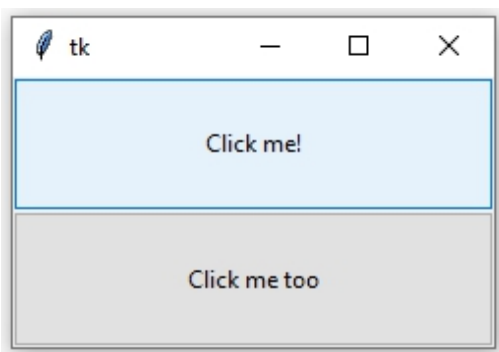
```
b2 = Button(pane, text = "Click me too")

b2.pack(fill = BOTH, expand = True)

# Execute Tkinter

master.mainloop()
```

Output:



Code #2: Placing widgets on top of each other and side by side. We can do this by side option.

- Python3

```
# Importing tkinter module

from tkinter import *

# from tkinter.ttk import *

# creating Tk window

master = Tk()

# creating a Frame which can expand according
```

```
# to the size of the window

pane = Frame(master)

pane.pack(fill = BOTH, expand = True)


# button widgets which can also expand and fill
# in the parent widget entirely

# Button 1

b1 = Button(pane, text = "Click me !",

            background = "red", fg = "white")

b1.pack(side = TOP, expand = True, fill = BOTH)


# Button 2

b2 = Button(pane, text = "Click me too",

            background = "blue", fg = "white")

b2.pack(side = TOP, expand = True, fill = BOTH)


# Button 3

b3 = Button(pane, text = "I'm also button",

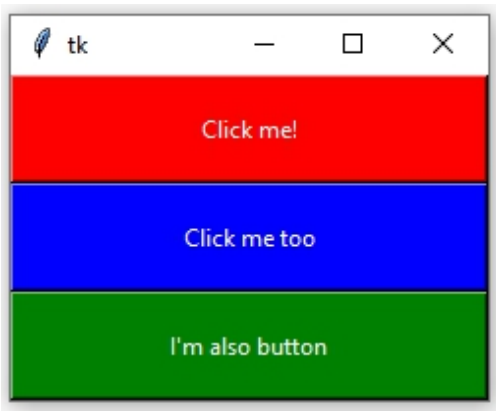
            background = "green", fg = "white")

b3.pack(side = TOP, expand = True, fill = BOTH)


# Execute Tkinter

master.mainloop()
```

Output:



Code #3:

- Python3

```
# Importing tkinter module

from tkinter import *

# from tkinter.ttk import *

# creating Tk window

master = Tk()

# creating a Frame which can expand according
# to the size of the window

pane = Frame(master)

pane.pack(fill = BOTH, expand = True)
```



```
# button widgets which can also expand and fill

# in the parent widget entirely

# Button 1

b1 = Button(pane, text = "Click me !",

            background = "red", fg = "white")

b1.pack(side = LEFT, expand = True, fill = BOTH)


# Button 2

b2 = Button(pane, text = "Click me too",

            background = "blue", fg = "white")

b2.pack(side = LEFT, expand = True, fill = BOTH)


# Button 3

b3 = Button(pane, text = "I'm also button",

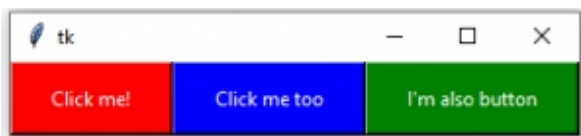
            background = "green", fg = "white")

b3.pack(side = LEFT, expand = True, fill = BOTH)


# Execute Tkinter

master.mainloop()
```

Output:



CHAPTER 4: `forget_pack()` and `forget_grid()` method in Tkinter

If we want to unmap any widget from the screen or toplevel then `forget()` method is used. There are two types of forget method `forget_pack()` (similar to `forget()`) and `forget_grid()` which are used with `pack()` and `grid()` method respectively.

`forget_pack()` method –

Syntax: `widget.forget_pack()`

widget can be any valid widget which is visible.

Code #1:

```
# Imports tkinter and ttk module

from tkinter import *

from tkinter.ttk import *

# toplevel window

root = Tk()

# method to make widget invisible

# or remove from toplevel

def forget(widget):
```

```
# This will remove the widget from toplevel

# basically widget do not get deleted

# it just becomes invisible and loses its position

# and can be retrieve

widget.forget()


# method to make widget visible

def retrieve(widget):

    widget.pack(fill = BOTH, expand = True)


# Button widgets

b1 = Button(root, text = "Btn 1")

b1.pack(fill = BOTH, expand = True)


# See, in command forget() method is passed

b2 = Button(root, text = "Btn 2", command = lambda : forget(b1))

b2.pack(fill = BOTH, expand = True)

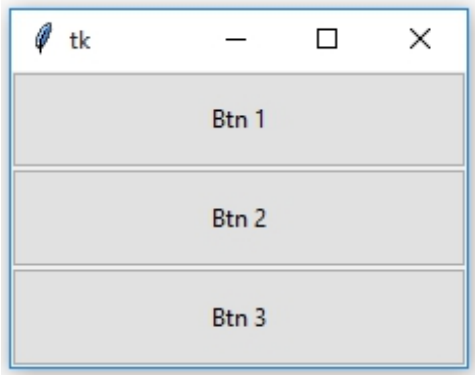

# In command retrieve() method is passed

b3 = Button(root, text = "Btn 3", command = lambda : retrieve(b1))

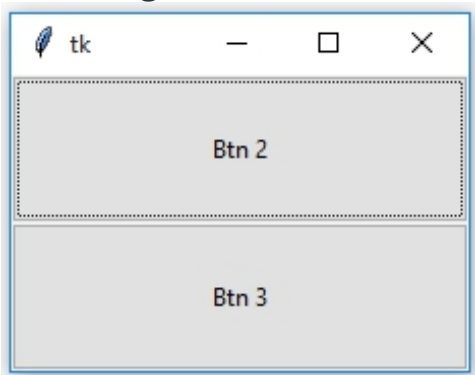
b3.pack(fill = BOTH, expand = True)
```

```
# infinite loop, interrupted by keyboard or mouse  
  
mainloop()
```

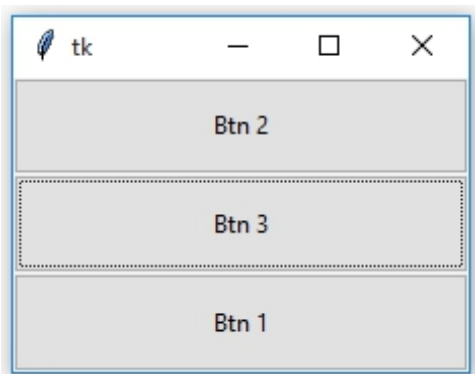
Output:



After forget



After retrieval



Notice the difference in the position of Button 1 before and after forget as well as after retrieval.

forget_grid() method –

Syntax: widget.forget_grid()

widget can be any valid widget which is visible.

Note : This method can be used only with **grid()** geometry methods.

Code #2:

```
# Imports tkinter and ttk module

from tkinter import *

from tkinter.ttk import *


# toplevel window

root = Tk()


# method to make widget invisible

# or remove from toplevel

def forget(widget):


    # This will remove the widget from toplevel

    # basically widget do not get deleted

    # it just becomes invisible and loses its position

    # and can be retrieve

    widget.grid_forget()


# method to make widget visible
```

```
def retrieve(widget):

    widget.grid(row = 0, column = 0, ipady = 10, pady = 10, padx = 5)

# Button widgets

b1 = Button(root, text = "Btn 1")

b1.grid(row = 0, column = 0, ipady = 10, pady = 10, padx = 5)

# See, in command forget() method is passed

b2 = Button(root, text = "Btn 2", command = lambda : forget(b1))

b2.grid(row = 0, column = 1, ipady = 10, pady = 10, padx = 5)

# In command retrieve() method is passed

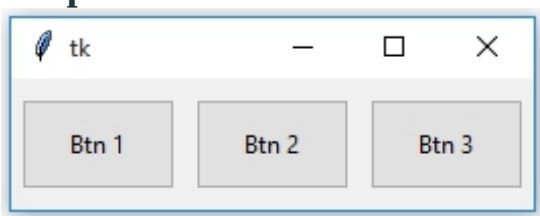
b3 = Button(root, text = "Btn 3", command = lambda : retrieve(b1))

b3.grid(row = 0, column = 2, ipady = 10, pady = 10, padx = 5)

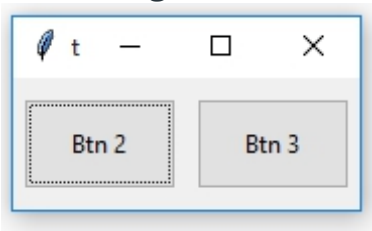
# infinite loop, interrupted by keyboard or mouse

mainloop()
```

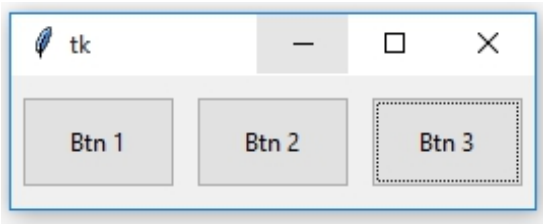
Output:



After Forget



After Retrieval



Notice that the position of Button 1 remains same after **forget** and **retrieval**. With `grid_forget()` method, you can place it at any grid after retrieval but generally, the original grid is chosen.

CHAPTER 5: PanedWindow Widget in Tkinter

Tkinter supports a variety of widgets to make GUI more and more attractive and functional. The **PanedWindow** widget is a geometry manager widget, which can contain one or more child widgets **panes**. The child widgets can be resized by the user, by moving separator lines **sashes** using the mouse.

Syntax: `PanedWindow(master, **options)`

Parameters:

master: parent widget or main `Tk()` object

options: which are passed in config method or directly in the constructor

PanedWindow can be used to implement common 2-panes or 3-panes but multiple panes can be used.

Code #1:PanedWindow with only two panes

```
# Importing everything from tkinter module

from tkinter import * from tkinter import ttk

# main tkinter window

root = Tk()

# panedwindow object

pw = PanedWindow(orient ='vertical')

# Button widget

top = ttk.Button(pw, text ="Click Me !\nI'm a Button")

top.pack(side = TOP)

# This will add button widget to the panedwindow

pw.add(top)

# Checkbutton Widget

bot = Checkbutton(pw, text ="Choose Me !")

bot.pack(side = TOP)

# This will add Checkbutton to panedwindow
```



```
pw.add(bot)

# expand is used so that widgets can expand
# fill is used to let widgets adjust itself
# according to the size of main window

pw.pack(fill = BOTH, expand = True)

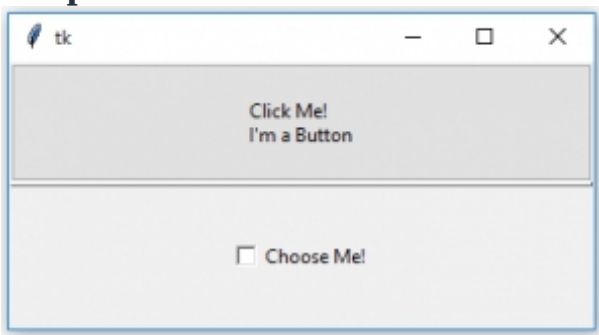
# This method is used to show sash

pw.configure(sashrelief = RAISED)

# Infinite loop can be destroyed by
# keyboard or mouse interrupt

mainloop()
```

Output:



Code #2: PanedWindow with multiple panes

```
# Importing everything from tkinter module

from tkinter import * from tkinter import ttk
```

```
# main tkinter window

root = Tk()

# panedwindow object

pw = PanedWindow(orient ='vertical')

# Button widget

top = ttk.Button(pw, text ="Click Me !\nI'm a Button")

top.pack(side = TOP)

# This will add button widget to the panedwindow

pw.add(top)

# Checkbutton Widget

bot = Checkbutton(pw, text ="Choose Me !")

bot.pack(side = TOP)

# This will add Checkbutton to panedwindow

pw.add(bot)

# adding Label widget

label = Label(pw, text ="I'm a Label")

label.pack(side = TOP)

pw.add(label)
```

```
# Tkinter string variable

string = StringVar()

# Entry widget with some styling in fonts

entry = Entry(pw, textvariable = string, font = ('arial', 15, 'bold'))

entry.pack()

# Focus force is used to focus on particular

# widget that means widget is already selected for operations

entry.focus_force()

pw.add(entry)

# expand is used so that widgets can expand

# fill is used to let widgets adjust itself

# according to the size of main window

pw.pack(fill = BOTH, expand = True)

# This method is used to show sash

pw.configure(sashrelief = RAISED)

# Infinite loop can be destroyed by

# keyboard or mouse interrupt

mainloop()
```

Output:



CHAPTER 6: Geometry Method in Python Tkinter

Tkinter is a Python module that is used to develop GUI (Graphical User Interface) applications. It comes along with Python, so you do not have to install it using the **pip** command.

Tkinter provides many methods, one of them is the **geometry()** method. This method is used to set the dimensions of the Tkinter window and is used to set the position of the main window on the user's desktop.

Tkinter window without using geometry method.

- Python3

```
# importing only those functions which are needed
```

```
from tkinter import Tk, mainloop, TOP

from tkinter.ttk import Button

# creating tkinter window

root = Tk()

# Create Button and add some text

button = Button(root, text = 'Geeks')

# pady is used for giving some padding in y direction

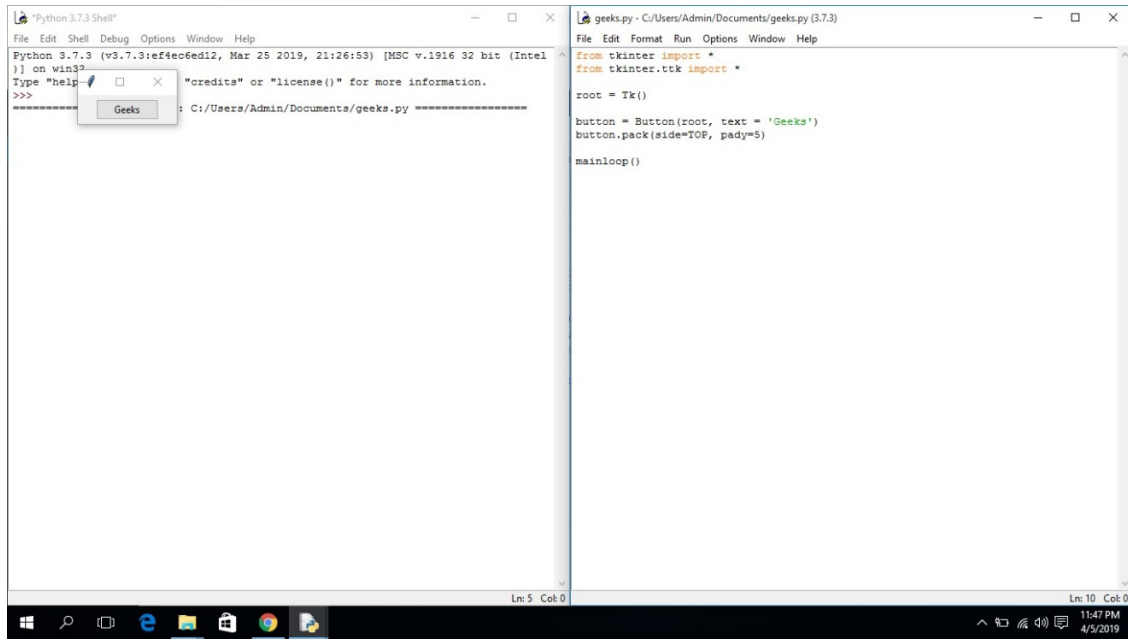
button.pack(side = TOP, pady = 5)

# Execute Tkinter

root.mainloop()
```

Output:

As soon as you run the application, you'll see the position of the Tkinter window is at the northwest position of the screen and the size of the window is also small as shown in the output below.



Tkinter window without geometry method

Examples of Tkinter Geometry Method in Python

Now, let us see a few examples of using the geometry method in Python Tkinter.

Using the Tkinter geometry method to set the dimensions of the window

- Python3

```
# importing only those functions which
```

```
# are needed
```

```
from tkinter import Tk, mainloop, TOP
```

```
from tkinter.ttk import Button
```

```
# creating tkinter window
```

```
root = Tk()

# creating fixed geometry of the
# tkinter window with dimensions 150x200
root.geometry('200x150')

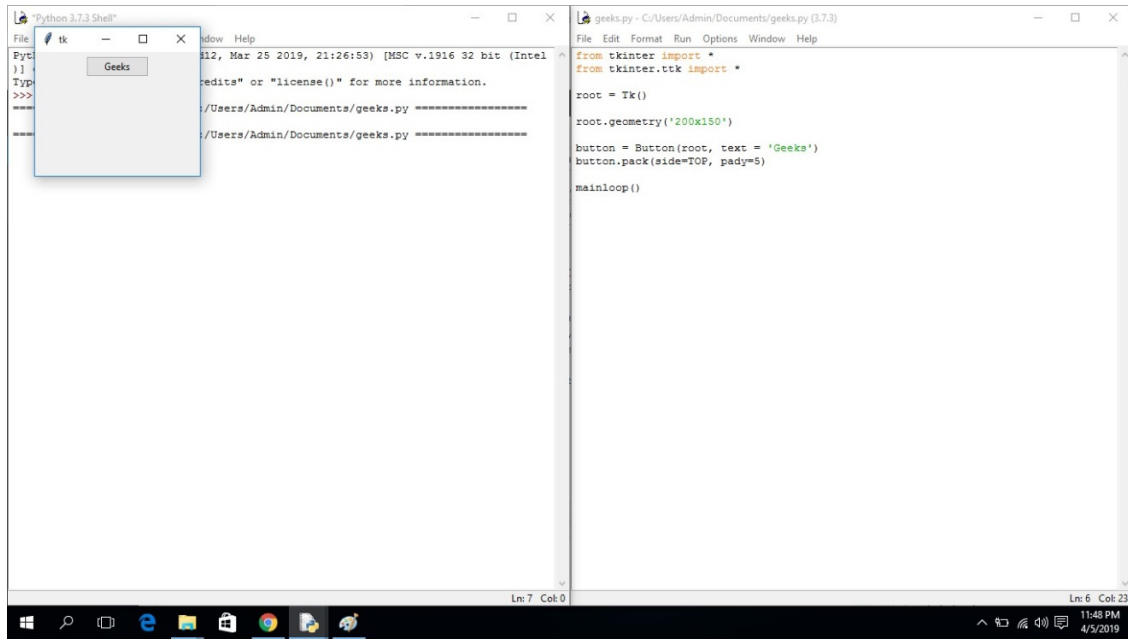
# Create Button and add some text
button = Button(root, text = 'Geeks')

button.pack(side = TOP, pady = 5)

# Execute Tkinter
root.mainloop()
```

Output:

After running the application, you'll see that the size of the Tkinter window has changed, but the position on the screen is the same.



geometry method to set window dimensions

Using the geometry method to set the dimensions and positions of the Tkinter window.

- Python3

```
# importing only those functions which
```

```
# are needed
```

```
from tkinter import Tk, mainloop, TOP
```

```
from tkinter.ttk import Button
```

```
# creating tkinter window
```

```
root = Tk()
```



```
# creating fixed geometry of the

# tkinter window with dimensions 150x200

root.geometry('200x150+400+300')


# Create Button and add some text

button = Button(root, text = 'Geeks')

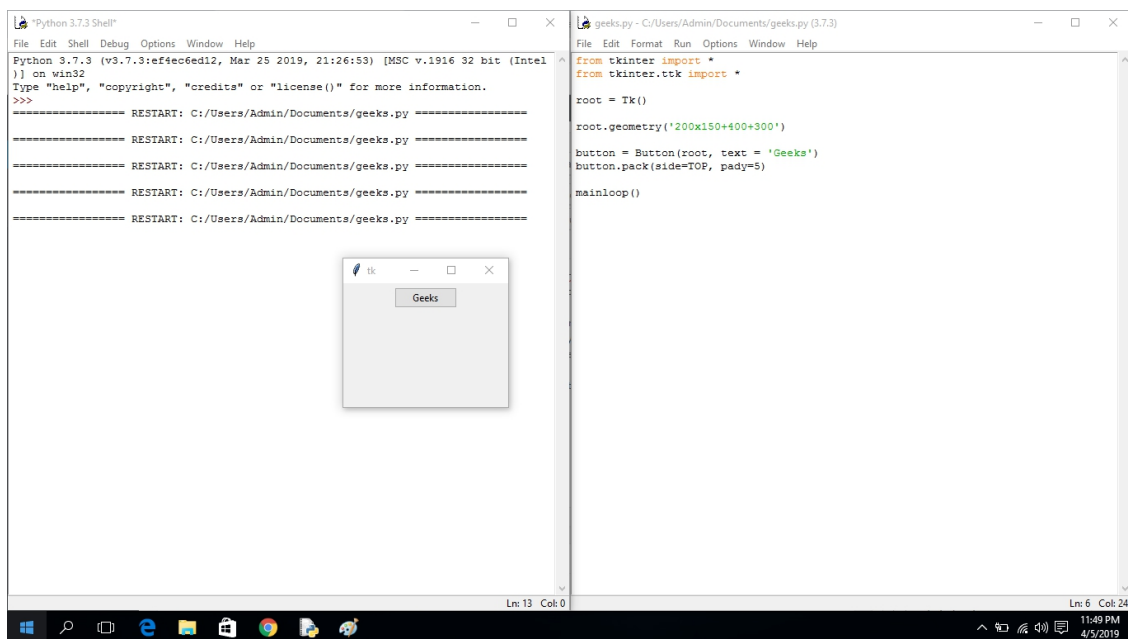
button.pack(side = TOP, pady = 5)


# Execute Tkinter

root.mainloop()
```

Output:

When you run the application, you'll observe that the position and size both are changed. Now the Tkinter window is appearing at a different position (400 shifted on X-axis and 300 shifted on Y-axis).



geometry method to set the dimensions and position of the Tkinter window

Note: We can also pass a variable argument in the geometry method, but it should be in the form **(variable1) x (variable2)**; otherwise, it will raise an error.

CHAPTER 7: Setting the position of TKinter labels

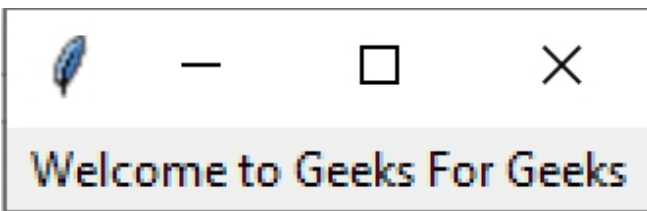
Tkinter is the standard GUI library for Python. Tkinter in Python comes with a lot of good widgets. Widgets are standard GUI elements, and the Label will also come under these Widgets

Note: For more information, refer to Python GUI – tkinter

Label:

Tkinter Label is a widget that is used to implement display boxes where you can place text or images. The text displayed by this widget can be changed by the developer at any time you want. It is also used to perform tasks such as to underline the part of the text and span the text across multiple lines.

Example:



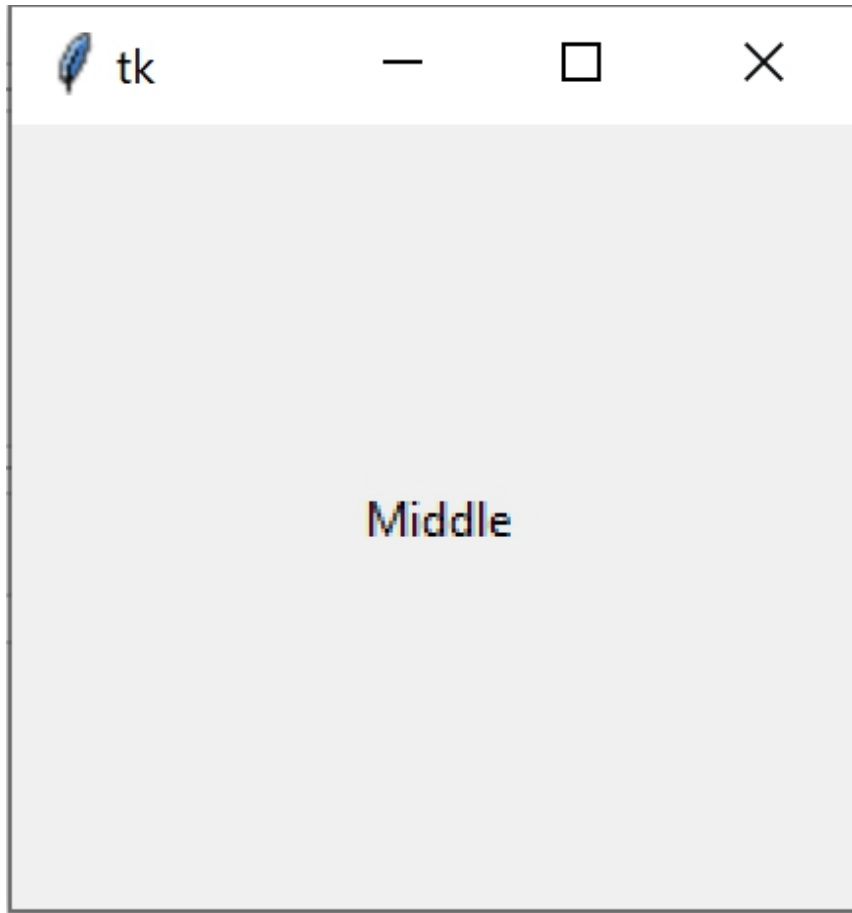
Setting the position of Tkinter labels

We can use `place()` method to set the position of the Tkinter labels.

Example 1: Placing label at the middle of the window


```
root.mainloop()
```

Output:



Example 2: Placing label at the lower left side of window

- Python3

```
# Import Module
```

```
import tkinter as tk
```

```
# Create Object

root = tk.Tk()

# Create Label and add some text

Lower_left = tk.Label(root,text ='Lower_left')

# using place method we can set the position of label

Lower_left.place(relx = 0.0,

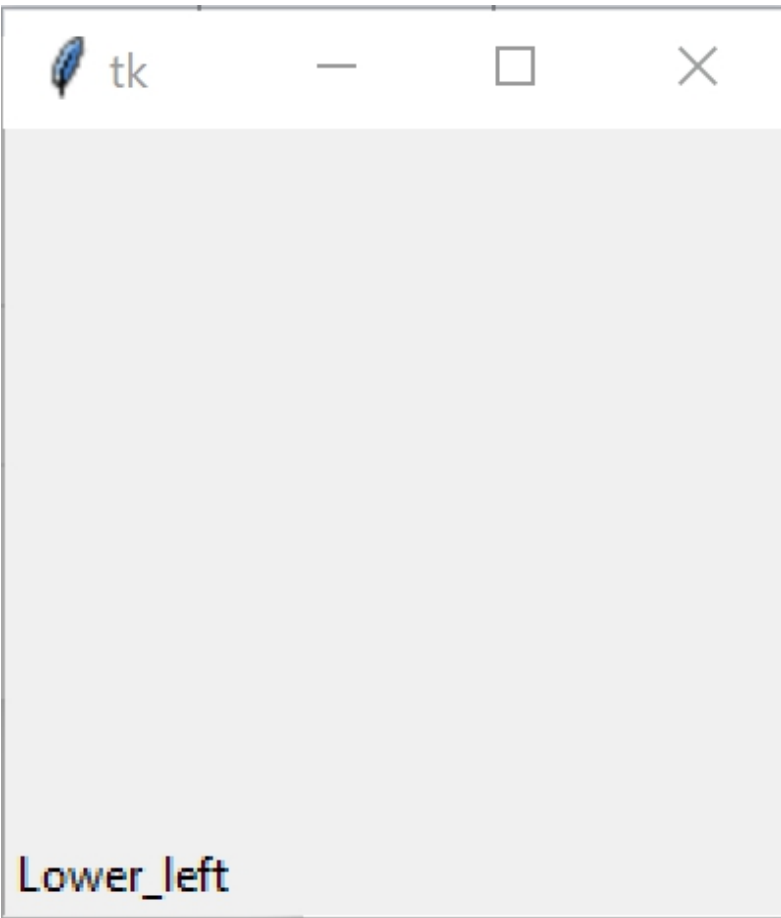
                  rely = 1.0,

                  anchor ='sw')

# Execute Tkinter

root.mainloop()
```

Output



Example 3: Placing label at the upper right side of window

- Python3

```
# Import Module

import tkinter as tk

# Create Object

root = tk.Tk()
```

```
# Create Label and add some text

Upper_right = tk.Label(root,text ='Upper_right')


# using place method we can set the position of label

Upper_right.place(relx = 1.0,

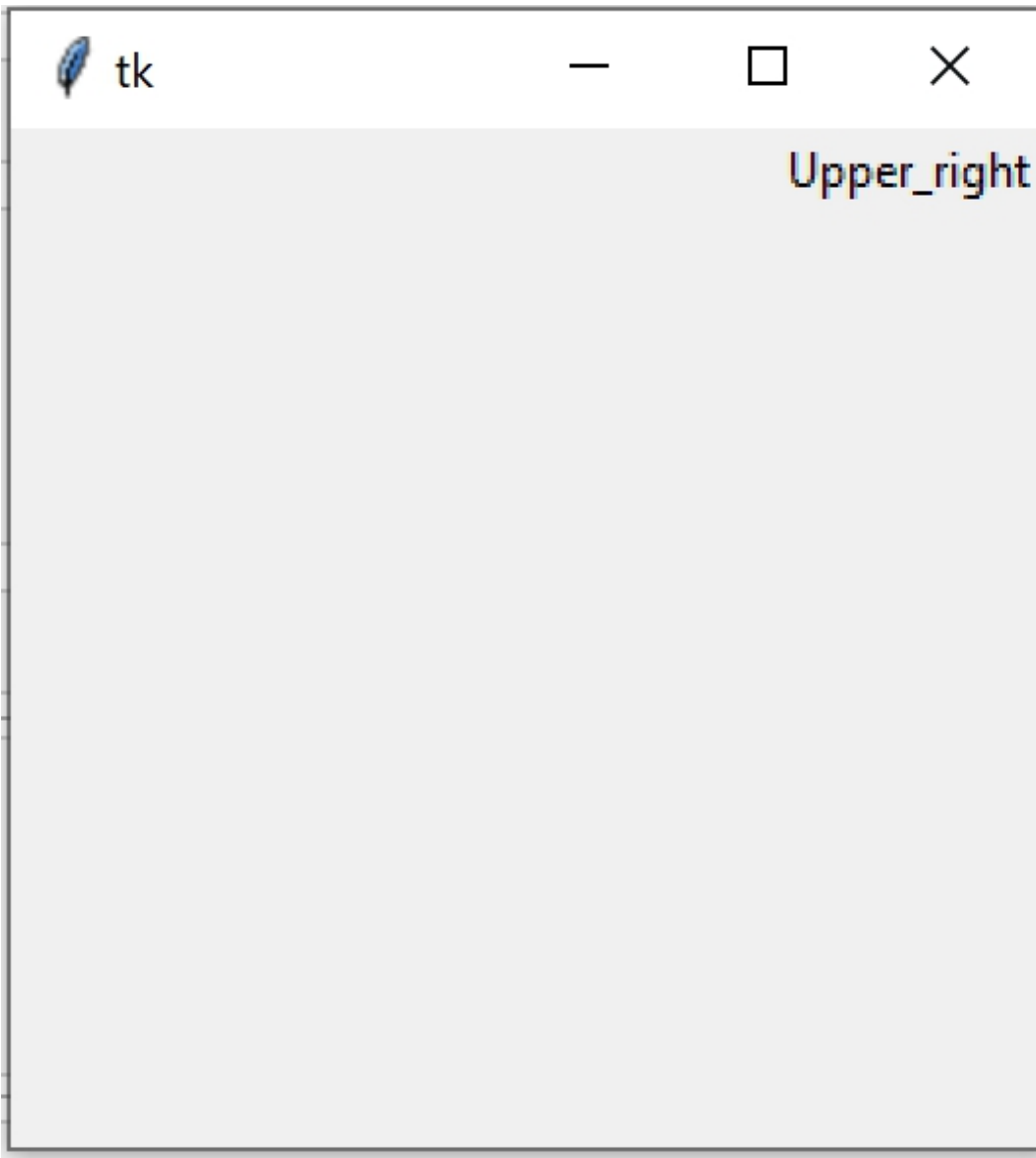
                  rely = 0.0,

                  anchor ='ne')


# Execute Tkinter

root.mainloop()
```

Output



PART 5: Binding Functions

CHAPTER 1: Binding function in Tkinter

[Tkinter](#) is a GUI (Graphical User Interface) module that is widely used in desktop applications. It comes along with the Python, but you can also install it externally with the help of *pip* command.

It provides a variety of Widget classes and functions with the help of which one can make our GUI more attractive and user-friendly in terms of both looks and functionality.

The binding function is used to deal with the events. We can bind **Python's Functions** and methods to an event as well as we can bind these functions to any particular widget.

Code #1: Binding mouse movement with tkinter Frame.

Python3

```
# Import all files from  
# tkinter and overwrite  
# all the tkinter files  
# by tkinter.ttk
```

```
from tkinter import *

from tkinter.ttk import *


# creates tkinter window or root window

root = Tk()

root.geometry('200x100')


# function to be called when mouse enters in a frame

def enter(event):

    print('Button-2 pressed at x = % d, y = % d'%(event.x, event.y))


# function to be called when mouse exits the frame

def exit_(event):

    print('Button-3 pressed at x = % d, y = % d'%(event.x, event.y))


# frame with fixed geometry

frame1 = Frame(root, height = 100, width = 200)


# these lines are showing the
# working of bind function
# it is universal widget method

frame1.bind('<Enter>', enter)

frame1.bind('<Leave>', exit_)


frame1.pack()
```

```
mainloop()
```

Code #2: Binding Mouse buttons with Tkinter Frame

Python3

```
# Import all files from
# tkinter and overwrite
# all the tkinter files
# by tkinter.ttk

from tkinter import *

from tkinter.ttk import *

# creates tkinter window or root window

root = Tk()

root.geometry('200x100')

# function to be called when button-2 of mouse is pressed

def pressed2(event):

    print('Button-2 pressed at x = % d, y = % d'%(event.x, event.y))

# function to be called when button-3 of mouse is pressed

def pressed3(event):
```

```
print('Button-3 pressed at x = % d, y = % d'%(event.x, event.y))

## function to be called when button-1 is double clocked

def double_click(event):

    print('Double clicked at x = % d, y = % d'%(event.x, event.y))

frame1 = Frame(root, height = 100, width = 200)

# these lines are binding mouse
# buttons with the Frame widget
frame1.bind('<Button-2>', pressed2)
frame1.bind('<Button-3>', pressed3)
frame1.bind('<Double 1>', double_click)

frame1.pack()

mainloop()
```

Code #3: Binding keyboard buttons with the root window (tkinter main window).

Python3

```
# Import all files from
# tkinter and overwrite
```

```
# all the tkinter files

# by tkinter.ttk

from tkinter import *

from tkinter.ttk import *


# function to be called when
# keyboard buttons are pressed
def key_press(event):

    key = event.char

    print(key, 'is pressed')


# creates tkinter window or root window

root = Tk()

root.geometry('200x100')


# here we are binding keyboard
# with the main window
root.bind('<Key>', key_press)


mainloop()
```

Note: When we bind keyboard buttons with the tkinter window, whenever we press special characters we will only get space while in the case of alphabets and numerical we will get actual values (in the string).

CHAPTER 2: Binding Function with double click with Tkinter ListBox

Prerequisites: [Python GUI – tkinter](#), [Python | Binding function in Tkinter Tkinter](#) in Python is GUI (Graphical User Interface) module which is widely used for creating desktop applications. It provides various basic widgets to build a GUI program.

To bind Double click with Listbox we use [Binding functions in Python](#) and then we execute the required actions based on the item selected in Listbox.

Below is the implementation:

```
from tkinter import *

def go(event):

    cs = Lb.curselection()

    # Updating label text to selected option

    w.config(text=Lb.get(cs))

    # Setting Background Colour

    for list in cs:

        if list == 0:

            top.configure(background='red')

        elif list == 1:

            top.configure(background='green')
```

```
        elif list == 2:

            top.configure(background='yellow')

        elif list == 3:

            top.configure(background='white')


top = Tk()

top.geometry('250x275')

top.title('Double Click')


# Creating Listbox

Lb = Listbox(top, height=6)

# Inserting items in Listbox

Lb.insert(0, 'Red')

Lb.insert(1, 'Green')

Lb.insert(2, 'Yellow')

Lb.insert(3, 'White')


# Binding double click with left mouse

# button with go function

Lb.bind('<Double-1>', go)

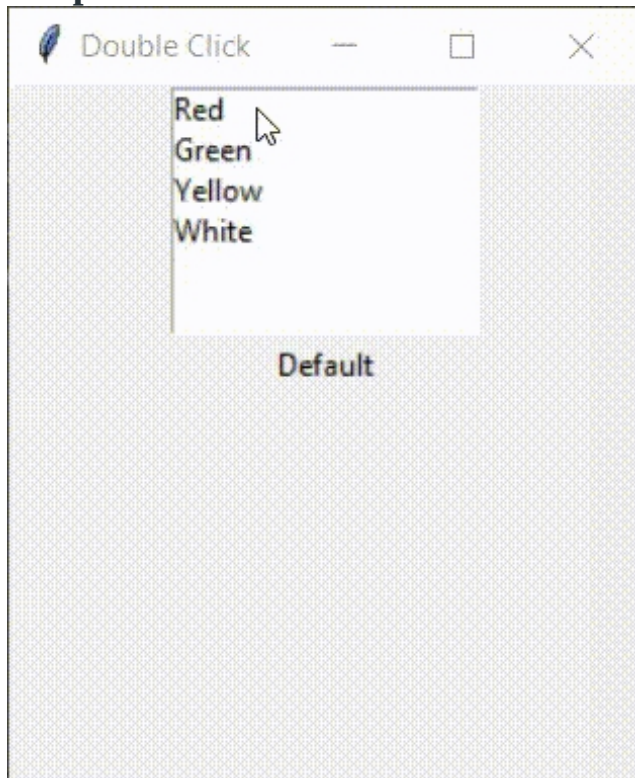
Lb.pack()


# Creating Edit box to show selected option

w = Label(top, text='Default')
```

```
w.pack()  
top.mainloop()
```

Output:



CHAPTER 3: Right Click menu using Tkinter

Python 3.x comes bundled with the **Tkinter** module that is useful for making GUI based applications. Of all the other frameworks supported by Python Tkinter is the simplest and fastest. Tkinter offers a plethora of widgets that

can be used to build GUI applications along with the main event loop that keeps running in the background until the application is closed manually.

Note: For more information, refer to [Python GUI – tkinter](#)

Tkinter provides a mechanism to deal with events. The event is any action that must be handled by a piece of code inside the program. Events include mouse clicks, mouse movements or a keystroke of a user. Tkinter uses event sequences that allow the users to bind events to handlers for each widget.

Syntax:

```
widget.bind(event, handler)
```

Tkinter widget is capable of capturing a variety of events such as Button, ButtonRelease, Motion, Double-Button, FocusIn, FocusOut, Key and many more. The code in this article basically deals with event handling where a popup menu with various options is displayed as soon as right-click is encountered on the parent widget. The Menu widget of Tkinter is used to implement toplevel, pulldown, and popup menus.

1. Import tkinter module

```
import tkinter
```

2. Import tkinter sub-module

```
from tkinter import *
```

3. Creating the parent widget

```
root = Tk()
```

Syntax: Tk(screenName=None, baseName=None, className='Tk', useTk=1)

Parameter: In this example, Tk class is instantiated without arguments.

Explanation:

The Tk() method creates a blank parent widget with close, maximize, and minimize buttons on the top.

4. Creating the label to be displayed

```
L = Label(root, text="Right-click to display menu", width=40, height=20)
```

Syntax: Label(master, **options)

Parameter:

- **master:** The parent window (root) acts as the master.
- **options:** Label() method supports the following options – text, anchor, bg, bitmap, bd, cursor, font, fg, height, width, image, justify, relief, padx, pady, textvariable, underline and wraplength. Here the text option is used display informative text to the user and width and height specifies the position of the Label Widget in the parent window .

Explanation:

The Label widget is used to display text or images corresponding to a widget. The text displayed on the screen can further be formatted using the other options available under Label widget.

5. Positioning the label

```
L.pack()
```

Syntax: pack(options)

Parameter:

- **options:** The options supported by pack() method are expand, fill and side which are used to position the widget on the parent window. However, pack() method is used without any options here.

Explanation:

The pack() method is used to position the child widget in the parent widget.

6. Creating the menu

```
m = Menu(root, tearoff=0)
```

Syntax: Menu(master, options)

Parameter:

- **master:** root is the master or parent widget.
- **options:** The options supported by Menu widget are title, tearoff, selectcolor, font, fg, postcommand, relief, image, bg, bd, cursor, activeforeground, activeborderwidth and activebackground. The 'tearoff' option is used here.

Explanation:

The tearoff is used to detach menus from the main window creating floating menus. If tearoff=1, it creates a menu with dotted lines at the top which when clicked the menu tears off the parent window and becomes floating. To restrict the menu in the main window tearoff=0 here.

7. Adding options to the menu

```
8. m.add_command(label="Cut")
9. m.add_command(label="Copy")
    10.
    m.add_command(label="Paste")
    11.
    m.add_command(label="Reload")
    12.
    m.add_separator()
```

```
m.add_command(label="Rename")
```

Syntax: add_command(options)

Parameter:

- **options:** The options available are label, command, underline and accelerator. The label option is used here to specify names of the menu items.

Explanation:

The add_command() method adds menu items to the menu. The add_separator() creates a thin line between the menu items.

```
13.
def do_popup(event):
    14.
    try:
        15.
        m.tk_popup(event.x_root, event.y_root)
    16.
```

```
finally:  
    17.  
        m.grab_release()
```

18.

Syntax: tk_popup(x_root, y_root)

Parameter: This procedure posts a menu at a given position on the screen, x_root and y_root is the current mouse position relative to the upper left corner of the screen.

Explanation:

This method is the event handler. When the event(right click) happens the method is called and the menu appears on the parent widget at the position where the event occurs. The finally block ensures that the grab_release() method releases the event grab.

```
19.  
    L.bind("<Button-3>", do_popup)
```

Syntax: bind(event, handler)

Parameter:

- **event:** Here, right click is the event and it is denoted by <Button-3>.
- **handler:** The handler is a piece of code that performs some particular task when the event triggered.

Explanation:

The right mouse button is pressed with the mouse pointer over the widget and this triggers an event which is handled by the event handler(do_popup() function). The do_popup() function displays the menu.

20.

Run the application

```
mainloop()
```

Syntax: mainloop()

Parameter: Takes no arguments.

Explanation:

It acts like an infinite loop which keeps the application running until the main window is closed manually.

```
import tkinter

from tkinter import *

root = Tk()

L = Label(root, text="Right-click to display menu",

           width = 40, height = 20)
L.pack()

m = Menu(root, tearoff = 0)
m.add_command(label="Cut")
m.add_command(label="Copy")
m.add_command(label="Paste")
m.add_command(label="Reload")
m.add_separator()
m.add_command(label="Rename")

def do_popup(event):

    try:

        m.tk_popup(event.x_root, event.y_root)

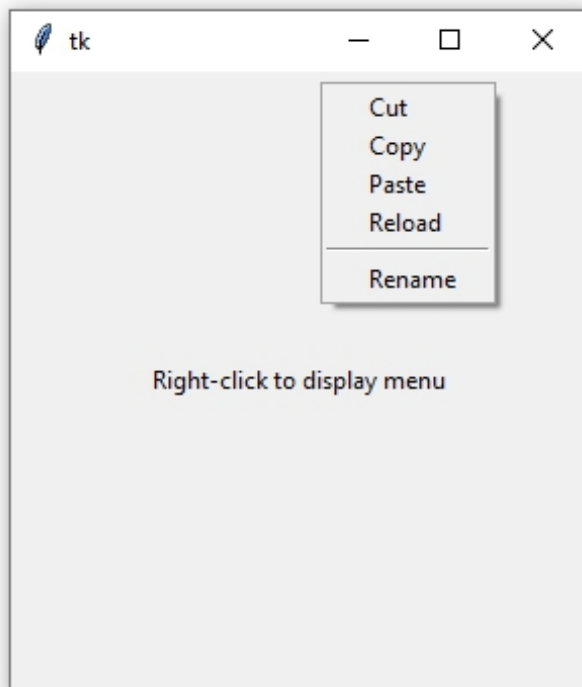
    finally:

        m.grab_release()
```

```
L.bind("<Button-3>", do_popup)
```

```
mainloop()
```

Output



Working with Images in Tkinter=====

CHAPTER 4: Reading Images With Python – Tkinter

There are numerous tools for designing GUI (Graphical User Interface) in Python such as tkinter, wxPython, JPython, etc where Tkinter is the standard

Python GUI library, it provides a simple and efficient way to create GUI applications in Python.

Reading Images With Tkinter

In order to do various operations and manipulations on images, we require Python Pillow package. If the Pillow package is not present in the system then it can be installed using the below command.

- In Command prompt:

```
pip install Pillow
```

- In Anaconda prompt:

```
conda install -c anaconda pillow
```

Example 1: The below program demonstrates how to read images with tkinter using PIL.

```
# importing required packages

import tkinter

from PIL import ImageTk, Image

import os


# creating main window

root = tkinter.Tk()


# loading the image

img = ImageTk.PhotoImage(Image.open("gfg.jpeg"))


# reading the image
```

```
panel = tkinter.Label(root, image = img)

# setting the application

panel.pack(side = "bottom", fill = "both",

           expand = "yes")

# running the application

root.mainloop()
```

Output:



In the above program, an image is loaded using the `PhotoImage()` method and then it is read by using the `Label()` method. The `pack()` method arranges the main window and the `mainloop()` function is used to run the application in an infinite loop.

Example 2: Let us look at another example where we arrange the image parameters along with application parameters.


```
# importing required packages

import tkinter

from PIL import ImageTk, Image

# creating main window

root = tkinter.Tk()

# arranging application parameters

canvas = tkinter.Canvas(root, width = 500,
                        height = 250)

canvas.pack()

# loading the image

img = ImageTk.PhotoImage(Image.open("gfg.ppm"))

# arranging image parameters

# in the application

canvas.create_image(135, 20, anchor = NW,
                  image = img)

# running the application
```

```
root.mainloop()
```

Output:



In the above program, the application parameters are handled by using the `Canvas()` method and the image parameters are handled using `create_image()` method such that the image `gfg.ppm` is displayed in the main window having defined height and width.

Note: The Canvas method `create_image(x0,y0, options ...)` is used to draw an image on a canvas. `create_image` doesn't accept an image directly. It uses an object which is created by the `PhotoImage()` method. The `PhotoImage` class can only read GIF and PGM/PPM images from files.

CHAPTER 5: `iconphoto()` method in Tkinter

iconphoto() method is used to set the titlebar icon of any tkinter/toplevel window. But to set any image as the icon of titlebar, image should be the object of **PhotoImage** class.

Syntax:

```
iconphoto(self, default = False, *args)
```

Steps to set icon image –

```
from tkinter import Tk

master = Tk()

photo = PhotoImage(file = "Any image file")

master.iconphoto(False, photo)
```

Set the titlebar icon for this window based on the named photo images passed through args. If default is *True*, this is applied to all future created toplevels as well. The data in the images is taken as a snapshot at the time of invocation. If the images are later changed, this is not reflected in the *titlebar* icons. The function also scales provided icons to an appropriate size.

Code #1: When PhotoImage is provided.

- Python3

```
# Importing Tkinter module

from tkinter import *

from tkinter.ttk import *

# Creating master Tkinter window

master = Tk()
```

```
# Creating object of photoimage class

# Image should be in the same folder
# in which script is saved

p1 = PhotoImage(file = 'info.png')


# Setting icon of master window

master.iconphoto(False, p1)


# Creating button

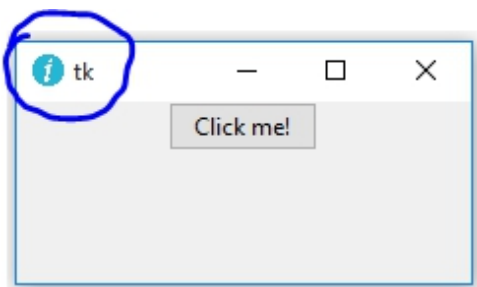
b = Button(master, text = 'Click me !')

b.pack(side = TOP)


# Infinite loop can be terminated by
# keyboard or mouse interrupt
# or by any predefined function (destroy())

mainloop()
```

Output:



Exception: If you provide an image directly instead of PhotoImage object then it will show the following error.

Code #2: When PhotoImage object is not provided.

- Python3

```
# Importing Tkinter module

from tkinter import *

from tkinter.ttk import *

# Creating master Tkinter window

master = Tk()

# Setting icon of master window

master.iconphoto(False, 'info.png')

# Creating button

b = Button(master, text = 'Click me !')

b.pack(side = TOP)

# Infinite loop can be terminated by
# keyboard or mouse interrupt
# or by any predefined function (destroy())

mainloop()
```

Output:

Traceback (most recent call last):

File "C:\Users\Admin\Documents\GUI_python\geeks.py", line 14, in

master.iconphoto(False, 'info.png')

File "C:\Users\Admin\AppData\Local\Programs\Python\Python37-32\lib\tkinter__init__.py", line 1910, in wm_iconphoto

self.tk.call('wm', 'iconphoto', self._w, *args)

_tkinter.TclError: can't use "info.png" as iconphoto: not a photo image

CHAPTER 6: Loading Images in Tkinter using PIL

In this article, we will learn how to load images from user system to Tkinter window using PIL module. This program will open a dialogue box to select the required file from any directory and display it in the tkinter window.

Install the requirements –

Use this command to install Tkinter :

pip install python-tk

Use this command to install PIL :

pip install pillow

Importing modules –

- Python3

```
from tkinter import *
```

```
# loading Python Imaging Library
```

```
from PIL import ImageTk, Image

# To get the dialog box to open when required

from tkinter import filedialog
```

Note: The ImageTk module contains support to create and modify Tkinter BitmapImage and PhotoImage objects from PIL images and filedialog is used for the dialog box to appear when you are opening file from anywhere in your system or saving your file in a particular position or place.

Function to create a Tkinter window consisting of a button –

- Python3

```
# Create a window

root = Tk()

# Set Title as Image Loader
root.title("Image Loader")

# Set the resolution of window
root.geometry("550x300 + 300 + 150")

# Allow Window to be resizable
root.resizable(width = True, height = True)
```

```
# Create a button and place it into the window using grid layout

btn = Button(root, text='open image', command = open_img).grid(

                                row = 1, columnspan = 4)

root.mainloop()
```

The Button object is created with text 'open image'. On clicking it the open_image function will be invoked.

Function to place the image onto the window –

- Python3

```
def open_img():

    # Select the Imagename from a folder

    x = openfilename()

    # opens the image

    img = Image.open(x)

    # resize the image and apply a high-quality down sampling filter

    img = img.resize((250, 250), Image.ANTIALIAS)

    # PhotoImage class is used to add image to widgets, icons etc

    img = ImageTk.PhotoImage(img)
```



```
# create a label

panel = Label(root, image = img)


# set the image as img

panel.image = img

panel.grid(row = 2)
```

The openfilename function will return the file name of image.

Function to return the file name chosen from a dialog box –

- Python3

```
def openfilename():

    # open file dialog box to select image

    # The dialogue box has a title "Open"

    filename = filedialog.askopenfilename(title ="pen')

    return filename
```

To run this code, save it by the extension .py and then open cmd (command prompt) and move to the location of the file saved and then write the following –

```
python "filename".py
```

and press enter and it will run. Or can be run directly by simply double-clicking your .py extension file.

Output:

<https://media.geeksforgeeks.org/wp-content/uploads/20191121233525/Screencast-from-Thursday-21-November-2019-111137-IST2.webm>

PART 6: Applications and Projects

CHAPTER 1: Simple GUI calculator using Tkinter

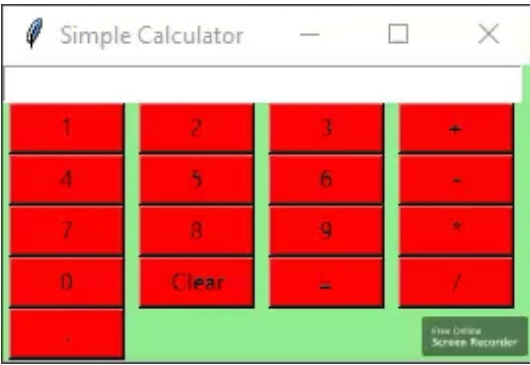
Prerequisite: [Tkinter Introduction](#), [lambda function](#)

Python offers multiple options for developing a GUI (Graphical User Interface). Out of all the GUI methods, Tkinter is the most commonly used method. It is a standard Python interface to the Tk GUI toolkit shipped with Python. Python with Tkinter outputs the fastest and easiest way to create GUI applications. Creating a GUI using Tkinter is an easy task.

To create a Tkinter:

1. Importing the module – tkinter
2. Create the main window (container)
3. Add any number of widgets to the main window
4. Apply the event Trigger on the widgets.

Below is what the GUI looks like:



Let's create a GUI-based simple calculator using the Python Tkinter module, which can perform basic arithmetic operations addition, subtraction, multiplication, and division.

Below is the implementation:

- Python3

```
# Python program to create a simple GUI
# calculator using Tkinter

# import everything from tkinter module
from tkinter import *

# globally declare the expression variable
expression = ""

# Function to update expression
# in the text entry box
def press(num):
```

```
# point out the global expression variable
```

```
    global expression
```

```
# concatenation of string
```

```
    expression = expression + str(num)
```

```
# update the expression by using set method
```

```
equation.set(expression)
```

```
# Function to evaluate the final expression
```

```
def equalpress():
```

```
    # Try and except statement is used
```

```
    # for handling the errors like zero
```

```
    # division error etc.
```

```
    # Put that code inside the try block
```

```
    # which may generate the error
```

```
    try:
```

```
        global expression
```

```
        # eval function evaluate the expression
```

```
        # and str function convert the result
```

```
        # into string
```

```

        total = str(eval(expression))

equation.set(total)

# initialize the expression variable
# by empty string

expression = ""

# if error is generate then handle
# by the except block
except:

    equation.set(" error ")

    expression = ""

# Function to clear the contents
# of text entry box

def clear():

    global expression

    expression = ""

    equation.set("")

# Driver code

```

```
if __name__ == "__main__":  
    # create a GUI window  
  
    gui = Tk()  
  
    # set the background colour of GUI window  
    gui.configure(background="light green")  
  
    # set the title of GUI window  
    gui.title("Simple Calculator")  
  
    # set the configuration of GUI window  
    gui.geometry("270x150")  
  
    # StringVar() is the variable class  
    # we create an instance of this class  
  
    equation = StringVar()  
  
    # create the text entry box for  
    # showing the expression .  
  
    expression_field = Entry(gui, textvariable=equation)  
  
    # grid method is used for placing  
    # the widgets at respective positions  
    # in table like structure .  
  
    expression_field.grid(columnspan=4, ipadx=70)
```

```
# create a Buttons and place at a particular
# location inside the root window .
# when user press the button, the command or
# function affiliated to that button is executed .

button1 = Button(gui, text=' 1 ', fg='black', bg='red',
                  command=lambda: press(1), height=1, width=7)
button1.grid(row=2, column=0)

button2 = Button(gui, text=' 2 ', fg='black', bg='red',
                  command=lambda: press(2), height=1, width=7)
button2.grid(row=2, column=1)

button3 = Button(gui, text=' 3 ', fg='black', bg='red',
                  command=lambda: press(3), height=1, width=7)
button3.grid(row=2, column=2)

button4 = Button(gui, text=' 4 ', fg='black', bg='red',
                  command=lambda: press(4), height=1, width=7)
button4.grid(row=3, column=0)

button5 = Button(gui, text=' 5 ', fg='black', bg='red',
                  command=lambda: press(5), height=1, width=7)
button5.grid(row=3, column=1)

button6 = Button(gui, text=' 6 ', fg='black', bg='red',
```



```
        command=lambda: press(6), height=1, width=7)
button6.grid(row=3, column=2)
```

```
button7 = Button(gui, text=' 7 ', fg='black', bg='red',
        command=lambda: press(7), height=1, width=7)
button7.grid(row=4, column=0)
```

```
button8 = Button(gui, text=' 8 ', fg='black', bg='red',
        command=lambda: press(8), height=1, width=7)
button8.grid(row=4, column=1)
```

```
button9 = Button(gui, text=' 9 ', fg='black', bg='red',
        command=lambda: press(9), height=1, width=7)
button9.grid(row=4, column=2)
```

```
button0 = Button(gui, text=' 0 ', fg='black', bg='red',
        command=lambda: press(0), height=1, width=7)
button0.grid(row=5, column=0)
```

```
plus = Button(gui, text=' + ', fg='black', bg='red',
        command=lambda: press("+"), height=1, width=7)
plus.grid(row=2, column=3)
```

```
minus = Button(gui, text=' - ', fg='black', bg='red',
        command=lambda: press("-"), height=1, width=7)
```

```
minus.grid(row=3, column=3)
```

```
multiply = Button(gui, text=' * ', fg='black', bg='red',  
                  command=lambda: press("*"), height=1, width=7)
```

```
multiply.grid(row=4, column=3)
```

```
divide = Button(gui, text=' / ', fg='black', bg='red',  
                command=lambda: press("/"), height=1, width=7)
```

```
divide.grid(row=5, column=3)
```

```
equal = Button(gui, text=' = ', fg='black', bg='red',  
               command=equalpress, height=1, width=7)
```

```
equal.grid(row=5, column=2)
```

```
clear = Button(gui, text='Clear', fg='black', bg='red',  
               command=clear, height=1, width=7)
```

```
clear.grid(row=5, column='1')
```

```
Decimal= Button(gui, text='.', fg='black', bg='red',  
                command=lambda: press('.'), height=1, width=7)
```

```
Decimal.grid(row=6, column=0)
```

```
# start the GUI
```

```
gui.mainloop()
```

Code Explanation:

1. The code starts by importing the necessary modules.
2. The tkinter module provides all the basic functionality for creating graphical user interfaces.
3. Next, we create a global variable called expression which will store the result of the calculation.
4. We also create two functions to update and evaluate the expression.
5. Finally, we write driver code to initialize and manage our GUI window.
6. In order to create a simple calculator, we first need to define an expression variable.
7. This is done by using the global keyword and assigning it an empty string value ("").
8. Next, we create two functions to update and evaluate the expression.
9. The press function updates the contents of the text entry box while equalpress evaluates the final result of the calculation.

10.

We next need to create a table-like structure in which our widgets will be placed.

11.

We do this by using grid method which takes three arguments: colspan , padx , and rowspan .

12.

These parameters specify how many columns wide, how many rows high, and how many columns per row respectively should be used in our table layout.

13.

We set colspan to 4 , meaning that there will be four columns in our table, iPad width divided by 2 (70), multiplied by 1 for each row in our table (iPad height divided

14.

The code creates a simple calculator using the Tkinter module.

15.

First, the code imports everything from the Tkinter module.

16.

Next, the code creates two global variables: expression and total.

17.

The `press()` function is used to update the expression variable in the text entry box.

18.

The `equalpress()` function is used to evaluate the final expression.

19.

Finally, the `clear()` function is used to clear the contents of the text entry box.

20.

Next, the driver code is created.

21.

In this code, if `__name__ == "__main__"`: is executed which will create a GUI window and set its background color to light green and its title to Simple Calculator.

22.

Next, the `geometry()` method is used to set the size of the GUI window (270

23.

The code starts with a few basic objects: a `Button` object, which has properties for text, font, background color, and command; and a `grid` object.

24.

The first three buttons (`button1` through `button3`) each have their own individual commands associated with them.

25.

When the user clicks on one of these buttons, the corresponding command is executed.

26.

For example, when the user clicks on `button1`, its command is to press the number 1 key.

27.

Similarly, when the user clicks on `button2`'s command, it will be to press the number 2 key; and so on.

28.

Similarly, when the user clicks on `button4`'s command (to increase the value by 1), its `grid` row and column values will be set to 3 and 0 respectively.

29.

And finally when clicking on button5's command (to decrease the value by 1), its grid row and column values will be set to 2 and 1 respectively.

30.

The code creates seven buttons, each with its own function.

31.

When the user presses one of the buttons, the corresponding command is executed.

32.

The first button, button1, has the function press(1).

33.

When clicked, this button will execute the code lambda: press(1).

34.

The second button, button2, has the function press(2), and so on.

35.

When all seven buttons have been clicked, their functions will be executed in order.

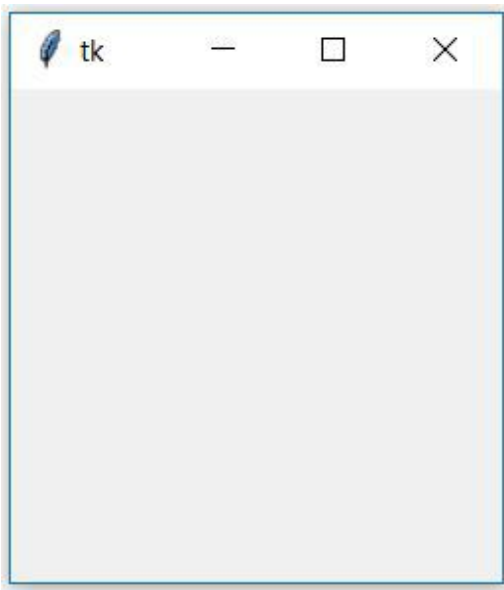
CHAPTER 2: Create a stopwatch using python

This article focus on creating a stopwatch using Tkinter in python
Tkinter : Tkinter is the standard GUI library for Python. Python when combined with Tkinter provides a fast and easy way to create GUI applications. Tkinter provides a powerful object-oriented interface to the Tk GUI toolkit. It's very easy to get started with Tkinter, here are some sample codes to get your hands on Tkinter in python.

- Python3

```
# Python program to create a  
# a new window using Tkinter  
# importing the required libraries  
  
import tkinter  
  
# creating a object 'top' as instance of class Tk  
  
top = tkinter.Tk()  
  
# This will start the blank window  
  
top.mainloop()
```

Output:



Creating Stopwatch using Tkinter

Now lets try to create a program using Tkinter module to create a stopwatch. A stopwatch is a handheld timepiece designed to measure the amount of time elapsed from a particular time when it is activated to the time when the piece is deactivated. A large digital version of a stopwatch designed for viewing at a distance, as in a sports stadium, is called a stop clock. In manual timing, the clock is started and stopped by a person pressing a button. In fully automatic time, both starting and stopping are triggered automatically, by sensors.

Required Modules: We are only going to use Tkinter for creating GUI and no other libraries will be used in this program.

Source Code:

- Python3

```
# Python program to illustrate a stop watch

# using Tkinter

#importing the required libraries

import tkinter as Tkinter

from datetime import datetime

counter = 66600

running = False

def counter_label(label):

    def count():

        if running:

            global counter

            # get current time
```

```

# To manage the initial delay.

if counter==66600:

    display="Starting..."

else:

    tt = datetime.fromtimestamp(counter)

    string = tt.strftime("%H:%M:%S")

    display=string


label['text']=display # Or label.config(text=display)


# label.after(arg1, arg2) delays by
# first argument given in milliseconds
# and then calls the function given as second argument.
# Generally like here we need to call the
# function in which it is present repeatedly.
# Delays by 1000ms=1 seconds and call count again.

label.after(1000, count)

    counter += 1


# Triggering the start of the counter.

count()


# start function of the stopwatch

```



```
def Start(label):

    global running

    running=True

    counter_label(label)

    start['state']='disabled'

    stop['state']='normal'

    reset['state']='normal'


# Stop function of the stopwatch

def Stop():

    global running

    start['state']='normal'

    stop['state']='disabled'

    reset['state']='normal'

    running = False


# Reset function of the stopwatch

def Reset(label):

    global counter

    counter=66600


# If rest is pressed after pressing stop.
```

```
    if running==False:

        reset['state']='disabled'

        label['text']='Welcome!'

    # If reset is pressed while the stopwatch is running.

    else:

        label['text']='Starting...'

root = Tkinter.Tk()

root.title("Stopwatch")

# Fixing the window size.

root.minsize(width=250, height=70)

label = Tkinter.Label(root, text="Welcome!", fg="black", font="Verdana 30 bold")

label.pack()

f = Tkinter.Frame(root)

start = Tkinter.Button(f, text='Start', width=6, command=lambda:Start(label))

stop = Tkinter.Button(f, text='Stop',width=6,state='disabled', command=Stop)

reset = Tkinter.Button(f, text='Reset',width=6, state='disabled', command=lambda:Reset(label))

f.pack(anchor = 'center',pady=5)

start.pack(side="left")

stop.pack(side ="left")
```

```
reset.pack(side="left")

root.mainloop()
```

Output:



CHAPTER 3: Real time currency converter using Tkinter

Prerequisites : Introduction to tkinter | Get the real time currency exchange rate

Python offers multiple options for developing GUI (Graphical User Interface). Out of all the GUI methods, tkinter is the most commonly used method. It is a standard Python interface to the Tk GUI toolkit shipped with Python. Python with tkinter outputs the fastest and easiest way to create the GUI applications.

To create a tkinter :

- Importing the module – tkinter
- Create the main window (container)
- Add any number of widgets to the main window.
- Apply the event Trigger on the widgets.

Let's create a GUI based simple real-time currency converter (Using Alpha Vantage API) which can convert amounts from one currency to another currency.

Requirements:

tkinter

requests

json

Note: You need an API key to use the Alpha Vantage API.

Complete code:

Python3

```
# import all functions from the tkinter

from tkinter import *

# Create a GUI window

root = Tk()
```

```
# create a global variables

variable1 = StringVar(root)

variable2 = StringVar(root)


# initialise the variables

variable1.set("currency")

variable2.set("currency")


# Function to perform real time conversion

# from one currency to another currency

def RealTimeCurrencyConversion():


    # importing required libraries

    import requests, json


    # currency code

    from_currency = variable1.get()

    to_currency = variable2.get()


    # enter your api key here

    api_key = "Your_Api_Key"
```

```
# base_url variable store base url

base_url = r"https://www.alphavantage.co/query?function =
CURRENCY_EXCHANGE_RATE"

# main_url variable store complete url

main_url = base_url + "&from_currency =" + from_currency +

    "&to_currency =" + to_currency + "&apikey =" + api_key

# get method of requests module

# return response object

req_ob = requests.get(main_url)

# json method converts json data type

#into python dictionary data type

# result contains list of nested dictionaries

result = req_ob.json()

# parsing the required information

Exchange_Rate = float(result["Realtime Currency Exchange Rate"]

                        ['5. Exchange Rate'])

# get method of Entry widget
```

```
# returns current text as a
# string from text entry box.

amount = float(Amount1_field.get())

# calculation for the conversion

new_amount = round(amount * Exchange_Rate, 3)

# insert method inserting the
# value in the text entry box.

Amount2_field.insert(0, str(new_amount))


# Function for clearing the Entry field

def clear_all():

    Amount1_field.delete(0, END)

    Amount2_field.delete(0, END)


# Driver code

if __name__ == "__main__" :

    # Set the background colour of GUI window

    root.configure(background = 'light green')
```

```
# Set the configuration of GUI window (WidthxHeight)

root.geometry("400x175")


# Create "welcome to Real Time Currency Convertor" label

headlabel = Label(root, text = 'welcome to Real Time Currency Convertor',

                    fg = 'black', bg = 'red')


# Create a "Amount :" label

label1 = Label(root, text = "Amount :",

                fg = 'black', bg = 'dark green')


# Create a "From Currency :" label

label2 = Label(root, text = "From Currency",

                fg = 'black', bg = 'dark green')


# Create a "To Currency: " label

label3 = Label(root, text = "To Currency :",

                fg = 'black', bg = 'dark green')


# Create a "Converted Amount :" label

label4 = Label(root, text = "Converted Amount :",

                fg = 'black', bg = 'dark green')
```



```
# grid method is used for placing
# the widgets at respective positions
# in table like structure.

headlabel.grid(row = 0, column = 1)

label1.grid(row = 1, column = 0)

label2.grid(row = 2, column = 0)

label3.grid(row = 3, column = 0)

label4.grid(row = 5, column = 0)


# Create a text entry box
# for filling or typing the information.

Amount1_field = Entry(root)

Amount2_field = Entry(root)


# padx keyword argument set width of entry space.

Amount1_field.grid(row = 1, column = 1, padx = "25")

Amount2_field.grid(row = 5, column = 1, padx = "25")


# list of currency codes

CurrencyCode_list = ["INR", "USD", "CAD", "CNY", "DKK", "EUR"]


# create a drop down menu using OptionMenu function
```

```
# which takes window name, variable and choices as

# an argument. use * before the name of the list,

# to unpack the values

FromCurrency_option = OptionMenu(root, variable1, *CurrencyCode_list)

ToCurrency_option = OptionMenu(root, variable2, *CurrencyCode_list)


FromCurrency_option.grid(row = 2, column = 1, ipadx = 10)

ToCurrency_option.grid(row = 3, column = 1, ipadx = 10)


# Create a Convert Button and attach

# with RealTimeCurrencyExchangeRate function

button1 = Button(root, text = "Convert", bg = "red", fg = "black",

                  command = RealTimeCurrencyConversion)


button1.grid(row = 4, column = 1)


# Create a Clear Button and attach

# with delete function

button2 = Button(root, text = "Clear", bg = "red",

                  fg = "black", command = clear_all)


button2.grid(row = 6, column = 1)
```

```
# Start the GUI
```

```
root.mainloop()
```

Output :



CHAPTER 4: Create Table Using Tkinter

Python offers multiple options for developing a GUI (Graphical User Interface). Out of all the GUI methods, **Tkinter** is the most commonly used method. It is a standard Python interface to the Tk GUI toolkit shipped with Python. Python with Tkinter is the fastest and easiest way to create GUI applications. Creating a GUI using Tkinter is an easy task.

Note: For more information, refer to [Python GUI – tkinter](#)

Creating Tables Using Tkinter

A table is useful to display data in the form of rows and columns. Unfortunately, Tkinter does not provide a Table widget to create a table. But we can create a table using alternate methods. For example, we can make a table by repeatedly displaying entry widgets in the form of rows and columns.

To create a table with five rows and four columns we can use two for loops as:

```
for i in range(5):  
    for j in range(4):
```

Inside these loops, we have to create an Entry widget by creating an object of Entry class, as:

```
e = Entry(root, width=20, fg='blue', font=('Arial', 16, 'bold'))
```

Now, we need logic to place this Entry widget in rows and columns. This can be done by using grid() method to which we can pass row and column positions, as:

```
# here i and j indicate  
# row and column positions
```

```
e.grid(row=i, column=j)
```

We can insert data into the Entry widget using insert() method, as:

```
e.insert(END, data)
```

Here, 'END' indicates that the data continuous to append at the end of previous data in the Entry widget.

This is the logic that is used in the program given below using the data that is coming from a list. We have taken a list containing 5 tuples and each tuple contains four values which indicate student id, name, city and age.

Hence, we will have a table with 5 rows and 4 columns in each row. This program can also be applied on data coming from a database to display the entire data in the form of a table.

Source Code:

- Python3

```
# Python program to create a table
```

```
from tkinter import *
```

```
class Table:
```

```
    def __init__(self,root):
```

```
        # code for creating table
```

```
        for i in range(total_rows):
```

```
        for j in range(total_columns):

            self.e = Entry(root, width=20, fg='blue',
                           font=('Arial',16,'bold'))

            self.e.grid(row=i, column=j)

            self.e.insert(END, lst[i][j])

# take the data
lst = [(1,'Raj','Mumbai',19),
       (2,'Aaryan','Pune',18),
       (3,'Vaishnavi','Mumbai',20),
       (4,'Rachna','Mumbai',21),
       (5,'Shubham','Delhi',21)]

# find total number of rows and
# columns in list

total_rows = len(lst)

total_columns = len(lst[0])

# create root window

root = Tk()

t = Table(root)

root.mainloop()
```

Output:



1	Raj	Mumbai	19
2	Aaryan	Pune	18
3	Vaishnavi	Mumbai	20
4	Rachna	Mumbai	21
5	Shubham	Delhi	21

CHAPTER 5: GUI Calendar using Tkinter

Prerequisites: [Introduction to](#) Tkinter

Python offers multiple options for developing a GUI (Graphical User Interface). Out of all the GUI methods, Tkinter is the most commonly used method. Python with Tkinter outputs the fastest and easiest way to create GUI applications. In this article, we will learn how to create a GUI Calendar application using Tkinter, with a step-by-step guide.

To create a Tkinter:

- Importing the module – Tkinter
- Create the main window (container)
- Add any number of widgets to the main window.
- Apply the event Trigger on the widgets.

The GUI would look like below:



Let's create a GUI-based Calendar application that can show the calendar with respect to the given year, given by the user.

Below is the implementation:

Python3

```
# import all methods and classes from the tkinter

from tkinter import *

# import calendar module

import calendar

# Function for showing the calendar of the given year

def showCal() :
```



```
# Create a GUI window

new_gui = Tk()

# Set the background colour of GUI window

new_gui.config(background = "white")

# set the name of tkinter GUI window

new_gui.title("CALENDAR")

# Set the configuration of GUI window

new_gui.geometry("550x600")

# get method returns current text as string

fetch_year = int(year_field.get())

# calendar method of calendar module return

# the calendar of the given year .

cal_content = calendar.calendar(fetch_year)

# Create a label for showing the content of the calendar

cal_year = Label(new_gui, text = cal_content, font = "Consolas 10 bold")

# grid method is used for placing

# the widgets at respective positions

# in table like structure.
```

```
cal_year.grid(row = 5, column = 1, padx = 20)
```

```
# start the GUI
```

```
new_gui.mainloop()
```

```
# Driver Code
```

```
if __name__ == "__main__" :
```

```
    # Create a GUI window
```

```
    gui = Tk()
```

```
    # Set the background colour of GUI window
```

```
    gui.config(background = "white")
```

```
    # set the name of tkinter GUI window
```

```
    gui.title("CALENDAR")
```

```
    # Set the configuration of GUI window
```

```
    gui.geometry("250x140")
```

```
    # Create a CALENDAR : label with specified font and size
```

```
    cal = Label(gui, text = "CALENDAR", bg = "dark gray",
```

```
                font = ("times", 28, 'bold'))
```

```
# Create a Enter Year : label
```

```
year = Label(gui, text = "Enter Year", bg = "light green")
```

```
# Create a text entry box for filling or typing the information.
```

```
year_field = Entry(gui)
```

```
# Create a Show Calendar Button and attached to showCal function
```

```
Show = Button(gui, text = "Show Calendar", fg = "Black",
```

```
                bg = "Red", command = showCal)
```

```
# Create a Exit Button and attached to exit function
```

```
Exit = Button(gui, text = "Exit", fg = "Black", bg = "Red", command = exit)
```

```
# grid method is used for placing
```

```
# the widgets at respective positions
```

```
# in table like structure.
```

```
cal.grid(row = 1, column = 1)
```

```
year.grid(row = 2, column = 1)
```

```
year_field.grid(row = 3, column = 1)
```

```
Show.grid(row = 4, column = 1)
```

```
Exit.grid(row = 6, column = 1)
```

```
# start the GUI
```

```
gui.mainloop()
```

Code Explanation:

1. The code starts by creating a new Tkinter window.
2. The window has a white background and the title “CALENDAR”.
3. Next, the window’s geometry is set to be 550×600 pixels.
4. The calendar module is imported and the showCal() function is defined.
5. This function will be used to display the calendar for the given year.
6. The showCal() function first creates a GUI window and sets its background color to white.
7. It also sets the title of the GUI window to “CALENDAR”.
8. Finally, it sets the configuration of the GUI window.
9. Next, two labels are created: CALENDAR and ENTER YEAR.
 10. CALENDAR’s text is set to be “CALENDAR”, its background color is changed from white to dark gray, and its font size is set to 28 points bold.
 11. ENTER YEAR’s text is set to be “Enter Year”, its background color is changed from light green to red, and its font size is set to 10 points bold.
 12. A text entry box named year_field is created next and assigned as an attribute of year_field object instance .
 13. Finally, a Show Calendar button (with appropriate attributes) and an Exit button (with appropriate attributes) are created and
 - 14.

The code creates a window and sets its background color to white.
15.

The window's title is set to "CALENDAR" and its geometry is set to 550×600.

16.

The calendar module is imported and the method `calendar.calendar()` is called with the given year as an argument.

17.

This returns the calendar for the given year.

18.

A label named `cal_year` is created and it has a grid with five rows and one column.

19.

The first row of the grid will be at position (5, 1), the second row at (4, 1), etc., until row 5 which will be at position (0, 0).

20.

Similarly, column 1 will be at position (0, 0), column 2 at (1, 0).

CHAPTER 6: File Explorer in Python using Tkinter

Prerequisites:

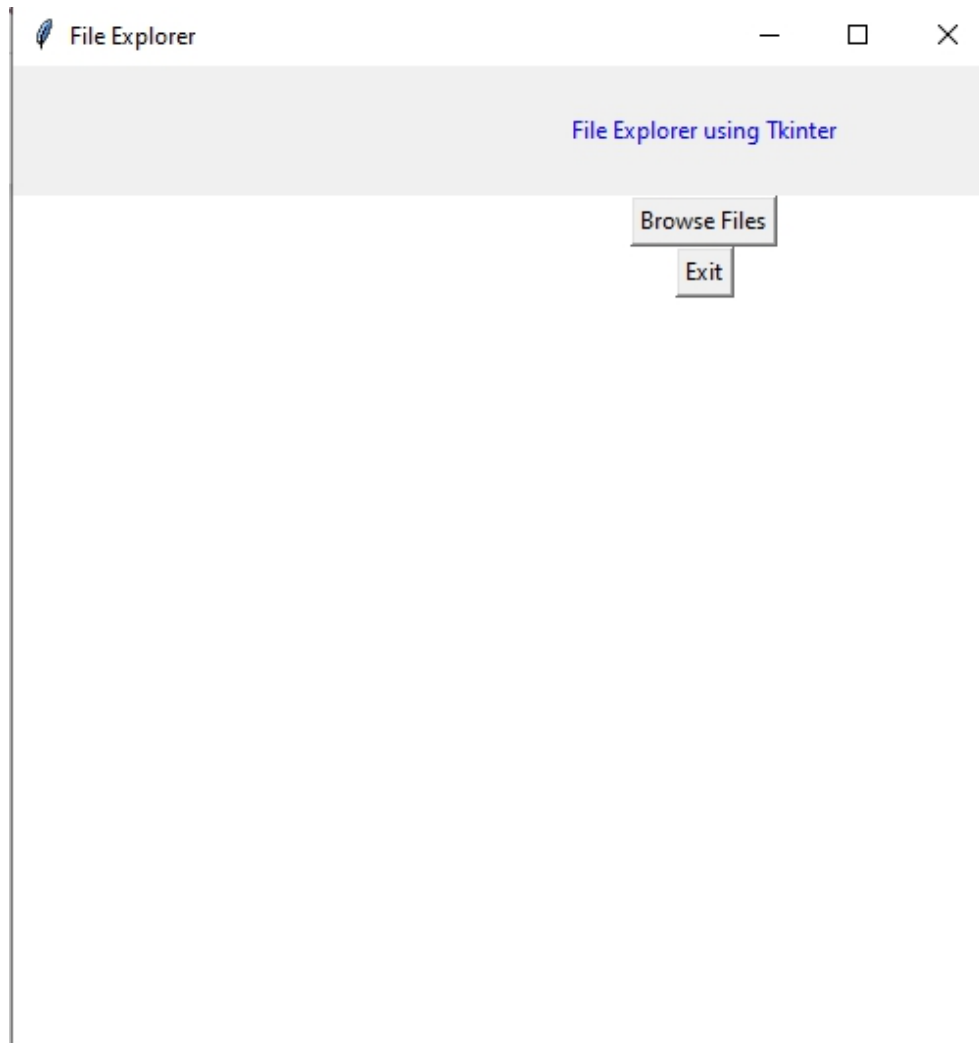
Python offers various modules to create graphics programs. Out of these Tkinter provides the fastest and easiest way to create GUI applications.

The following steps are involved in creating a tkinter application:

- Importing the Tkinter module.
- Creation of the main window (container).
- Addition of widgets to the main window

- Applying the event Trigger on widgets like buttons, etc.

The GUI would look like below:



Creating the File Explorer

In order to do so, we have to import the **filedialog** module from Tkinter. The File dialog module will help you open, save files or directories.

In order to open a file explorer, we have to use the method, `askopenfilename()`. This function creates a file dialog object.

Syntax: `tkFileDialog.askopenfilename(initialdir = "/",title = "Select file",filetypes = (("file_type", "*.extension"),("all files", "*..*")))`

Parameters:

1. ***initialdir:*** *We have to specify the path of the folder that is to be opened when the file explorer pops up.*
2. ***title:*** *The title of file explorer opened.*
3. ***filetypes:*** *Here we can specify different kinds of file extensions so that the user can filter based on different file types*

Below is the implementation

- Python3

```
# Python program to create
# a file explorer in Tkinter

# import all components
# from the tkinter library
from tkinter import *

# import filedialog module
from tkinter import filedialog

# Function for opening the
# file explorer window
def browseFiles():
```

```
filename = filedialog.askopenfilename(initialdir = "/",

                                     title = "Select a File",

                                     filetypes = (("Text files",
                                                  "*.txt*"),
                                                  ("all files",
                                                  "*.*")))

# Change label contents

label_file_explorer.configure(text="File Opened: "+filename)


# Create the root window

window = Tk()

# Set window title

window.title('File Explorer')

# Set window size

window.geometry("500x500")

#Set window background color

window.config(background = "white")
```



```
# Create a File Explorer label

label_file_explorer = Label(window,

                                text = "File Explorer using Tkinter",

                                width = 100, height = 4,

                                fg = "blue")
```

```
button_explore = Button(window,

                           text = "Browse Files",

                           command = browseFiles)
```

```
button_exit = Button(window,

                       text = "Exit",

                       command = exit)
```

```
# Grid method is chosen for placing
# the widgets at respective positions
# in a table like structure by
# specifying rows and columns
```

```
label_file_explorer.grid(column = 1, row = 1)
```

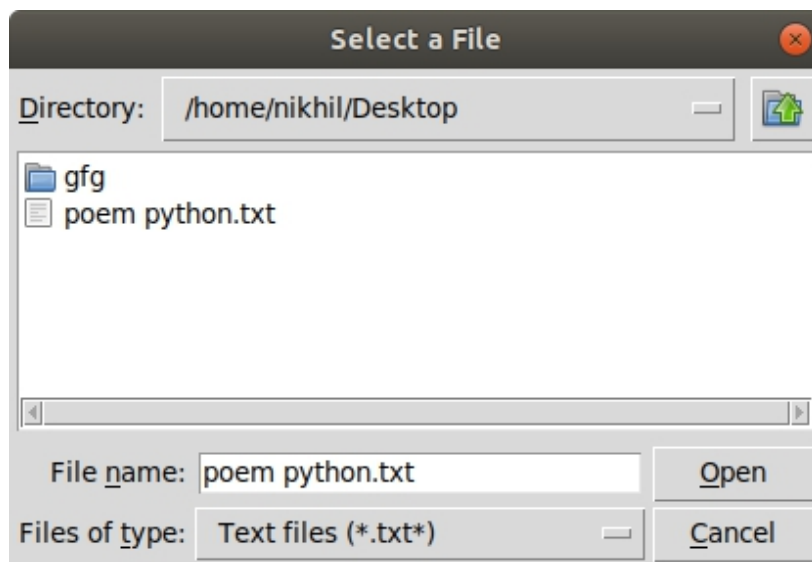
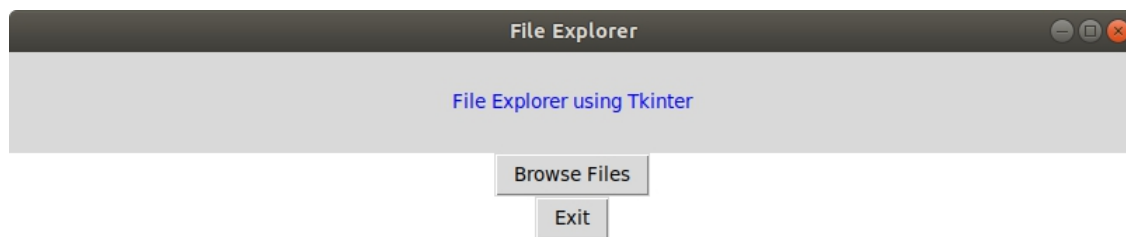
```
button_explore.grid(column = 1, row = 2)
```

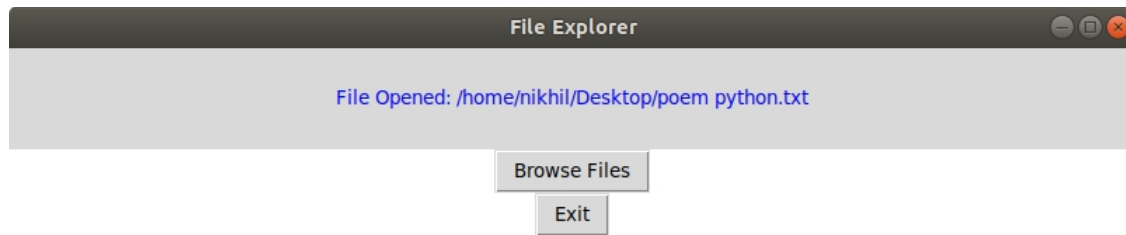
```
button_exit.grid(column = 1,row = 3)
```

```
# Let the window wait for any events
```

```
window.mainloop()
```

Output:





Conclusion

This book is dedicated to the readers who take time to write me each day. Every morning I'm greeted by various emails — some with requests, a few with complaints, and then there are the very few that just say thank you. All these emails encourage and challenge me as an author — to better both my books and myself.

Thank you!

OceanofPDF.com